

Modern Embedded C++

Modern Embedded C++

*From C structs to classes, interfaces,
templates and threads*

Erik Pihl

Modern Embedded C++
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit <https://creativecommons.org/licenses/by-nc-sa/4.0/>.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the example programs, the test suites and the solutions to the exercises:

<https://github.com/qrotech-academy/modern-embedded-cpp>

The C++ in this book is C++17 throughout, compiled with GCC. ESP32-S3, STM32 and ATmega328P appear in the text as examples of targets a driver might be written for; they are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with Lua^AT_EX.

Typeset from the six lectures of the course, their exercises, and the two self-assessment papers. This edition: 4 October 2026.

Contents

Preface	vii
1 Modern Embedded C++ Concepts	1
1.1 Compiling and Executing Code in Linux	2
1.2 From C to Modern Embedded C++	7
1.3 Review	37
1.4 Exercises	38
2 Classes	47
2.1 Classes in C++	47
2.2 Advanced Discussion on Classes	67
2.3 Review	76
2.4 Exercises	76
3 Inheritance and Interfaces	89
3.1 Inheritance in C++	89
3.2 Interfaces in C++	93
3.3 Example: An LED Interface in C++	102
3.4 Review	112
3.5 Exercises	113
4 The Factory Pattern	119
4.1 A Simple Driver Factory with Raw Pointers	119
4.2 Driver Factory with Smart Pointers (Abstract Factory)	132
4.3 Review	139
4.4 Exercises	140
5 Templates	151
5.1 Function Templates	151
5.2 Type Traits and Constraints	154
5.3 Class Templates	157
5.4 Review	165

5.5	Exercises	166
6	Multithreading and Synchronization	177
6.1	Multithreading and Synchronization in C++	178
6.2	Review	199
6.3	Exercises	200
A	The Self-Assessment Papers	213
B	Self-Assessment Paper A	217
C	Self-Assessment Paper B	229
D	Model Answers to Paper A	237
	Question 1: Crossing the line between C and C++	238
	Question 2: Object lifetime, and a driver that nearly works	240
	Question 3: Classes, and what belongs to the type	242
	Question 4: The class that owns something	243
	Question 5: Interfaces, and the keyword that is missing	245
	Question 6: The factory, and who owns what	247
	Question 7: Templates	249
	Question 8: Two threads and a shared struct	251
E	Model Answers to Paper B	255
	Question 1: The vocabulary	255
	Question 2: The same driver, twice	257
	Question 3: Enumerations, files, and lifetime	259
	Question 4: What the compiler writes for you	262
	Question 5: Designing against an abstraction	264
	Question 6: Factories	266
	Question 7: A container with its size in the type	269
	Question 8: Sharing state between threads	271

Preface

This book is for experienced C developers who want to learn how modern C++ can be used effectively in embedded systems. It covers the development of embedded systems in modern C++ (C++17), with a focus on practical design techniques for resource-constrained systems.

Unlike an introductory C++ text, it assumes strong prior experience with C, and it covers only the C++ concepts that are useful in embedded systems:

- The transition from C to modern embedded C++.
- Compile-time programming (`constexpr`, templates).
- Deterministic embedded design (`noexcept`, controlled use of dynamic memory allocation).
- Structs and classes for hardware drivers.
- Encapsulation and object-oriented design in embedded systems.
- Interfaces and polymorphism.
- Factory patterns and dependency management.
- Templates and generic programming.
- Copy and move semantics in embedded C++.
- Template utilities for low-level programming.
- Threading and synchronization primitives.
- Designing reusable and testable driver libraries.

Along the way you will write code, not just read it. You will implement simple drivers using modern C++, apply compile-time techniques to low-level programming, refactor traditional C patterns into safer C++ constructs, and design modular embedded software using interfaces and factories, always with an eye on how C++ can improve modularity and code structure *without* introducing runtime overhead. The examples and exercises stay close to typical embedded problems:

- Bit manipulation utilities.
- Register access helpers.
- Simple driver abstractions.
- Compile-time utilities for low-level code.

- Interface-based driver design.
- Template-based utilities.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Use modern C++ features effectively in embedded systems.
- Write low-level code using templates and compile-time techniques.
- Design simple hardware drivers using structs and classes.
- Use interfaces to decouple system logic from hardware implementations.
- Apply factory patterns to construct embedded components.
- Use templates to build reusable embedded utilities.
- Apply object-oriented design without sacrificing performance or determinism.
- Understand how C++ abstractions translate to machine code in embedded environments.

What you are assumed to know

The subject of this book is **the C++ that pays its way on a microcontroller**, not embedded programming in general. You are expected to arrive already comfortable with:

- C programming: functions, pointers, structs and the rest of the language.
- Bit manipulation, and the embedded programming concepts it is used for.

None of that is taught from scratch. What the book does teach from nothing is every C++ feature it uses, starting in Chapter 1 with the difference between a `.h` and a `.hpp` file.

How the book is organised

The book is typeset from a six-lecture course, and each chapter is one lecture. Its sections are the lecture's appendices, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side:

Chapter	Lecture	Topic
1	L01	Modern embedded C++ concepts
2	L02	Classes
3	L03	Inheritance and interfaces
4	L04	The factory pattern
5	L05	Templates
6	L06	Multithreading and synchronization

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold the two written papers described below.

What the exercises look like

The exercises come in **sets**. A set builds one small program, or one part of a larger one, and each exercise in a set adds a piece of it. Every exercise is labelled with its kind:

Kind	What it asks of you
CODE	Write C++, compile it, and check it with <code>make test</code> , which also compares its output with the one the exercise gives.
REFLECTION	Answer in prose: explain what your code does, or why one design is better than another.

The solutions are not in this book. They are published in the companion repository, under `lectures/LNN/appendix/solutions`, grouped the same way as the exercises, each program with the Makefile that builds and runs it, and with the answers to the reflection questions in the README beside them. Clone the repository (see *Setting up* below) and you can run a reference program beside your own. Try each exercise before you read its solution.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercise set after every chapter, each with a published solution, and the questions in every chapter's review.

Appendices [A](#) to [E](#) hold two three-hour papers with worked solutions, and they check something else: **what you can reconstruct on paper, with nothing in front of you**. Eight questions each, mixing theory with C++ you either read and repair or write from scratch, and no compiler in the room to tell you which it is.

They exist purely so that you can test your own knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter. Both papers draw on all six chapters, so sitting one partway through examines material you have not read yet.

Setting up

Everything in this book runs in a Linux terminal: on Linux itself, or on Windows through WSL. You need three things:

- **GCC and G++**, to compile the example programs.
- **Linux or WSL**, to compile and run them in a terminal environment.

- **An editor;** Visual Studio Code is the one the course uses, available at <https://code.visualstudio.com/download>.

§1.1 walks through installing WSL, Ubuntu and the compiler, and writing the Makefile every program in the book is built with. Then clone the companion repository, together with the test framework it keeps as a submodule:

```
git clone --recursive https://github.com/qrtech-academy/modern-embedded-cpp.git
```

Three commands at its root cover everything the book asks of it:

```
make build           # Build every demo and solution that has a Makefile.
make test            # Test your exercise solutions.
make format-check    # Check the formatting of every C++ file.
```

You write your exercise solutions in the repository, in `lectures/LNN/exercises`, where each lecture's README says which file goes where. `make test` then runs every lecture's tests against them: a set you have not started is reported as `SKIP`, with the file its tests are waiting for, and a set you have written is tested, including its output, which is compared with the example output in the exercise character for character. `make test SOLUTIONS=1` runs the same tests against the reference solutions.

Modern Embedded C++ Concepts

What C++ adds to C, and what it costs on a microcontroller.

IN THIS CHAPTER

- The transition from C to modern C++ for embedded systems.
- Namespaces.
- `constexpr` and compile-time constants.
- `noexcept` and exception handling in embedded systems.
- Default arguments.
- Modern C++ structs: member functions, constructor and destructor, and encapsulation.
- References.
- Function templates and parameter packs.
- `[[nodiscard]]` and `std::optional`.

This chapter introduces the modern C++ language features most commonly used in embedded systems, and shows how C++ can improve the structure and safety of embedded software. You will see how a modern C++ struct groups data and behavior in a single type, and how a simple driver can be implemented as one, with member functions, a constructor and destructor, and encapsulation. You will see how references simplify function interfaces compared to pointers, and how templates enable generic utilities such as bit manipulation helpers, which is also your first intuition for compile-time programming with `constexpr` and templates.

Work the chapter in order. §1.1 installs WSL and the GCC compiler and shows how to build a program with `make`; do that first, because every program in the book is built the same way. §1.2 is the substance of the chapter, and the exercises in §1.4 are where you write the code yourself. Their solutions are in the companion repository.

1.1 Compiling and Executing Code in Linux

If you are using a Linux environment, you can use `make` to compile and execute your code. This section describes how to install **WSL** in order to build and run code in a Linux environment on a Windows computer.

1.1.1 Installation

WSL (*Windows Subsystem for Linux*) allows running a Linux distribution in a terminal environment directly inside Windows, without using a virtual machine. This book uses the Linux distribution **Ubuntu**.

WSL is used here to:

- Build, store, and run code examples.
- Compile and execute C and C++ programs.
- Work in a Linux environment using standard development tools such as `gcc`, `g++`, `make`, and `cmake`.

Installing WSL and Ubuntu

1. Open **Windows PowerShell as Administrator**.
2. Run the following commands:

```
wsl --install  
wsl --set-default-version 2
```

Alternatively, Ubuntu can be installed directly using:

```
wsl --install -d Ubuntu
```

Note: If the system asks for a restart, restart the computer before continuing. Verify the installation with:

```
wsl --status
```

Ensure that **Default Version: 2** is displayed.

If Ubuntu was not installed automatically

1. Open **Microsoft Store**.
2. Search for Ubuntu.
3. Select the latest LTS version (currently Ubuntu 24.04 LTS).
4. Click **Install**.

Starting Ubuntu

1. Type Ubuntu in the Windows search bar.
2. Launch the distribution.

At the first startup:

- Choose a short and clear username (for example your first name).
- Choose a password.
- Store the password safely.

Note: The password is used only inside the Linux environment.

Installing basic development tools

After Ubuntu has started, update the system and install the necessary tools:

```
sudo apt -y update
sudo apt -y upgrade
sudo apt -y install build-essential cmake
```

This installs:

- gcc and g++.
- make.
- Various development libraries.
- cmake.

Verify the installation:

```
gcc --version
g++ --version
```

1.1.2 Creating a new program

For each program, create a new directory, for example `example-dir`:

```
mkdir example-dir
cd example-dir
```

Inside this directory, create a file called `Makefile`:

```
touch Makefile
```

1.1.3 A simple Makefile

In this `Makefile`, add the following content:

```
# Build and run the application as default.
default: build run

# Build the application.
build:
    @g++ main.cpp -o main -Wall -Werror -std=c++17

# Run the application.
run:
    @./main

# Clean the application.
clean:
    @rm -f main
```

Note:

- The indentation under the targets `build`, `run`, and `clean` must consist of **tabs**, not spaces.
- This Makefile builds an executable named `main`.
- Regarding the compiler flags:
 - `-Wall` enables most compiler warnings.
 - `-Werror` converts warnings into errors, which helps prevent subtle bugs.
 - `-std=c++17` sets the C++ version to C++17.
- To see the compilation commands executed by `make`, remove the `@` prefix.

1.1.4 A simple Makefile with parameters

It is also possible to use parameters to make the Makefile easier to maintain, especially when the number of source files increases:

```
# Target application.
TARGET := main

# C++ compiler.
CXX_COMPILER := g++

# C++ compiler flags.
CXX_FLAGS := -Wall -Werror -std=c++17

# Source files.
SRC_FILES := main.cpp

# Build and run the application as default.
```

```

default: build run

# Build the application.
build:
    @$(CXX_COMPILER) $(SRC_FILES) -o $(TARGET) $(CXX_FLAGS)

# Run the application.
run:
    @./$(TARGET)

# Clean the application.
clean:
    @rm -f $(TARGET)

```

1.1.5 Tips

For compilation to work correctly:

- Place all header and source files (.hpp and .cpp files) directly in this directory.
- Make sure to include all source files (.cpp files) between g++ and -o in the build target.
- For simple examples, all source files can be placed in the same directory.

Embedded project structure

In larger embedded projects, source files are typically organized in separate directories such as:

```

include/
  driver/
    gpio.hpp
source/
  driver/
    gpio.cpp
  main.cpp
Makefile

```

The Makefile can then be extended to support these directories:

- The include directory is included via the compiler flags.
- The source files are listed one per line using line continuation (\).

```

# Target application.
TARGET := main

# C++ compiler.

```

```

CXX_COMPILER := g++

# C++ compiler flags.
CXX_FLAGS := -Wall -Werror -std=c++17 -Iinclude

# Source files.
SRC_FILES := source/driver/gpio.cpp \
             source/main.cpp \

# Build and run the application as default.
default: build run

# Build the application.
build:
    @$(CXX_COMPILER) $(SRC_FILES) -o $(TARGET) $(CXX_FLAGS)

# Run the application.
run:
    @./$(TARGET)

# Clean the application.
clean:
    @rm -f $(TARGET)

```

1.1.6 Compiling a test program

You can compile and run your program using the following command:

```
make
```

You can also build the program without running it:

```
make build
```

You can run the program without compiling first:

```
make run
```

You can remove compiled files:

```
make clean
```

Create and run the following test program in a file named `main.cpp`:

```
#include <cstdio>

int main()
{
    std::printf("Hello to C++!\n");
    return 0;
}
```

Run the program using:

```
make
```

Expected output:

```
Hello to C++!
```

1.1.7 Notes

- We begin by including the standard header `<cstdio>`, which is the C++ version of the C header `<stdio.h>` and contains functionality for input and output operations.
- Most functionality in the C++ standard library exists inside the namespace `std` (*standard*).
- A namespace works somewhat like a directory in a filesystem and helps prevent naming conflicts.
- When using something from the standard library we therefore prefix it with `std::`, for example:
 - `std::uint8_t`
 - `std::size_t`
 - `std::vector`
- `std::printf` is the C++ version of the C function `printf` and is used to print text to the terminal.

1.2 From C to Modern Embedded C++

1.2.1 Why use C++ in embedded systems?

Embedded systems increasingly use C++ because it enables:

- Better abstraction (via classes and interfaces).
- Stronger type safety through stricter compile-time checks.
- Compile-time guarantees (e.g. `static_assert`, templates).
- Improved modularity and separation of concerns.

- Reusable and testable drivers.

Importantly, C++ provides these benefits while still allowing low-level hardware access and zero-cost abstractions.

Unlike many desktop C++ applications, embedded systems often:

- Disable exceptions (and mark functions `noexcept`).
- Avoid dynamic allocation (`malloc/free` or `new/delete`), or limit it to system initialization.
- Prefer deterministic execution, meaning that the execution time of critical code paths must be predictable and bounded.

1.2.2 Key differences between C and C++

Feature	C	C++
Namespaces	No	Yes
Classes	No	Yes
Templates	No	Yes
References	No	Yes
Compile-time polymorphism	No	Yes
Standard containers	Limited	Rich STL
RAII	No	Yes

Note: RAII (*Resource Acquisition Is Initialization*) means that a resource is acquired when an object is created and automatically released when the object goes out of scope.

1.2.3 .h vs .hpp

In C and C++, header files are conventionally given the extension `.h`. In C++-only code, the extension `.hpp` is also commonly used.

From the compiler's perspective, the extension makes no difference:

- `#include` works identically regardless of the file extension.
- The compiler does not use the extension to decide whether to compile a file as C or C++; that is determined by the compiler invocation (e.g. `gcc` vs `g++`) or by the extension of the *source* file that includes the header.

Even though the choice has no effect on the generated code, the extension is still useful as a convention:

- `.h` is shared between C and C++, so a `.h` file could be valid C, valid C++, or both — the extension alone does not tell you which.
- `.hpp` immediately signals that a header uses C++-only features, such as classes, templates, namespaces, or references, and therefore requires a C++ compiler.

This is especially useful in a book like this one, where paired C and C++ implementations of the same driver are sometimes shown side by side (see e.g. the `cpp_interface` example in

§3.3, and the C interface demo from the embedded C course that Chapter 3 links to). Using .hpp for the C++ version removes any doubt about which language a given header belongs to.

Note: In this book, header files use the .hpp extension.

1.2.4 extern "C" and #ifdef __cplusplus

C and C++ have different linkage rules. C++ supports function overloading, so the compiler *mangles* function names to encode parameter types, allowing multiple functions with the same name but different signatures to coexist. C has no overloading and does not mangle names.

This becomes a problem when C++ code needs to call a function compiled by a C compiler (or vice versa): if one side expects a mangled name and the other exports an unmangled one, the linker cannot find a matching symbol.

extern "C" tells the C++ compiler to use C linkage for the enclosed declarations, disabling name mangling for them:

```
extern "C" void gpioInit();
```

Multiple declarations can be wrapped in a block:

```
extern "C"
{
    void gpioInit();
    void gpioWrite(std::uint8_t pin, bool state);
}
```

Important: extern "C" only affects linkage/name mangling. It does not make C++-only features usable from C: a function with C linkage cannot be overloaded or be a template, and a declaration that a C compiler is also going to read must be valid C (plain functions and data, no classes, templates, overloading, or references).

#ifdef __cplusplus

extern "C" is C++-only syntax; a C compiler does not understand it and will fail to compile a header containing it. Since headers are often shared between C and C++ (see §1.2.3), they are usually guarded with the predefined macro __cplusplus, which is only defined when the file is compiled by a C++ compiler:

```
#ifdef __cplusplus
extern "C" {
#endif
```

```

void gpioInit();
void gpioWrite(uint8_t pin, bool state);

#ifdef __cplusplus
}
#endif

```

- When compiled as C++, `__cplusplus` is defined, so the declarations are wrapped in extern "C".
- When compiled as C, `__cplusplus` is undefined, the preprocessor strips out both `#ifdef` blocks entirely, and the C compiler only sees the plain declarations.

This pattern is common in embedded systems, where a hardware vendor typically ships C headers and libraries (register definitions, HAL functions), while the application itself is written in C++. Wrapping the vendor's C declarations this way lets C++ application code link against the C library without modifying the vendor's code.

Note: A header written this way is meant to be compiled as either C or C++, which is exactly the kind of header that keeps the `.h` extension described in §1.2.3 — `.hpp` would incorrectly imply it is C++-only.

1.2.5 Namespaces

Namespaces are used to avoid symbol collisions in large software systems. They can be compared to folders on a computer:

- Just as different folders can contain files with the same name, different namespaces can contain functions, classes, variables, and other entities with the same name.
- For example, two different namespaces may both contain a function called `init()` without causing a conflict.

A namespace is defined using the `namespace` keyword, followed by a name and a block of code enclosed in braces.

Below is a simple example of a namespace named `driver`:

```

namespace driver
{

} // namespace driver

```

Anything declared inside this block becomes part of the `driver` namespace.

For example, the namespace may contain a function called `init()`:

```

namespace driver
{

```

```

void init()
{
    // Driver initialization code.
}
} // namespace driver

```

To call this function from outside the namespace, we prefix it with the namespace name followed by the scope resolution operator `::`:

```

int main()
{
    driver::init();
    return 0;
}

```

Note:

- Most functionality from the C++ standard library is accessed through namespace `std`.
- Therefore, it is common to write `std::` before standard library types and functions, for example `std::uint8_t`, `std::uint16_t`, `std::size_t`, and `std::printf`.
- C library functionality is also available in C++, but it is usually included through C++ headers such as `<cstdint>`, `<cstdio>`, and `<cstring>`.
- In modern C++, these headers are generally preferred over the older C-style forms `<stdint.h>`, `<stdio.h>`, and `<string.h>`.

Anonymous namespaces

In C++, an anonymous namespace can be used to restrict symbols (variables, functions, types, and compile-time constants) to a single translation unit (i.e. a single source file).

This is similar to using the keyword `static` for functions or variables in C.

Anonymous namespaces are typically used in `.cpp` files to hide internal helper functions or variables that should not be visible outside the file.

Example:

```

namespace
{
    // Content only visible within this source file.
    void helper() {}
} // namespace

```

Nested namespaces

Namespaces can also be nested, similar to how folders can contain subfolders:

- Namespaces are often used to reflect the logical structure of the project.

- For example, assume a project has the following directory structure:

```
include/
  driver/
    gpio.hpp
    timer.hpp
source/
  driver/
    gpio.cpp
    timer.cpp
  main.cpp
```

A common convention would be:

- The content of `driver/gpio.hpp` and `driver/gpio.cpp` belongs to the namespace `driver::gpio`.
- The content of `driver/timer.hpp` and `driver/timer.cpp` belongs to the namespace `driver::timer`.

This makes it easy to see which module a function belongs to.

Nested namespace syntax

Modern C++ allows nested namespaces to be declared in a compact form:

```
namespace driver::gpio
{
  void init()
  {
    // GPIO initialization code.
  }
} // namespace driver::gpio
```

Before C++17, the same namespace would typically be written as:

```
namespace driver
{
  namespace gpio
  {
    void init()
    {
      // GPIO initialization code.
    }
  } // namespace gpio
} // namespace driver
```

Both forms are equivalent, but the compact form is preferred in modern C++.

Using functions from nested namespaces

A function defined in namespace `driver::gpio` can be called using its fully qualified name:

```
int main()
{
    driver::gpio::init();
    return 0;
}
```

This makes it clear which module the function belongs to and prevents naming conflicts in larger projects.

In short: In embedded systems, namespaces are commonly used to organize drivers, hardware abstraction layers (HAL), and communication protocols.

1.2.6 constexpr

The keyword `constexpr` indicates that a value or expression (including simple functions) can be evaluated at compile time rather than at runtime. This allows the compiler to compute values in advance, which can improve performance and ensure that such values are usable in constant-expression contexts.

In embedded systems, `constexpr` is particularly useful for defining hardware-related constants such as clock frequencies, register offsets, buffer sizes, and bit masks.

For example:

```
/** CPU frequency in Hz. */
constexpr std::uint32_t cpuFrequency{16000000U};

/** Baud rate in bps (bits per second). */
constexpr std::uint32_t baudrate{115200U};

/** Buffer length in bytes. */
constexpr std::size_t bufLen{20U};
```

Note: Brace initialization `{}` is used instead of `=` above:

- This is advantageous, since `{}` can be used to initialize everything.
- An exception is when using the `auto` keyword prior to C++17; then `auto` should be used in combination with `=` to avoid creating so-called initializer lists.

Because these values are known at compile time, the compiler can replace their usage directly with the computed value in the generated machine code.

Compared to macros (`#define`), `constexpr` has several advantages:

- It is type-safe.
- It obeys normal C++ scoping rules (functions, local blocks, and namespaces).
- It can be used in constant expressions.

For example, `constexpr` values can be used for:

- Array sizes.
- Template parameters.
- Compile-time calculations.

In modern C++, `constexpr` is generally preferred over `#define` for defining constants. `constexpr` can be used for functions when the values are known at compile time:

```
constexpr int add(const int x, const int y) { return x + y; }
```

1.2.7 noexcept

The keyword `noexcept` indicates that a function is guaranteed not to throw exceptions.

For reference, an example of throwing an exception is shown below. In this example, an exception is thrown if the value of `val` exceeds 100U:

```
#include <cstdint>
#include <stdexcept>

int main()
{
    constexpr std::uint8_t limit{100U};
    std::uint8_t val{};

    while (1)
    {
        if (limit < val++)
        {
            throw std::out_of_range("Value exceeded limit!");
        }
    }
    return 0;
}
```

Exceptions are a mechanism used in many C++ programs to signal runtime errors:

- When an exception is thrown, the program begins stack unwinding, meaning that the call stack is traversed until a matching exception handler (i.e. a `catch` block) is found.
- If a function marked `noexcept` attempts to throw an exception, the program will terminate immediately.

In many embedded systems, exceptions are disabled entirely. This is typically done to:

- Reduce binary size.
- Simplify the runtime environment.
- Ensure deterministic behavior.

In such systems, errors are typically handled using C-style mechanisms such as:

- Return codes.
- Status flags.
- Error callbacks.

For a function that either returns a value or has nothing to return, modern C++ offers a safer alternative to a return code combined with an out-parameter; see §1.2.14.

Because of this, it is common practice in embedded C++ to mark functions that are not expected to fail with the `noexcept` keyword.

Doing so has several benefits:

- It documents that the function cannot throw exceptions.
- It enables additional compiler optimizations.
- It avoids the need for exception-related stack handling.

Example:

```
namespace driver::gpio
{
    bool read() noexcept { return false; }
} // namespace driver::gpio
```

Important:

- In embedded drivers, functions that perform simple hardware operations (such as reading or writing registers) are typically marked `noexcept`, since they are not expected to throw exceptions.
- Functions that may perform dynamic memory allocation, such as resizing a vector, should not be marked `noexcept`, since memory allocation may fail.

1.2.8 Default arguments

C++ allows functions to specify default values for parameters:

- If the caller does not provide a value for a parameter, the default value is used automatically.
- This can simplify APIs by allowing common cases to use fewer arguments.

For example, consider the function `log()` below:

```
namespace debug
{
void log(const char* message, std::uint8_t level = 0U)
{
    // Log message with the specified level.
}
} // namespace debug
```

The parameter `level` has a default value of `0U`. This means the function can be called in two different ways:

- If the log level is not important, the function can be called with only the message:

```
debug::log("System started");
```

In this case, the value `0U` will automatically be used for `level`.

- If a specific log level is desired, the caller can provide it explicitly:

```
debug::log("Driver failure", 2U);
```

Default arguments are commonly used in embedded software for functions that have sensible default behavior, such as:

- Logging systems.
- Driver configuration functions.
- Initialization routines.

They allow APIs to remain flexible while keeping common usage simple.

Default argument placement

In C++, default arguments may only be omitted from right to left. This means that parameters with default values must be placed at the end of the parameter list.

A valid example is shown below:

```
void printValue(const std::uint32_t value, const bool hex = false) noexcept;
```

This function can be called in either of the following ways:

```
printValue(42U);
printValue(42U, true);
```

The following is not valid:

```
void printValue(const std::uint32_t value = 0U, const bool hex) noexcept;
```

This is not allowed because the second parameter does not have a default value, even though the first one does.

If this were allowed, a call such as this:

```
printValue(true);
```

would become unclear and potentially cause errors (true can be implicitly converted to value as 1).

1.2.9 Modern C++ structs

In C++, a struct can contain not only data members but also member functions, often referred to as methods. This allows related data and operations to be grouped together in a single type.

Consider the following driver::Gpio driver, implemented as a stub (for simulation):

```
#include <cstdint>

namespace driver
{
    /**
     * @brief GPIO stub driver.
     */
    struct Gpio
    {
        /** Pin the GPIO is connected to. */
        const std::uint8_t pin;

        /** GPIO state. */
        bool state;

        /**
         * @brief Set GPIO state.
         *
         * @param[in] state GPIO state (true = enabled, false = disabled).
         */
        void write(const bool state) noexcept
        {
            // Note: The 'this' keyword is used here to refer to our member variable 'state',
            // since we have an input argument with the same name.
            this->state = state;
        }
    }
}
```

```

/**
 * @brief Get GPIO state.
 *
 * @return True if the GPIO is enabled, false otherwise.
 *
 * @note The 'const' keyword is used after the method name to set the GPIO ↵
↵ instance to
 *       read-only in the scope of this method.
 */
bool read() const noexcept { return state; }
};
} // namespace driver

```

The `driver::Gpio` struct contains:

- Two data members (pin and state).
- Two methods (write and read).

These functions operate on the data stored in the struct.

Instances of the struct can be created as follows:

```

// Initialize LED connected to pin 9, state set to false.
driver::Gpio led{9U, false};

// Initialize button connected to pin 13, simulate pressdown at startup.
driver::Gpio button{13U, true};

```

Note that unlike in C, the keyword `struct` is not required when creating an instance of a struct and can therefore be omitted.

The functions can then be called using the dot operator, just like the data members:

```

// Enable the LED if the button is pressed.
const bool buttonPressed{button.read()};
led.write(buttonPressed);

```

In traditional C, similar functionality would typically be implemented using a struct combined with separate functions that take a pointer to the struct, as shown below:

```

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

/**
 * @brief GPIO stub driver.

```

```

*/
typedef struct
{
    /** Pin the GPIO is connected to. */
    const uint8_t pin;

    /** GPIO state. */
    bool state;
} gpio_t;

/**
 * @brief Set GPIO state.
 *
 * @param[in, out] self Pointer to the GPIO.
 * @param[in] state GPIO state (true = enabled, false = disabled).
 */
void gpio_write(gpio_t* self, const bool state)
{
    if (NULL == self) { return; }
    self->state = state;
}

/**
 * @brief Get GPIO state.
 *
 * @param[in] self Pointer to the GPIO.
 *
 * @return True if the GPIO is enabled, false otherwise.
 */
bool gpio_read(const gpio_t* self)
{
    return NULL != self ? self->state : false;
}

```

Instances of the struct can then be created and used as shown below:

```

// Initialize LED connected to pin 9, state set to false.
gpio_t led = {9U, false};

// Initialize button connected to pin 13, simulate pressdown at startup.
gpio_t button = {13U, true};

// Enable the LED if the button is pressed.
const bool button_pressed = gpio_read(&button);

```

```
gpio_write(&led, button_pressed);
```

Comparison

C-style approach.

```
gpio_write(&led, true);
```

C++ approach.

```
led.write(true);
```

The C++ syntax improves readability and keeps related functionality together, making the code easier to understand and maintain.

Constructor and destructor

It is also possible to add initialization and cleanup methods. These methods are known as:

- **Constructor:**
 - Has the same name as the struct.
 - Called automatically when an object is created.
 - Can be used to automatically initialize an object on creation (for instance pin configuration).
 - For example, a constructor for the `driver::Gpio` struct above can be implemented as follows:

```
/**
 * @brief Constructor.
 *
 * @param[in] pin Pin the GPIO is connected to.
 * @param[in] initialState Initial state (default = disabled).
 */
Gpio(const std::uint8_t pin, const bool initialState = false) noexcept
    : pin{pin}
    , state{initialState}
{
    std::printf("Initializing GPIO at pin %u!\n", pin);
    // Configure pin here!
}
```

- **Destructor:**
 - Has the same name as the struct, but with prefix `~`.
 - Called automatically when an object is destroyed.

- For example, this happens when an object goes out of scope or when `delete` is used.
- Can be used to automatically release allocated resources or undo hardware-related setup.
- Often used to release peripherals, disable interrupts, or restore hardware state in embedded systems.

```
/**
 * @brief Destructor.
 */
~Gpio() noexcept
{
    std::printf("Releasing resources reserved for GPIO at pin %u!\n", pin);
    // Release allocated resources here.
}
```

After adding the constructor and destructor, the struct `driver::Gpio` looks like this:

```
#include <cstdint>
#include <cstdio>

namespace driver
{
    /**
     * @brief GPIO stub driver.
     */
    struct Gpio
    {
        /** Pin the GPIO is connected to. */
        const std::uint8_t pin;

        /** GPIO state. */
        bool state;

        /**
         * @brief Constructor.
         *
         * @param[in] pin Pin the GPIO is connected to.
         * @param[in] initialState Initial state (default = disabled).
         */
        Gpio(const std::uint8_t pin, const bool initialState = false) noexcept
            : pin{pin}
            , state{initialState}
        {
```

```

        std::printf("Initializing GPIO at pin %u!\n", pin);
        // Configure pin here!
    }

    /**
     * @brief Destructor.
     */
    ~Gpio() noexcept
    {
        std::printf("Releasing resources reserved for GPIO at pin %u!\n", pin);
        // Release allocated resources here.
    }

    /**
     * @brief Set GPIO state.
     *
     * @param[in] state GPIO state (true = enabled, false = disabled).
     */
    void write(const bool state) noexcept
    {
        // Note: The 'this' keyword refers to the current object.
        // It is used here because the parameter name 'state' hides the member ↵
        ↵ variable.
        this->state = state;
    }

    /**
     * @brief Get GPIO state.
     *
     * @return True if the GPIO is enabled, false otherwise.
     *
     * @note The 'const' keyword is used after the method name to set the GPIO ↵
        ↵ instance to
        *      read-only in the scope of this method.
     */
    bool read() const noexcept { return state; }
};
} // namespace driver

```

Before a constructor is defined, the struct can be initialized using aggregate initialization. After a user-defined constructor is added, the same brace syntax may still be used, but it now calls the constructor instead.

For example, the following initialization:

```
int main()
{
    driver::Gpio led{9U, false};
    led.write(true);
}
```

will generate the following output because the constructor and destructor are called automatically:

```
Initializing GPIO at pin 9!
Releasing resources reserved for GPIO at pin 9!
```

This illustrates the idea behind **RAII** (*Resource Acquisition Is Initialization*):

- Initialization is performed during object creation.
- Cleanup is performed automatically when the object is destroyed.

Introduction to encapsulation

In C++, parts of a struct can be made private using the `private` keyword, as shown below for the `driver::Gpio` struct:

- The `public` keyword has been added for clarity; in a C++ struct, everything is public by default.
- The `private` keyword has been added to create a private segment.
- The member variables and methods have been removed for simplicity.

```
namespace driver
{
    struct Gpio
    {
    public:
        // Public segment - accessible outside the struct.
    private:
        // Private segment - inaccessible outside the struct.
    };
} // namespace driver
```

Using the `private` keyword has an important advantage:

- Symbols (member variables, constants, and methods) declared in the private segment are inaccessible outside the struct.
- This allows us to hide implementation details and data that should not be accessed directly by other parts of the program.

- In many designs, member variables are declared private. This ensures that the internal state of the object can only be modified through its member functions.

For instance, the member variables of the `driver::Gpio` struct are typically made private, which is shown below:

- The private segment is typically placed at the bottom of the struct, while the public interface (constructors and methods) is placed first. This makes it easier to see how the struct should be used.
- Here we add the `my` prefix to make it clear that these are member variables and to reduce the need to use the `this` keyword (`this->state = state` was used earlier).

```
namespace driver
{
    struct Gpio
    {
    private:
        /** Pin the GPIO is connected to. */
        const std::uint8_t myPin;

        /** GPIO state. */
        bool myState;
    };
} // namespace driver
```

Note: The snippet above only shows the private segment of `driver::Gpio`. Then we update the struct:

- Member variables `pin` and `state` are replaced with `myPin` and `myState`.
- The `pin()` method is added so that users still can read the pin number.

```
#include <cstdint>
#include <cstdio>

namespace driver
{
    /**
     * @brief GPIO stub driver.
     */
    struct Gpio
    {
        /**
         * @brief Constructor.
         *
         * @param[in] pin Pin the GPIO is connected to.
```

```

    * @param[in] initialState Initial state (default = disabled).
    */
    Gpio(const std::uint8_t pin, const bool initialState = false) noexcept
        : myPin{pin}
        , myState{initialState}
    {
        std::printf("Initializing GPIO at pin %u!\n", myPin);
        // Configure pin here!
    }

    /**
     * @brief Destructor.
     */
    ~Gpio() noexcept
    {
        std::printf("Releasing resources reserved for GPIO at pin %u!\n", myPin);
        // Release allocated resources here.
    }

    /**
     * @brief Get GPIO pin number.
     *
     * @return Pin the GPIO is connected to.
     */
    std::uint8_t pin() const noexcept { return myPin; }

    /**
     * @brief Set GPIO state.
     *
     * @param[in] state GPIO state (true = enabled, false = disabled).
     */
    void write(const bool state) noexcept { myState = state; }

    /**
     * @brief Get GPIO state.
     *
     * @return True if the GPIO is enabled, false otherwise.
     */
    bool read() const noexcept { return myState; }

private:
    /** Pin the GPIO is connected to. */
    const std::uint8_t myPin;

```

```

    /** GPIO state. */
    bool myState;
};
} // namespace driver

```

After making the member variables private, the compiler will generate an error if we try to access them directly:

```

int main()
{
    driver::Gpio led{90};

    // Allowed: pin() provides controlled read access to the pin number.
    std::printf("The LED is connected to pin %u!\n", led.pin());

    // Allowed: write() is part of the public interface.
    led.write(true);

    // Not allowed: myPin is private.
    // std::printf("Pin: %u!\n", led.myPin);

    // Not allowed: myState is private.
    // led.myState = true;
}

```

Chapter 2 introduces classes:

- In C++, structs and classes support almost the same language features, but classes use `private` as the default access level, while structs use `public`.
- Structs are often used for simple data-oriented types, while classes are commonly used for more complex objects that may involve inheritance, polymorphism, or custom copy and move semantics.

1.2.10 References

References provide an alternative way to pass variables to functions.

A reference acts as an alias for another variable. This means that operations performed on the reference directly affect the original variable. References behave similarly to pointers but can be used with normal variable syntax.

References are declared using the `&` symbol.

Consider the following example:

```
void toggle(bool& state)
{
    state = !state;
}
```

The parameter `state` is a reference to a `bool`.

This means the function modifies the original variable that was passed to it.

Example usage:

```
bool state{false};

toggle(state);
```

After the function call, the value of `state` will be `true`.

In C, similar behavior would typically be implemented using pointers:

```
void toggle(bool* state)
{
    *state = !(*state);
}
```

Comparison

C-style approach (with pointer).

```
toggle(&state);
```

C++ approach (with reference).

```
toggle(state);
```

While pointers are still used in C++, references are often preferred because they:

- Cannot be `nullptr` (NULL in C).
- Do not require the address operator (`&`) to pass addresses.
- Do not require dereferencing (`*`) to read the value a pointer is pointing at.
- Provide clearer syntax.

As a result, references are commonly used in modern C++ when a function needs to modify an existing variable. However, a reference cannot be reseated to refer to another object after initialization; pointers are still used when we need to change the address we refer to, for instance when implementing dynamic arrays.

1.2.11 The auto keyword

The keyword `auto` allows the compiler to automatically deduce the type of a variable from its initializer.

For example:

```
auto number = 5;    // Deduced as int.
auto counter = 100; // Deduced as unsigned int.
auto voltage = 3.3; // Deduced as double.
```

This can improve readability and reduce the need to repeat long type names.

However, care must be taken when using `auto` with brace initialization `{}` in C++11/14. In some cases the compiler may deduce an `std::initializer_list` instead of the intended type.

For example:

```
auto a = 10; // Deduced as int.
auto b{10};  // Deduced std::initializer_list<int> prior to C++17.
```

Because of this, when using `auto` before C++17 it was generally recommended to use the assignment operator `=` instead of brace initialization `{}`.

In modern C++ (C++17 and later), the behavior was improved and brace initialization works more intuitively. However, a common rule of thumb is still:

- When using `auto`, prefer `=` instead of `{}` unless you explicitly want an `std::initializer_list`.

However, in this book the data type will usually be specified explicitly. The `auto` keyword will mainly be used in situations where the type would otherwise be verbose, for example when iterating over containers:

```
// Enable all LEDs in a list.
for (auto& led : leds)
{
    led.on();
}
```

Without `auto`, the type may become very long.

1.2.12 Function templates: a first look

Note: Here we only introduce the basic idea. Templates are covered in more detail in Chapter 5.

Templates allow functions and classes to operate on multiple data types without duplicating code.

In embedded systems, templates are often used to implement generic utilities such as bit manipulation helpers.

Consider the following example to set a bit `bit` in a given register `reg`:

- The data type of the register for which to write is set to `T`. The type is set at compile time when the function is instantiated.
- We constrain this function to only work for integral types with a `static_assert`. If this function is called with some other type, such as a floating point number, a compiler error with error message `Failed to set bit in register: T must be of integral type!` is generated.
- We check the data type using `std::is_integral<T>` from `<type_traits>`, specifically via its member variable `value`:
 - If `T` is of integral type, `std::is_integral<T>::value` is `true` and the assertion succeeds.
 - If `T` isn't of integral type, `std::is_integral<T>::value` is `false` and the assertion fails.
 - For more information about type traits, see §1.2.15.
- The value `1` is cast to type `T` via a `static_cast`, which is a safer alternative to regular C casts.

```
#include <stdint>
#include <type_traits>

/**
 * @brief Set bit in register.
 *
 * @tparam T The register type. Must be integral.
 *
 * @param[in, out] reg Register to write to.
 * @param[in] bit The bit to set.
 */
template<typename T>
constexpr void set(T& reg, const std::uint8_t bit) noexcept
{
    static_assert(std::is_integral<T>::value,
                  "Failed to set bit in register: T must be of integral type!");
    reg |= (static_cast<T>(1U) << bit);
}
```

The function can be used to set bits in registers of arbitrary integral type. In the example below, bits 1–2 in an 8-bit register are set by calling the `set()` function twice:

```
std::uint8_t reg{};
set(reg, 1U);
```

```
set(reg, 2U);
```

Note: Unlike C, functions in C++ can have the same name as long as their parameter lists differ. The compiler determines which version to use at each function call based on the types of the arguments. This is called function overloading.

Template instantiation and code size

Templates are instantiated separately for each data type used in the program. For example, if the function `set()` is used with both `std::uint8_t` and `std::uint32_t`:

```
std::uint8_t reg1{};
std::uint32_t reg2{};

set(reg1, 1U);
set(reg2, 12U);
```

Then the compiler generates two separate versions of the function:

```
set(std::uint8_t&, std::uint8_t)
set(std::uint32_t&, std::uint8_t)
```

Each version is compiled independently and included in the final binary:

- This behavior is one of the reasons templates are considered zero-cost abstractions; the generated code is specialized for the exact type being used, which allows the compiler to optimize the implementation.
- However, in embedded systems this also means that excessive template usage can increase the binary size, since multiple versions of the same function may be generated.
- For this reason, templates should be used carefully in resource-constrained systems, especially when many different data types are involved.

Parameter packs

We can also implement parameter packs so that multiple bits can be set at once:

```
#include <cstdint>
#include <type_traits>

template<typename T, typename... Bits>
constexpr void set(T& reg, const Bits... bits) noexcept
{
    static_assert(std::is_integral<T>::value,
        "Failed to set bit in register: T must be of integral type!");
```

```

    for (const auto& bit : {bits...})
    {
        reg |= (static_cast<T>(1U) << bit);
    }
}

```

In the example below, bits 1–5 in an 8-bit register are set by calling the `set()` function once:

```

// Set bit 1-5 in a register.
std::uint8_t reg{};
set(reg, 1U, 2U, 3U, 4U, 5U);

```

The compiler will generate code similar to the following:

```

reg |= (static_cast<T>(1U) << 1U);
reg |= (static_cast<T>(1U) << 2U);
reg |= (static_cast<T>(1U) << 3U);
reg |= (static_cast<T>(1U) << 4U);
reg |= (static_cast<T>(1U) << 5U);

```

Fold expressions (C++17)

In the implementation above we iterate over the bits using a loop and a so-called **braced initializer list**. This approach is easy to read and resembles how similar logic might be written in C.

Since C++17, we can omit the loop by using a fold expression:

```

((reg |= (static_cast<T>(1U) << bits)), ...);

```

This expression expands the parameter pack `bits...` and performs the operation once for each element.

Note: The operation being folded needs its own pair of parentheses, hence the double parentheses at the start. The language requires each operand of a fold expression to be a so-called *cast-expression*, and an assignment such as `reg |= x` is not one. Writing

```

(reg |= (static_cast<T>(1U) << bits), ...); // Won't compile.

```

therefore produces a compiler error such as *binary expression in operand of fold-expression*.

Because of this feature, the `set()` function can be implemented more compactly:

```

template<typename T, typename... Bits>
constexpr void set(T& reg, const Bits... bits) noexcept

```

```
{
    static_assert(std::is_integral<T>::value,
        "Failed to set bit in register: T must be of integral type!");
    ((reg |= (static_cast<T>(1U) << bits)), ...);
}
```

1.2.13 `[[nodiscard]]`

`[[nodiscard]]` is a C++17 attribute that can be added to a function declaration. It tells the compiler to emit a warning if the caller ignores (discards) the function's return value.

Like `constexpr` and templates, `[[nodiscard]]` is a zero-cost abstraction: it exists purely as a compile-time aid and has no effect on the generated machine code or runtime performance.

Consider the `read()` method of the `driver::Gpio` struct introduced earlier:

```
[[nodiscard]] bool read() const noexcept { return myState; }
```

Since `read()` has no side effects, calling it and ignoring the result does nothing useful and is almost always a mistake. With `[[nodiscard]]` in place, the following now produces a compiler warning:

```
led.read(); // Warning: ignoring return value of function declared 'nodiscard'.
```

While the return value must be used to avoid the warning:

```
const bool state{led.read()};
```

`[[nodiscard]]` is particularly useful in embedded systems for functions such as:

- Query/read functions, similar to `read()` above.
- Functions returning error or status codes, where ignoring the result may hide a failure.
- Factory functions that return an owning resource, such as `std::unique_ptr` (introduced in Chapter 4), where discarding the result would immediately release the resource.
- Functions returning a `std::optional` (see §1.2.14), where the result is the only way to learn whether there was a value at all.

Note: `[[nodiscard]]` should not be added indiscriminately. Functions that are called mainly for their side effects, where the return value is genuinely optional, do not need it.

1.2.14 `std::optional`

Some functions do not always have a value to return. Reading a byte from a serial port is a typical example: if nothing has been received, there is no byte to hand back.

C has no direct way to express “a value, or nothing”, so one of two workarounds is normally used:

- **A sentinel value:** one value of the return type is reserved to mean “nothing”, such as -1 or 0xFF. This only works when the type has a value to spare. A received byte can be anything from 0x00 to 0xFF, so no value is left over, and the return type has to be widened to make room.
- **A status return and an out-parameter:** the function returns true or false, and writes the value through a pointer.

The second workaround is the more common one in embedded C:

```
#include <stdbool.h>
#include <stdint.h>

// Assume RX bytes are received via another function.
static bool rx_available = false;
static uint8_t last_rx = 0U;

bool read_byte(uint8_t* byte)
{
    if (!rx_available) { return false; }
    *byte = last_rx;
    rx_available = false;
    return true;
}
```

This works, but it has several weaknesses:

- The caller must declare a variable before the call, so the variable cannot be const, and it exists even when nothing was received.
- Nothing stops the caller from ignoring the return value and using the variable anyway, in which case it holds whatever it held before the call.
- A null pointer passed by mistake is dereferenced, unless the function adds a check for it.
- The signature does not say which parameters are inputs and which are outputs.

A value, or nothing

Since C++17, the standard library provides `std::optional<T>` in the header `<optional>`. An `std::optional<T>` either holds a value of type `T`, or it is empty. The empty state is written `std::nullopt`.

With it, the function can return the byte itself, or nothing:

```
#include <cstdint>
#include <optional>

namespace
{
```

```
// Once again, assume RX bytes are received via another function.
bool rxAvailable{false};
std::uint8_t lastRx{};
} // namespace

[[nodiscard]] std::optional<std::uint8_t> readByte() noexcept
{
    if (!rxAvailable) { return std::nullopt; }
    rxAvailable = false;
    return lastRx;
}
```

Note that:

- The out-parameter is gone. The function takes only its inputs and returns only its output.
- A T converts implicitly to an `std::optional<T>`, so the byte is returned as it is.
- `std::nullopt` is returned when there is nothing to return.
- The function is marked `[[nodiscard]]`: an optional that is never looked at has told the caller nothing.

Using the result

An `std::optional` can be compared with `std::nullopt` to find out whether it holds a value. Once we know that a value is present, we reach it with the dereference operator `*`, just as with a pointer:

```
const auto byte = readByte();
if (std::nullopt != byte) { std::printf("Received byte %u!\n", *byte); }
else { std::printf("No bytes available!\n"); }
```

The check can be written in two other ways that mean the same thing:

- The method `has_value()` returns true when a value is present.
- The optional itself converts to true in a condition when it holds a value, so `if (byte)` works as well.

With `std::optional`, the value and the knowledge of whether it exists travel together in one object. The value cannot be reached without going through that object, which makes the check hard to forget.

std::optional as a member variable

`std::optional` is not limited to return types. A value paired with a flag that says whether the value is valid is a common pattern in drivers, and the pair can be replaced by a single optional. The variables `rxAvailable` and `lastRx` in the example above are such a pair: declared as one

`std::optional<std::uint8_t>`, a byte is stored by assigning to it, it is emptied by assigning `std::nullopt` to it, and it is checked and read in the same way as a returned optional.

The two can then no longer disagree: there is no flag that says “valid” while the value is stale, since there is no separate flag.

What it costs, and what to avoid

- **No dynamic allocation.** The value is stored inside the optional itself, not on the heap, which makes `std::optional` safe to use in systems that forbid dynamic memory.
- **A little more memory.** An optional is the size of a `T` plus a flag, rounded up to the alignment of `T`. An `std::optional<std::uint8_t>` is two bytes, and an `std::optional<std::uint32_t>` is eight.
- **Check before dereferencing.** Using `*` on an empty optional is undefined behavior, exactly as with a null pointer.
- **Avoid `value()` when exceptions are disabled.** The method `value()` also returns the value, but throws `std::bad_optional_access` if the optional is empty. In a system without exceptions, check first and use `*`.
- **The condition tests presence, not the value.** An `std::optional<std::uint8_t>` holding `0` is true in a condition, because it holds a value. For the same reason, `std::optional<bool>` is easy to misread and best avoided.
- **It says that there is nothing, not why.** When the caller needs to know the reason for a failure, a status code is still the right tool.

1.2.15 Additional information about type traits

Type traits allow programs to inspect and reason about types at compile time.

In the C++ standard library, many type traits are implemented as small structs containing a static constant boolean value (static means that the value belongs to the type itself rather than to an instance of the struct).

We can implement a simplified version of a trait that checks whether a type is unsigned:

```
template<typename T>
struct isUnsigned
{
    static constexpr bool value{false};
};
```

Note:

- The type trait is nothing more than a struct template containing a boolean constant value.
- By default, this boolean value is `false` for all types `T`.
- We can specialize the template for specific types `T` so that `value` becomes `true`, as shown below, where the types `std::uint8_t`, `std::uint16_t`, `std::uint32_t`, and `std::uint64_t` are treated as unsigned types:

```

template<>
struct isUnsigned<std::uint8_t>
{
    static constexpr bool value{true};
};

template<>
struct isUnsigned<std::uint16_t>
{
    static constexpr bool value{true};
};

template<>
struct isUnsigned<std::uint32_t>
{
    static constexpr bool value{true};
};

template<>
struct isUnsigned<std::uint64_t>
{
    static constexpr bool value{true};
};

```

Note: There is no separate specialization for `std::size_t`. It is not a type of its own but an alias of one of the unsigned types above (`std::uint64_t` on a 64-bit Linux machine, typically `std::uint32_t` on a 32-bit microcontroller), so it is already covered, and specializing it as well would be a redefinition that fails to compile.

We can use our own type trait `isUnsigned<T>` to check if a type `T` is unsigned as shown below:

```

static_assert(isUnsigned<T>::value, "T must be of unsigned type!");

```

Since C++17, it is common to also provide a variable template ending in `_v` that exposes the boolean value directly:

```

template<typename T>
inline constexpr bool isUnsigned_v{isUnsigned<T>::value};

```

Note: `inline` allows the variable template to be defined in a header file and included in multiple translation units without causing multiple definition errors.

This variable template allows us to write:

```
static_assert(isUnsigned_v<T>, "T must be of unsigned type!");
```

In this book, the traditional form `isUnsigned<T>::value` will be used for clarity.

1.2.16 Summary

This section introduced the following modern C++ features commonly used in embedded software:

- `.h` vs `.hpp` header file conventions.
- `extern "C"` and `#ifdef __cplusplus` for headers shared between C and C++.
- Namespaces.
- `constexpr` and compile-time constants.
- `noexcept` and exception handling in embedded systems.
- Default arguments.
- Structs with member functions, constructors/destructors, and encapsulation.
- References.
- The `auto` keyword.
- Function templates.
- `[[nodiscard]]` for catching ignored return values.
- `std::optional` for functions that may have no value to return.

These features allow developers to write safer, clearer, and more maintainable embedded software while still maintaining full control over hardware and performance.

1.3 Review

What you should be able to do now

After this chapter, you should:

- Be familiar with the modern C++ language features commonly used in embedded systems.
- Understand how C++ can improve structure and safety in embedded software.
- Understand how modern C++ structs can group data and behavior in a single type.
- Know how simple drivers can be implemented using modern C++ structs with member functions, constructors and destructors, and encapsulation.
- Understand how references simplify function interfaces compared to pointers.
- Understand how templates enable generic utilities such as bit manipulation helpers.
- Have an intuition for compile-time programming (`constexpr` and templates).
- Understand how `std::optional` expresses a return value that may be absent.

Questions to test yourself

You should be able to explain:

1. What is the purpose of `constexpr`?
2. Why is `noexcept` often used in embedded systems?
3. How does a C++ struct differ from a traditional C struct?
4. Why are constructors and destructors useful in embedded C++?
5. What is the purpose of making member variables private?
6. What advantages do references have compared to pointers?
7. Why can templates increase binary size in embedded systems?
8. What are parameter packs used for?
9. What problem does `std::optional` solve compared to a `bool` return value and an out-parameter?
10. Why should `value()` be avoided on an `std::optional` when exceptions are disabled?

Looking ahead

Chapter 2 is about classes in C++:

- A deeper look at constructors and destructors.
- The keywords `explicit`, `final`, `default`, and `delete`.
- Copy constructors.
- Move constructors.
- Assignment operators.

In it, you will implement your first class-based drivers.

1.4 Exercises

How to work these. These exercises practice the concepts from §1.2. Each set is one program: write it in its own directory under `lectures/L01/exercises` in the companion repository, with the file names its README gives, and build and run it with the Makefile from §1.1.3.

How to check your work. Run `make test` from the repository root. A set's tests switch on as soon as its files exist, and compare your program's output with the example output each exercise gives, character for character; a set you have not started is reported as `SKIP`.

The solutions are published in the companion repository, one program per set under `lectures/L01/appendix/solutions`, each with its Makefile. Try each exercise yourself before looking at them.

EXERCISE SET 1

Default Arguments and Functions

Exercise 1.1 Debug logger

CODE

Create a namespace called `debug` containing a function:

```
void log(const char* message, const std::uint8_t level = 0U);
```

Tasks

1. Implement the function so it prints the message using `std::printf` from `<cstdio>`.
2. Include the log level in the output.
3. Mark the function `noexcept`.

Example usage:

```
debug::log("System started");
debug::log("Sensor failure", 2U);
```

Example output:

```
System started, log level = 0
Sensor failure, log level = 2
```

Exercise 1.2 System delay

CODE

In the same program as in Exercise 1.1, create a function that simulates a simple software delay:

```
void delay_ms(const std::uint32_t ms = 1U);
```

Tasks

1. Place the function inside namespace `app`.
2. Mark the function `noexcept`.
3. Inside the function, define a `constexpr` constant named `maxCount` and assign it the value `100000000UL`.
4. Implement a delay using nested loops:
 - The outer loop should iterate `ms` times.
 - The inner loop should iterate up to `maxCount`.
 - To reduce the risk of the compiler optimizing the loop away, add a volatile dummy variable `volatile std::uint32_t dummy{}` inside the function and increment it inside the inner loop.

5. Use this function to simulate a software delay of 100 ms between the debug prints in Exercise 1.1.

EXERCISE SET 2

Struct Driver**Exercise 1.3 Software timer**

CODE

In this exercise you will implement a simple software timer driver `driver::Timer` in a header file `driver/timer.hpp`.

Add the following lines at the top of the file:

```
/**
 * @file Timer driver implementation.
 */
#pragma once

#include <cstdint>
#include <cstdio>
```

The driver shall simulate a timer that counts milliseconds and generates a timeout when a configured timeout value is reached.

The struct shall:

- Use private member variables with the prefix `my`.
- Have a constructor.
- Have a destructor.

Use `std::printf` from `<cstdio>` for terminal prints.

a) Private member variables

Add three private member variables:

- The first member variable shall:
 - Store the timeout in milliseconds.
 - Have the type `std::uint16_t`.
 - Be named `myTimeout_ms`.
- The second member variable shall:
 - Store the internal counter in milliseconds.
 - Have the type `std::uint16_t`.
 - Be named `myCounter_ms`.
- The third member variable shall:

- Indicate whether the timer is running.
- Have the type `bool`.
- Be named `myRunning`.

b) Constructor

Add a constructor that:

- Takes two input arguments:
 - The timeout in milliseconds.
 - The initial running state, which shall be set to `false` by default.
- Initializes:
 - `myTimeout_ms` with the provided timeout value.
 - `myCounter_ms` with `0U`.
 - `myRunning` with `false`.
- Prints that the timer is being created.
- Calls the public method `start()` if the initial running state is `true`.

Example:

```
Creating timer!
```

c) Destructor

Add a destructor that:

- Calls the public method `stop()`.
- Prints that the timer is being destroyed.

Example:

```
Destroying timer!
```

d) Public methods

Add the following public methods:

- `timeout_ms()` shall:
 - Return `myTimeout_ms`.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
- `isRunning()` shall:
 - Return `myRunning`.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
- `start()` shall:
 - Set `myRunning` to `true`.

- Print Starting timer!.
- `stop()` shall:
 - Set `myRunning` to `false`.
 - Print Stopping timer!.
- `toggle()` shall:
 - Toggle `myRunning`.
 - Print that the timer was toggled and whether it is now running or stopped: Toggling timer: running! when enabled, Toggling timer: stopped! when disabled.
- `tick()` shall:
 - Increment `myCounter_ms` if `myRunning` is `true`.
 - Otherwise do nothing.
- `hasTimedOut()` shall:
 - Return `true` if `myCounter_ms >= myTimeout_ms`, otherwise `false`.
 - If `myCounter_ms >= myTimeout_ms`, reset `myCounter_ms` to `0U`.
 - Be marked `noexcept` and `[[nodiscard]]`.

e) Create and use a timer instance

In `main()`:

- Create a timer instance:
 - Name the instance `timer`.
 - Set the timeout to `1000 ms`.
 - Set the timer to running at startup.
- Create a loop that runs for `5000` iterations:
 - Call `tick()` each iteration.
 - Call `hasTimedOut()` to check whether the timer has expired.
 - Print Timeout after `x ms!`, where `x` is the configured timeout, whenever the timer times out.

Since the timer timeout is `1000 ms` and the loop runs for `5000` iterations, the timer should generate five timeouts.

Example output:

```
Creating timer!
Starting timer!
Timeout after 1000 ms!
Timeout after 1000 ms!
Timeout after 1000 ms!
Timeout after 1000 ms!
```

```
Timeout after 1000 ms!
Stopping timer!
Destroying timer!
```

EXERCISE SET 3

References

Exercise 1.4 Swap values

CODE

Implement the following function in an anonymous namespace:

```
constexpr void swap(std::uint32_t& a, std::uint32_t& b) noexcept;
```

Tasks

1. Swap the values using a temporary variable temp.
2. Test the function with two variables.

Example output:

```
Before swap: a = 3, b = 10
After swap: a = 10, b = 3
```

EXERCISE SET 4

Bit Manipulation Templates

Exercise 1.5 Clear bit

CODE

Create a function template to clear a bit in a register:

```
template<typename T>
constexpr void clear(T& reg, std::uint8_t bit) noexcept;
```

Implement this function template inside an anonymous namespace.

Tasks

1. Use `static_assert` with `std::is_integral<T>::value` from `<type_traits>` to ensure that `T` is an integral type.
2. Use the error message `Cannot perform bit operation with non-integral type!` in the `static_assert`.
3. Clear the selected bit in the register.
4. Print the result in binary using C++ I/O functionality:

- Output stream `std::cout` from `<iostream>` for printing.
- `std::bitset<N>` from `<bitset>` to generate a bitset representation.

Example usage:

```
std::uint8_t reg{0xFFU};
clear(reg, 2U);
std::cout << "Register content: " << std::bitset<8U>(reg) << "\n";
```

Expected output:

```
Register content: 11111011
```

Exercise 1.6 Toggle bit(s)

CODE

Create a function template for toggling one or several bits in a register using a parameter pack:

```
template<typename T, typename... Bits>
constexpr void toggle(T& reg, const Bits... bits) noexcept;
```

Implement this function template in the same anonymous namespace as Exercise 1.5.

Tasks

1. Use a parameter pack.
2. Ensure the type is integral as in Exercise 1.5.
3. Iterate over the bits (for example using `{bits...}`).
4. Toggle all specified bits in the register.
5. Print the result in binary using C++ I/O functionality:
 - Output stream `std::cout` from `<iostream>` for printing.
 - `std::bitset<N>` from `<bitset>` to generate a bitset representation.

Example usage:

```
std::uint8_t reg{0xFFU};
clear(reg, 2U);
toggle(reg, 0U, 2U, 4U, 6U);
std::cout << "Register content: " << std::bitset<8U>(reg) << "\n";
```

Expected output:

```
Register content: 10101110
```

EXERCISE SET 5

std::optional**Exercise 1.7** Lowest set bit

CODE

Create a function template that finds the lowest set bit in a register:

```
template<typename T>
[[nodiscard]] constexpr std::optional<std::uint8_t> lowestSetBit(const T reg) noexcept;
```

Implement this function template inside an anonymous namespace.

Tasks

1. Ensure that `T` is an integral type, with the same `static_assert` and error message as in Exercise 1.5.
2. Return the number of the lowest bit that is set in the register, where bit 0 is the least significant bit.
3. Return `std::nullopt` if no bit is set.
4. In `main()`, call the function for a `std::uint8_t` register with the value `0x28U`, and for one with the value `0x00U`.
5. For each register, print the register value in hexadecimal together with the bit number if a bit was found, and otherwise that no bit is set:
 - Check that the optional holds a value before using it, and read the value with `*`. Do not use `value()`.
 - Output stream `std::cout` from `<iostream>` for printing.
 - The manipulator `std::hex` to print the register value in hexadecimal.

Note: `std::cout` prints a `std::uint8_t` as a character rather than as a number. Cast the register value and the bit number to `int` with a `static_cast` before printing them.

Expected output:

```
Lowest set bit in value 0x28: 3
No set bit in value 0x0!
```

Exercise 1.8 Reflection

REFLECTION

- a) Why is 0 not a suitable return value for a register with no bit set?
- b) The function could instead return 0xFFU when no bit is set. What would the caller have to know, and what happens if the caller forgets?
- c) Why is the value read with * after a check, rather than with value()?
- d) Why is lowestSetBit() marked [[nodiscard]]?

Classes

A type that owns its data, and decides how it may be created, copied and used.

IN THIS CHAPTER

- An introduction to object-oriented programming in C++.
- Classes and encapsulation.
- Constructors and destructors.
- File organization for classes: separation into header and source files.
- Copy and move semantics.
- The keywords `explicit`, `final`, `default`, and `delete`.

This chapter is about creating simple classes in C++. You will see what constructors and destructors are for, what encapsulation means in object-oriented programming, and how a class is organized across a header file and a source file. You will also meet copy and move semantics, the keywords `explicit`, `final`, `default`, and `delete`, and enumeration classes (`enum class`).

The chapter builds directly on the modern C++ concepts of Chapter 1, in particular the structs with member functions of §1.2.9, and otherwise assumes only basic C: functions, pointers, and structs.

§2.1 is an introduction to classes in C++, built around a GPIO driver, and is the part to read first. §2.2 is a deeper discussion of classes: the keyword `explicit`, static members, and how copy and move operations are written. The exercises in §2.4 are where you write your first class-based drivers, and their solutions are in the companion repository.

2.1 Classes in C++

2.1.1 Background

To maintain good structure in larger programs, there is an increasing need to group related data and related behavior into a single unit instead of spreading everything across multiple separate variables and functions. For example, it may be beneficial to represent a GPIO pin

through a class that stores the pin number, direction, and current state, while the class also contains functions for writing, reading, and toggling the pin.

In programming, the most common data structures used to store related data as a single unit are so-called structs and classes:

- **Structs** are mainly used in C (the concept of classes does not exist at all in C).
- **Classes** are used in many modern programming languages, such as C++, Java, and Python.
- In C++, both structs and classes are used.
- Classes can be seen as a further development of structs, which are relatively simple. In terms of structure, however, these data structures are very similar.

Languages that support classes are said to support object orientation, or OOP (*Object Oriented Programming*). C is considered a procedural language because it is based on programming via functions. Through structs, however, some object-oriented principles can be mimicked, which will be demonstrated below.

Some object-oriented languages, such as Java and C#, are purely object-oriented, meaning that classes must be used. Other programming languages, such as C++ and Python, are said to be multi-paradigm languages, since it is possible to program both in an object-oriented way (as in Java) and in a procedural way (as in C), or a combination of both.

2.1.2 Terminology

As mentioned earlier, classes are used instead of structs in most conventional programming languages created after C.

In C++, however, structs are almost identical to classes; the only formal difference is that class members are private by default, whereas they are public by default in structs. Structs in C++ can therefore be used in the same way as classes.

To reduce confusion in the rest of this section, however, we will use the following **convention in this book**:

- **Struct** refers to simple data structures without member functions, for example:
 - Pure C-style structs (sometimes called POD, *Plain Old Data*).
 - Structs containing constants (`static constexpr`).
- **Class** refers to a more advanced variant of the data structure we have seen so far.

2.1.3 Differences between classes and structs

Classes can be seen as a further development of structs. Therefore, classes and structs are similar in terms of their fundamental structure, but classes offer much more functionality and flexibility. In this section, we will look at the most basic functionality that classes provide.

Some of the most significant differences between classes and structs are:

- **Classes support encapsulation**, which means that you can choose what is accessible/visible outside the class. This allows secret data or information that is not relevant outside the class (such as implementation details) to be hidden.
- **Classes can contain functions**, which means we do not need to implement associated functions using struct pointers, as shown previously.
- **Classes offer functionality for inheritance**, which means a new class can obtain the contents of a base class. This can significantly reduce the amount of code required.
- **Classes can be implemented using so-called class templates** to make one or more data types selectable, similar to the function templates we saw earlier. An example of a class template in the standard library is `std::vector`, which allows us to choose what should be stored in a given vector.

2.1.4 Basic structure of a class

We implement a simple example of a GPIO driver in C++ using a class named `Gpio`. The class is placed in the namespace `driver::gpio`, which is common in larger C++ projects to organize code and avoid name collisions.

We will use `camelCase` instead of `snake_case`, which means that:

- We use capital letters instead of underscores in function and type names. So instead of a function name such as `print_numbers()`, we will name this function `printNumbers()`.
- The same pattern is followed for types such as structs, classes, and enumerations. One difference, however, is that type names begin with a capital letter, for example `Gpio` instead of `gpio`, `Direction` instead of `direction`, and so on.

Step 1: Add include directives

We begin by including the standard header `<cstdint>`, so that we have access to data types such as `std::uint8_t`:

```
#include <cstdint>
```

Step 2: Create namespace and enumeration class

Next, we create the namespace `driver::gpio` and the enumeration class `Direction` to describe the direction of a GPIO pin. We set the underlying data type of the enumeration to `std::uint8_t`, which makes the type compact and ensures that no negative values can be represented (which can simplify validation):

```
namespace driver::gpio
{
    /**
     * @brief GPIO direction configuration.
     */
```

```
enum class Direction : std::uint8_t
{
    Input,          ///< Input without pull-up.
    InputPullup,    ///< Input with pull-up enabled.
    Output,         ///< Output.
    Count,          ///< Number of supported directions.
};
} // namespace driver::gpio
```

The enumeration class above is used to specify whether a pin should be:

- Input
- InputPullup
- Output

We can also easily check whether a given enumerator direction is valid by ensuring that it has a numeric value that is less than `Direction::Count`:

```
[[nodiscard]] constexpr bool isDirectionValid(const Direction direction) noexcept
{
    return static_cast<std::uint8_t>(Direction::Count) > static_cast<std::uint8_t>(direction);
}
```

The primary reason we choose to implement `Direction` as an enumeration class instead of a traditional enumeration is that we do not need to use prefixes to avoid name collisions; instead, this is built into the language.

Note above that the enumerators are now named

- `Input`,
- `InputPullup`, and
- `Output`

instead of, for example,

- `GPIO_DIRECTION_INPUT`,
- `GPIO_DIRECTION_INPUT_PULLUP`, and
- `GPIO_DIRECTION_OUTPUT`.

With the enumeration class above, we instead write

- `Direction::Input`,
- `Direction::InputPullup`, and
- `Direction::Output`.

In general, it is recommended to use enumeration classes instead of traditional enumerations when programming in modern C++, that is, C++11 and later.

Step 3: Define the class `Gpio`

Next, we define the class `Gpio` using the keyword `class`:

```
namespace driver::gpio
{
/**
 * @brief GPIO driver.
 */
class Gpio final
{
public:

private:
};
} // namespace driver::gpio
```

We have added two keywords to the class, `public` and `private`:

- Everything that falls under the keyword `public` belongs to the public section:
 - The contents of this section are visible and accessible outside the class.
 - Everything that we want the “user” of the class to be able to see and use should be declared here.
 - For example, this may include methods for writing to the pin, reading its state, and toggling the output.
- Everything that falls under the keyword `private` belongs to the private section:
 - The contents of this section are not visible or accessible outside the class.
 - Here we place everything that we want to be inaccessible to the “user” of the class.
 - For example, this may include internal data or implementation details.

In our case, we place all member variables in the private section. We can also place methods that should only be used internally within the class here. Hiding information in this way is called **encapsulation**.

Common practice for a class is the following:

- Member variables are kept private.
- The public section appears first in the class so that the “user” can easily see what methods and other features are available if needed.
- Implementation details are placed below in the private part of the class, and the idea is that the user should not even need (or want) to read this far.

Usually, only the public part of the class is documented; the rest of the class consists of implementation details that are only of interest to the developer of the class and are therefore not documented. Since this is a pedagogical example, however, all code is documented.

We have also added the keyword `final` after the class name. This means that the class may not be used as a base class. In other words, no other class may inherit from `Gpio`. In embedded contexts, this is often a reasonable design choice for small, concrete drivers where you want to keep the design simple and clear.

Step 4: Add member variables

Under the private section we add the member variables, that is, the variables in the class that store the pin number, direction, and current state. Note that:

- We use the prefix `my` to avoid name collisions with methods that we will add later.
- We mark member variables that should never change after initialization as `const`.

```
namespace driver::gpio
{
    /**
     * @brief GPIO driver.
     */
    class Gpio final
    {
    public:

    private:
        /** GPIO pin number. */
        const std::uint8_t myPin;

        /** GPIO direction. */
        const Direction myDirection;

        /** GPIO state. */
        bool myState;
    };
} // namespace driver::gpio
```

The member variables are private so that the “user” cannot read and write them arbitrarily. Instead, we later provide methods such as `write()`, `read()`, and `toggle()` to control how the object is used.

Step 5: Add constructors and destructors

Before adding the remaining methods, we add constructors, which can be seen as initialization routines that are automatically called when an object of the class is created.

First, we delete the so-called default constructor so that it is not possible to create a GPIO object without providing a pin number and direction:

```
public:
    Gpio() = delete; // No default constructor.
```

This constructor is called the default constructor because it has no input arguments. Therefore, it is not possible to create an empty object as shown below:

```
// Won't compile, since the default constructor is deleted.
driver::gpio::Gpio gpio{};
```

Normally, deleted functions, such as the default constructor above, are placed at the bottom of the public section.

We can also add a destructor, which can be seen as a function that is automatically called just before an instance of the given class is destroyed.

The destructor looks like the default constructor, with the difference that the tilde ~ is placed before the name of the class. If we do not need to do anything special before the object is destroyed, we can omit the destructor; the compiler will then create a default destructor for us. If we still want to create a destructor for clarity, we can explicitly set the destructor to default using the keyword `default`.

```
/**
 * @brief Destructor.
 */
~Gpio() noexcept = default;
```

In modern C++, the keyword `default` is often recommended, which will be used in this example. However, it would also be possible to implement a default destructor as shown below:

```
/**
 * @brief Destructor.
 */
~Gpio() noexcept {}
```

We want the user to specify the pin number, direction, and optionally the initial state directly when a GPIO object is created:

- We therefore create a constructor that takes the pin number, direction, and an initial state as input arguments.
- We mark the constructor with `explicit` so that it is not used for unintended implicit conversions.
- See §2.2.1 for additional information about the keyword `explicit`.

```

/**
 * @brief Create a new GPIO object.
 *
 * @param[in] pin GPIO pin number.
 * @param[in] direction GPIO direction.
 * @param[in] initialState Initial GPIO state (default = false).
 */
explicit Gpio(const std::uint8_t pin, const Direction direction,
              const bool initialState = false) noexcept
{}

```

In the constructor above, we initialize the member variables so that:

- myPin is assigned the pin number stored in the input argument pin.
- myDirection is assigned the value stored in the input argument direction.
- myState is assigned the value stored in the input argument initialState. By default, initialState is set to false.

This is achieved by adding an initialization section between the function header and the function body (marked with curly braces {}). Note that we begin the initialization section with a colon, initialize using curly braces, and separate the initialization of each member variable with a comma.

```

/**
 * @brief Create a new GPIO object.
 *
 * @param[in] pin GPIO pin number.
 * @param[in] direction GPIO direction.
 * @param[in] initialState Initial GPIO state (default = false).
 */
explicit Gpio(const std::uint8_t pin, const Direction direction,
              const bool initialState = false) noexcept
    : myPin{pin}
    , myDirection{direction}
    , myState{initialState}
{}

```

If we needed to perform additional actions after initializing the member variables, we could place this in the constructor's function body. In this case, however, we only want to initialize the member variables, so we do not need to add anything further.

The constructor shown above can now be used to initialize GPIO objects. Below is an example showing how we initialize the object led to represent an output on pin 13 with an initial low state:

```
driver::gpio::Gpio led{13U, driver::gpio::Direction::Output, false};
```

Since the default value for the input argument that stores the initial state is false, we can also omit this argument:

```
driver::gpio::Gpio led{13U, driver::gpio::Direction::Output};
```

We can also use the constructor to initialize an object button to represent an input with pull-up on pin 2:

```
driver::gpio::Gpio button{2U, driver::gpio::Direction::InputPullup};
```

Step 6: Copy and move operations

In addition to regular constructors, C++ also provides special member functions for copying and moving. These are very important in modern C++, and it is useful to be familiar with them already when learning about classes.

The most common operations are shown below:

Operation	Description	Signature for class Gpio
Copy constructor	Creates a new object as a copy of another object.	Gpio(const Gpio& other)
Move constructor	Creates a new object by moving resources from another object.	Gpio(Gpio&& other)
Copy assignment operator	Assigns an already existing object from another object.	Gpio& operator=(const Gpio& other)
Move assignment operator	Assigns an already existing object by moving resources from another object.	Gpio& operator=(Gpio&& other)

In many real embedded systems, copying and moving are often completely disallowed, since an object frequently represents a unique hardware resource. In this way, it is ensured that each Gpio object represents exactly one physical GPIO pin and cannot be duplicated by mistake.

For the class Gpio, copying and moving are deleted as shown below, normally placed at the bottom of the public section of the class.

```
Gpio(const Gpio&)           = delete; // No copy constructor.
Gpio(Gpio&&)               = delete; // No move constructor.
Gpio& operator=(const Gpio&) = delete; // No copy assignment.
Gpio& operator=(Gpio&&)     = delete; // No move assignment.
```

See §2.2.3 for more information about copy and move operations.

Step 7: Add public methods

For a GPIO object, it is more natural to expose functionality in the form of behaviors rather than traditional getter and setter methods. We therefore add the following public methods:

- `write(bool state)` to write a new state to the pin.
- `read()` to read the current state of the pin.
- `toggle()` to toggle the state of the pin.

Below is the method `write()`, which is used to write a new state to the GPIO instance:

```
/**
 * @brief Write a new state to the GPIO pin.
 *
 * @param[in] state The state to write.
 */
void write(const bool state) noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = state;
    }
}
```

In the example above, we only allow writing if the pin is configured as an output. This is a simple way to illustrate that the class can protect its internal data and control how the object may be used.

Below is the method `read()`, which returns the current state of the pin:

```
/**
 * @brief Read the current GPIO state.
 *
 * @return Current GPIO state.
 */
[[nodiscard]] bool read() const noexcept { return myState; }
```

Note that we write the keyword `const` directly after the function header, since we ensure that the class contents can only be read. This means that we cannot accidentally modify any of the class members.

We could use the method above to read and print the state of an LED that was previously implemented in software through the object `led`. Here, `std::printf()` from `<cstdio>` is used to print the state:

```
int main()
{
    driver::gpio::Gpio led{13U, driver::gpio::Direction::Output};
    std::printf("State:\t%s\n", led.read() ? "High" : "Low");
    return 0;
}
```

The output becomes the following:

```
State: Low
```

We then add the method `toggle()` to allow toggling the state of the pin:

```
/**
 * @brief Toggle the GPIO state.
 */
void toggle() noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = !myState;
    }
}
```

If, for example, we call `toggle()`, we ensure that the member variable `myState` switches between false and true, but only if the pin is configured as an output.

We can first call `write(true)` to set the LED pin high, and then call `toggle()` to switch it back to a low level. We print the state both before and after the calls:

```
int main()
{
    driver::gpio::Gpio led{13U, driver::gpio::Direction::Output};
    std::printf("Old state:\t%s\n", led.read() ? "High" : "Low");
    led.write(true);
    std::printf("After write:\t%s\n", led.read() ? "High" : "Low");
    led.toggle();
    std::printf("After toggle:\t%s\n", led.read() ? "High" : "Low");
    return 0;
}
```

The output becomes the following:

```

Old state:      Low
After write:    High
After toggle:   Low

```

After adding the methods, the class `Gpio` looks like this:

```

namespace driver::gpio
{
/**
 * @brief GPIO driver.
 */
class Gpio final
{
public:
/**
 * @brief Create a new GPIO object.
 *
 * @param[in] pin GPIO pin number.
 * @param[in] direction GPIO direction.
 * @param[in] initialState Initial GPIO state (default = false).
 */
explicit Gpio(const std::uint8_t pin, const Direction direction,
               const bool initialState = false) noexcept
    : myPin{pin}
    , myDirection{direction}
    , myState{initialState}
{}

/**
 * @brief Destructor.
 */
~Gpio() noexcept = default;

/**
 * @brief Write a new state to the GPIO pin.
 *
 * @param[in] state The state to write.
 */
void write(const bool state) noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = state;
    }
}
}

```

```

    }
}

/**
 * @brief Read the current GPIO state.
 *
 * @return Current GPIO state.
 */
[[nodiscard]] bool read() const noexcept { return myState; }

/**
 * @brief Toggle the GPIO state.
 */
void toggle() noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = !myState;
    }
}

Gpio() = delete; // No default constructor.
Gpio(const Gpio&) = delete; // No copy constructor.
Gpio(Gpio&&) = delete; // No move constructor.
Gpio& operator=(const Gpio&) = delete; // No copy assignment.
Gpio& operator=(Gpio&&) = delete; // No move assignment.

private:
    /** GPIO pin number. */
    const std::uint8_t myPin;

    /** GPIO direction. */
    const Direction myDirection;

    /** GPIO state. */
    bool myState;
};
} // namespace driver::gpio

```

Step 8: Splitting a class into header and source files

Objects of many different classes are often used in a program. These classes are also often larger than the class we have seen here. It is therefore impractical to implement all classes in a single file.

Typically, a header file dedicated to a specific class is created, including public enumeration classes. For example:

- The class `Gpio` and the enumeration class `Direction` in the namespace `driver::gpio` should be implemented via the header files `gpio.hpp` and `direction.hpp` in the directory `driver/gpio`. The file paths would then be:
 - `driver/gpio/direction.hpp`
 - `driver/gpio/gpio.hpp`

At the top of each header file, we place the directive `#pragma once`, which ensures that we do not accidentally define multiple copies of the contents of the header file if it is included in multiple files, in the same way that header guards work in C. It prevents the same header file from being included multiple times in the same translation unit (a source file together with all included header files):

```
#pragma once
```

We also include `<cstdint>` to gain access to data types such as `std::uint8_t`:

```
#include <cstdint>
```

File `driver/gpio/direction.hpp`. We implement the enumeration class `Direction` in this file:

```
/**
 * @file GPIO direction configurations.
 */
#pragma once

#include <cstdint>

namespace driver::gpio
{
    /**
     * @brief GPIO direction configuration.
     */
    enum class Direction : std::uint8_t
    {
        Input,          ///< Input without pull-up.
        InputPullup,    ///< Input with pull-up enabled.
        Output,          ///< Output.
        Count,           ///< Number of supported directions.
    };
} // namespace driver::gpio
```

File driver/gpio/gpio.hpp. We implement the class `Gpio` in this file. Since the enumeration class `Direction` is used directly in this file, `driver/gpio/direction.hpp` is included:

```
/**
 * @file GPIO driver implementation.
 */
#pragma once

#include <stdint>

#include "driver/gpio/direction.hpp"

namespace driver::gpio
{
/**
 * @brief GPIO driver.
 */
class Gpio final
{
public:
/**
 * @brief Create a new GPIO object.
 *
 * @param[in] pin GPIO pin number.
 * @param[in] direction GPIO direction.
 * @param[in] initialState Initial GPIO state (default = false).
 */
explicit Gpio(const std::uint8_t pin, const Direction direction,
               const bool initialState = false) noexcept
    : myPin{pin}
    , myDirection{direction}
    , myState{initialState}
{}

/**
 * @brief Destructor.
 */
~Gpio() noexcept = default;

/**
 * @brief Write a new state to the GPIO pin.
 *
 * @param[in] state The state to write.
 */
}
```

```

void write(const bool state) noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = state;
    }
}

/**
 * @brief Read the current GPIO state.
 *
 * @return Current GPIO state.
 */
[[nodiscard]] bool read() const noexcept { return myState; }

/**
 * @brief Toggle the GPIO state.
 */
void toggle() noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = !myState;
    }
}

Gpio() = delete; // No default constructor.
Gpio(const Gpio&) = delete; // No copy constructor.
Gpio(Gpio&&) = delete; // No move constructor.
Gpio& operator=(const Gpio&) = delete; // No copy assignment.
Gpio& operator=(Gpio&&) = delete; // No move assignment.

private:
    /** GPIO pin number. */
    const std::uint8_t myPin;

    /** GPIO direction. */
    const Direction myDirection;

    /** GPIO state. */
    bool myState;
};
} // namespace driver::gpio

```

Method definitions in the file `driver/gpio/gpio.cpp`. Class method definitions are often placed in a separate source file to keep the header file more readable. For example, to keep the file `driver/gpio/gpio.hpp` readable, we can place the method definitions in a source file named `driver/gpio/gpio.cpp`.

At the top of this source file, we include the following header files:

- `<cstdint>` to gain access to data types such as `std::uint8_t`.
- `driver/gpio/gpio.hpp` to gain access to the class definition.
- `driver/gpio/direction.hpp` to gain access to the enumeration class `Direction`.

Note: Both `<cstdint>` and `driver/gpio/direction.hpp` are included indirectly via `driver/gpio/gpio.hpp`, but to follow good practice we should not rely on indirect dependencies.

```
#include <cstdint>

#include "driver/gpio/direction.hpp"
#include "driver/gpio/gpio.hpp"
```

We then place the method definitions from the class `Gpio` in this file. We therefore copy all methods from the class `Gpio` into this file. It is worth noting that:

- For the compiler to understand that each method belongs to the class `Gpio`, we must use the prefix `Gpio`. For example, the method `write()` must be defined as `Gpio::write()`.
- Default values for input arguments should only be written in the method declaration in the header file.
- Keywords placed before method names, such as `explicit`, should not be included in the method definition.
- Attributes such as `[[nodiscard]]` should also only be written on the method declaration in the header file — the definition in the source file does not need to repeat it.
- Constructors, destructors, and similar functions marked with `default` or `delete` do not need to be placed in the source file.

We begin with the method `write()`. In the header file `driver/gpio/gpio.hpp`, we keep the method header together with its documentation:

```
/**
 * @brief Write a new state to the GPIO pin.
 *
 * @param[in] state The state to write.
 */
void write(const bool state) noexcept;
```

In the source file `driver/gpio/gpio.cpp`, we then place the method definition:

```
// -----
void Gpio::write(const bool state) noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = state;
    }
}
```

We implement the definitions for the remaining methods in the source file `driver/gpio/gpio.cpp`. We place a line of hyphens between each method to visually separate them now that the method documentation is not included.

File `driver/gpio/gpio.cpp`. After adding all method definitions, the file `driver/gpio/gpio.cpp` looks like this:

```
/**
 * @file GPIO driver implementation details.
 */
#include <stdint>

#include "driver/gpio/direction.hpp"
#include "driver/gpio/gpio.hpp"

namespace driver::gpio
{
    // -----
    Gpio::Gpio(const std::uint8_t pin, const Direction direction, const bool ↵
        ↵ initialState) noexcept
        : myPin{pin}
        , myDirection{direction}
        , myState{initialState}
    {}

    // -----
    void Gpio::write(const bool state) noexcept
    {
        if (myDirection == Direction::Output)
        {
            myState = state;
        }
    }

    // -----
```

```

bool Gpio::read() const noexcept { return myState; }

// -----
void Gpio::toggle() noexcept
{
    if (myDirection == Direction::Output)
    {
        myState = !myState;
    }
}
} // namespace driver::gpio

```

File driver/gpio/gpio.hpp. The readability of the header file driver/gpio/gpio.hpp now increases, since we have removed a large part of the implementation details by moving the method definitions:

- However, we must still keep the method declarations and the definitions of the member variables inside the class.
- We remove the keyword `const` from parameters passed *by value* to improve readability; marking these as `const` only has an effect in the function definition.

```

/**
 * @file GPIO driver implementation.
 */
#pragma once

#include <cstdint>

#include "driver/gpio/direction.hpp"

namespace driver::gpio
{
    /**
     * @brief GPIO driver.
     */
    class Gpio final
    {
    public:
        /**
         * @brief Create a new GPIO object.
         *
         * @param[in] pin GPIO pin number.
         * @param[in] direction GPIO direction.
         * @param[in] initialState Initial GPIO state (default = false).

```

```

    */
    explicit Gpio(std::uint8_t pin, Direction direction, bool initialState = false)
    noexcept;

    /**
     * @brief Destructor.
     */
    ~Gpio() noexcept = default;

    /**
     * @brief Write a new state to the GPIO pin.
     *
     * @param[in] state The state to write.
     */
    void write(bool state) noexcept;

    /**
     * @brief Read the current GPIO state.
     *
     * @return Current GPIO state.
     */
    [[nodiscard]] bool read() const noexcept;

    /**
     * @brief Toggle the GPIO state.
     */
    void toggle() noexcept;

    Gpio() = delete; // No default constructor.
    Gpio(const Gpio&) = delete; // No copy constructor.
    Gpio(Gpio&&) = delete; // No move constructor.
    Gpio& operator=(const Gpio&) = delete; // No copy assignment.
    Gpio& operator=(Gpio&&) = delete; // No move assignment.

private:
    /** GPIO pin number. */
    const std::uint8_t myPin;

    /** GPIO direction. */
    const Direction myDirection;

    /** GPIO state. */
    bool myState;
};

```

```
} // namespace driver::gpio
```

2.1.5 Summary

After going through this section, you as the reader should have enough knowledge to create a small class.

You should be able to:

- Control what is visible and not visible outside a class through encapsulation.
- Create constructors to initialize objects of the class in question.
- Understand what `explicit` is used for.
- Understand what `final` is used for.
- Recognize and write a destructor.
- Recognize the copy constructor, copy assignment operator, move constructor, and move assignment operator.
- Use `= default` and `= delete` to control which operations should be allowed.
- Add public methods that control how an object is used.
- Create enumeration classes.
- Place code in a namespace to structure projects and avoid name collisions.

2.2 Advanced Discussion on Classes

2.2.1 The keyword `explicit`

Consider the simple class `Led` below:

```
#include <cstdint>

class Led final
{
public:
    Led(const std::uint8_t pin) noexcept
        : myPin{pin}
        , myState{false}
    {}

    void setEnabled(const bool state) noexcept { myState = state; }
    [[nodiscard]] bool isEnabled() const noexcept { return myState; }

private:
    const std::uint8_t myPin;
    bool myState;
```

```
};
```

Using the constructor above, we can create a `Led` instance and pass it as a copy (*pass-by-value*) to a function `ledPrint`, as shown below. The output is printed using `std::printf()` from `<cstdio>`:

```
void ledPrint(const Led led) noexcept
{
    const char* state{led.isEnabled() ? "on" : "off"};
    std::printf("The LED is %s!\n", state);
}

int main()
{
    Led led1{8U};
    led1.setEnabled(true);
    ledPrint(led1);
    return 0;
}
```

The output appears as shown below:

```
The LED is on!
```

We can also create a `Led` instance through copy initialization, as shown below. Note that in this case we have not created a `Led` instance — instead, a pin number is passed (by mistake). This argument is implicitly converted into a `Led` instance:

```
void ledPrint(const Led led) noexcept
{
    const char* state{led.isEnabled() ? "on" : "off"};
    std::printf("The LED is %s!\n", state);
}

int main()
{
    ledPrint(8U);
    return 0;
}
```

The output appears as shown below:

```
The LED is off!
```

Note: In this example, `led` is passed as a copy (pass-by-value) to clearly demonstrate how an implicit conversion can occur through the constructor. For larger classes, a `const` reference is often used instead to avoid unnecessary copies.

What happens in this case is that:

- The compiler implicitly constructs a `Led` instance given from an unsigned integer used as the pin number.
- The compiler expected a `Led` instance, but since it received an unsigned integer instead, which matches the class constructor, it implicitly invoked this constructor and thereby created a `Led` instance.
- This might be seen as efficient or elegant, but it can also lead to hard-to-detect bugs and is therefore not recommended.
- It is therefore good practice to ensure that implicit conversion of `Led` instances is not allowed. This can be achieved by adding the keyword `explicit` before the constructor, as shown below:

```
#include <cstdint>

class Led final
{
public:
    explicit Led(const std::uint8_t pin) noexcept
        : myPin{pin}
        , myState{false}
    {}

    void setEnabled(const bool state) noexcept { myState = state; }
    [[nodiscard]] bool isEnabled() const noexcept { return myState; }

private:
    const std::uint8_t myPin;
    bool myState;
};
```

After marking the constructor `explicit`, the previous code example would have produced a compilation error.

2.2.2 static members

So far, every method and member variable we have seen belongs to a specific object: `myPin` and `myState` each have a different value for every `Gpio` instance, and `read()` operates on one particular instance's state. The keyword `static` lets a member instead belong to the *class itself*, shared by every instance rather than duplicated per object.

In this section we continue using the class `Gpio`.

Static methods

A static method is called through the class name, using the scope resolution operator `::`, rather than through an object:

```
Gpio::instanceCount();
```

Because a static method has no associated object, it has no `this` pointer and therefore cannot access non-static (instance) members such as `myState`. It can only access other static members.

We add a static method `instanceCount()` that reports how many `Gpio` instances currently exist:

```
/**
 * @brief Get the number of currently active GPIO instances.
 *
 * @return Number of active GPIO instances.
 */
[[nodiscard]] static std::uint8_t instanceCount() noexcept { return ~
    ~ ourInstanceCount; }
```

Note: A static method can technically also be called through an object, for example `led1.instanceCount()`, but this is discouraged; it misleadingly suggests that the result depends on `led1`, when it does not.

Static data members

A static data member is shared by all instances of the class, instead of each object storing its own copy. We use the prefix `our` for private static members, the same way `my` is used for private instance members; both exist to avoid name collisions with methods.

static constexpr is used for a compile-time constant shared by the class, for example the maximum number of `Gpio` instances supported by the target MCU:

```
/** Maximum number of GPIO instances supported by this MCU. */
static constexpr std::uint8_t MaxInstances{40U};
```

Note: Unlike regular camelCase members and methods, a static `constexpr` member is named starting with a capital letter, for example `MaxInstances` rather than `maxInstances`. Some C++ style guides instead use a `k` prefix for this (`kMaxInstances`); this book uses a leading capital letter.

Since C++17, a static `constexpr` data member is implicitly inline. This means `MaxInstances` needs no separate definition in a source file. Unlike the plain static member shown next, it can be fully declared and initialized directly inside the class.

A **plain static** member is used instead when the shared value must change at runtime, for example a counter tracking how many `Gpio` instances currently exist:

```
private:
    static std::uint8_t ourInstanceCount;
```

Unlike static constexpr, a plain static member must be defined exactly once in a source file, or the linker fails:

```
std::uint8_t Gpio::ourInstanceCount{0U};
```

We update the constructor and destructor to keep ourInstanceCount accurate:

```
Gpio::Gpio(const std::uint8_t pin, const Direction direction, const bool
    initialState) noexcept
    : myPin{pin}
    , myDirection{direction}
    , myState{initialState}
{
    ++ourInstanceCount;
}

Gpio::~Gpio() noexcept
{
    --ourInstanceCount;
}
```

Note: This is more than a bookkeeping exercise. Recall from §2.1.4 that a Gpio object represents a unique physical pin — ourInstanceCount combined with MaxInstances gives the constructor a way to actually enforce that limit, refusing to create more Gpio instances than the MCU has physical pins for.

We can now query the shared instance count without needing any particular Gpio object:

```
int main()
{
    Gpio led{13U, Direction::Output};
    std::printf("Active GPIO instances: %u\n", Gpio::instanceCount());
    return 0;
}
```

```
Active GPIO instances: 1
```

2.2.3 Copy and move operations

In addition to regular constructors, C++ also provides special member functions for copying and moving. These are very important in modern C++, and it is useful to become familiar with them when learning about classes.

The most common ones are:

- **Copy constructor:** creates a new object as a copy of another object.
- **Copy assignment operator:** assigns an already existing object from another object.
- **Move constructor:** creates a new object by moving resources from another object.
- **Move assignment operator:** assigns an already existing object by moving resources from another object.

In this section we continue using the class `Gpio`. This allows the same class to be used throughout the chapter while demonstrating the syntax for copy and move operations.

The following implementations are mainly shown to illustrate the syntax. For a class such as `Gpio`, it is usually better to delete these operations entirely.

We must keep in mind that the member variables `myPin` and `myDirection` are declared as `const`. This means that these values can only be set when the object is created and cannot be changed later. This affects how copy and move operations can be implemented.

Copy constructor

The copy constructor is used to create a new object as a copy of another object.

```
// Create led1.
Gpio led1{13U, Direction::Output, false};

// Create copy of led1.
Gpio led2{led1};
```

An explicit copy constructor can be implemented as follows:

```
/**
 * @brief Copy constructor.
 *
 * @param[in] other GPIO instance to copy.
 */
Gpio(const Gpio& other) noexcept
    : myPin{other.myPin}
    , myDirection{other.myDirection}
    , myState{other.myState}
    {}
```

Here we initialize the member variables of the new instance with the values from the object `other`.

Move constructor

The move constructor is used to create a new object by moving resources from another object.

For a simple class such as `Gpio`, where we only store small built-in types, there are actually no heavy resources to move. Despite this, it is still useful to demonstrate the syntax.

```
#include <utility>

// Create led1.
Gpio led1{13U, Direction::Output};

// Move led1 into led2.
Gpio led2{std::move(led1)};
```

Note: To invoke the move constructor, the function `std::move()` from `<utility>` is used. An explicit move constructor can be implemented as follows:

```
/**
 * @brief Move constructor.
 *
 * @param[in, out] other GPIO instance to move from.
 */
Gpio(Gpio&& other) noexcept
    : myPin{other.myPin}
    , myDirection{other.myDirection}
    , myState{other.myState}
{
    // Clear resources allocated by `other`
    other.myState = false;
}
```

Notes:

- `other` is an **rvalue reference**, written as `Gpio&&`. This allows the object's resources to be moved instead of copied.
- We cannot modify `other.myPin` or `other.myDirection`, because these member variables are declared as `const`.
- For this class, moving is in practice identical to copying, since no resources are owned dynamically.
- If `other` had contained other resources, such as dynamically allocated memory, ownership of these would have been transferred to the new object, and `other` would have been left without them (for example with its pointer set to `nullptr`).

Copy assignment operator

A copy assignment operator is used when an already existing object should be assigned from another object.

```
// Create led1 and led2.
Gpio led1{13U, Direction::Output};
Gpio led2{12U, Direction::Output};

// Make led2 a copy of led1.
led2 = led1;
```

For the class `Gpio`, however, this is problematic. During assignment, an already existing object must be overwritten, but `myPin` and `myDirection` are constant and may not be changed after initialization.

An otherwise simplified operator could have looked like this:

```
/**
 * @brief Copy assignment operator.
 *
 * @param[in] other GPIO instance to copy from.
 *
 * @return Reference to this instance.
 */
Gpio& operator=(const Gpio& other) noexcept
{
    if (&this != &other)
    {
        myState = other.myState;
    }
    return *this;
}
```

Note that we **cannot** write:

```
myPin      = other.myPin;
myDirection = other.myDirection;
```

since these are constant.

Move assignment operator

Similarly, there is a move assignment operator, which is used when an object is assigned from a temporary object or via `std::move` from `<utility>`:

```
#include <utility>

Gpio led1{13U, Direction::Output};
Gpio led2{12U, Direction::Output};

led2 = std::move(led1);
```

A simplified implementation could look like this:

```
/**
 * @brief Move assignment operator.
 *
 * @param[in, out] other GPIO instance to move from.
 *
 * @return Reference to this instance.
 */
Gpio& operator=(Gpio&& other) noexcept
{
    if (this != &other)
    {
        myState      = other.myState;
        other.myState = false;
    }
    return *this;
}
```

Here as well, however, we cannot move `myPin` or `myDirection`, since these are constant.

2.2.4 Recommendation

In many real embedded systems, copying and moving are often completely disallowed, since an object frequently represents a unique hardware resource.

The deleted copy and move operations are normally placed at the bottom of the public section:

```
Gpio(const Gpio&)           = delete; // No copy constructor.
Gpio(Gpio&&)               = delete; // No move constructor.
Gpio& operator=(const Gpio&) = delete; // No copy assignment.
Gpio& operator=(Gpio&&)     = delete; // No move assignment.
```

In this way, it is ensured that each `Gpio` object represents exactly one physical GPIO pin and cannot be duplicated by mistake.

2.3 Review

What you should be able to do now

After this chapter, you should:

- Be able to create simple classes in C++.
- Understand the purpose of constructors and destructors.
- Understand the concept of encapsulation in object-oriented programming.
- Understand how classes are organized across header and source files.
- Be familiar with copy and move semantics.
- Understand the purpose of the keywords `explicit`, `final`, `default`, and `delete`.
- Be able to define and use enumeration classes (`enum class`).

Questions to test yourself

You should be able to explain:

1. What is the difference between a struct and a class in C++?
2. What does encapsulation mean?
3. What is a constructor, and when is it called?
4. What is a destructor, and when is it called?
5. What is the meaning of the keywords `explicit` and `final`?
6. What are `default` and `delete` used for?

Looking ahead

Chapter 3 is about inheritance and interfaces. You will learn how larger systems can be structured using:

- Class inheritance.
- Abstract interfaces.
- Interface-based driver design in embedded systems.

2.4 Exercises

How to work these. These exercises reinforce the concepts from §2.1 and §2.2. Build each program with a Makefile like the one in §1.1.3; for a class split across header and source files, use the include/source layout and Makefile from §1.1.5. Write each set in its own directory under `lectures/L02/exercises` in the companion repository, with the file names its README gives.

How to check your work. Run `make test` from the repository root. A set's tests switch on as soon as its files exist, check the declarations each exercise asks for (`explicit`, `noexcept`, `final`, `deleted` operations) as well as the behaviour, and compare your program's output with the example output each exercise gives, character for character; a set you have not started is reported as `SKIP`.

The solutions are published in the companion repository, one program per set under `lectures/L02/appendix/solutions`, each with its `Makefile`. Try each exercise yourself before looking at them.

EXERCISE SET 1

First Class

Exercise 2.1 Simple LED class

CODE

In a new program, create a class `Led` in the namespace `driver::gpio`:

```
namespace driver::gpio
{
    class Led final
    {
    public:

    private:
    };
} // namespace driver::gpio
```

Tasks

a) Add two private member variables:

- The first member variable shall:
 - Be named `myPin`.
 - Represent the pin number to which the LED is connected.
 - Have the type `std::uint8_t`.
 - Be readable only after initialization.
- The second member variable shall:
 - Be named `myState`.
 - Represent the state of the LED (`true/false`).
 - Be `true` when the LED is enabled.

b) Add a constructor that:

- Takes a pin number (`std::uint8_t`).

- Takes an initial state (`bool`) with the default value `false`.
 - Uses an initialization list to initialize the member variables.
 - Is marked `explicit` and `noexcept`.
- c) Add a destructor and mark it `noexcept` and `default`.
- d) Implement the following methods directly in the class:
- The method `isOn()` shall:
 - Return `true` if the LED is enabled, otherwise `false`.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `on()` shall:
 - Take no parameters.
 - Set the LED state to `true`.
 - Return no value.
 - Be marked `noexcept`.
 - The method `off()` shall:
 - Take no parameters.
 - Set the LED state to `false`.
 - Return no value.
 - Be marked `noexcept`.
 - The method `toggle()` shall:
 - Toggle the LED state.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
- e) Delete the following methods:
- The default constructor.
 - The copy constructor.
 - The move constructor.
 - The copy assignment operator.
 - The move assignment operator.
- f) Create an instance of the class:

```
constexpr std::uint8_t ledPin{13U};
driver::gpio::Led led{ledPin};
```

Test the class by:

- Printing the state.

- Calling `on()`.
- Printing again.
- Calling `toggle()`.
- Printing once more.

Use `std::printf()` from `<cstdio>` for the output.

Example output:

```
Initial state: Off
After on(): On
After toggle(): Off
```

Exercise 2.2 Button class

CODE

In the same program, create a class `Button` in the namespace `driver::gpio`:

```
namespace driver::gpio
{
class Button final
{
public:

private:
};
} // namespace driver::gpio
```

Tasks

- Add two private member variables:
 - The first member variable shall:
 - Be named `myPin`.
 - Represent the pin number to which the button is connected.
 - Have the type `std::uint8_t`.
 - Be readable only after initialization.
 - The second member variable shall:
 - Be named `myPressed`.
 - Represent the state of the button (`true/false`).
 - Be `true` when the button is pressed.
- Add a constructor that:
 - Takes a pin number (`std::uint8_t`).

- Uses an initialization list to initialize the member variables, with the button released (not pressed).
 - Is marked `explicit` and `noexcept`.
- c) Add a destructor and mark it `noexcept` and `default`.
- d) Implement the following methods directly in the class:
- The method `pin()` shall:
 - Return the pin number.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `isPressed()` shall:
 - Return `true` if the button is pressed, otherwise `false`.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `setPressed()` shall:
 - Take an argument of type `bool`.
 - Update the internal state of the button.
 - Return no value.
 - Be marked `noexcept`.
- e) Delete the following methods:
- The default constructor.
 - The copy constructor.
 - The move constructor.
 - The copy assignment operator.
 - The move assignment operator.
- f) Create an instance:

```
constexpr std::uint8_t buttonPin{2U};
driver::gpio::Button button{buttonPin};
```

Simulate a button press:

```
button.setPressed(true);
```

Print the pin number and whether the button is pressed using `std::printf()` from `<cstdio>`.

Example output:

```
Button pin: 2
Pressed: Yes
```

EXERCISE SET 2

Interaction Between Classes

Exercise 2.3 Control LED with a button

[CODE](#)

Use the classes from the previous exercises.

Tasks

- a) Create an LED on pin 13.
- b) Create a button on pin 2.
- c) If the button is not pressed, the LED should be off.
- d) If the button is pressed, the LED should turn on.
- e) Print the LED state.

Example output:

```
Button released -> LED: Off  
Button pressed  -> LED: On
```

Exercise 2.4 Blink function

[CODE](#)

Create a loop that toggles the LED state.

Tasks

- a) Turn the LED off, then run a loop for six iterations.
- b) In each iteration:
 - Call `toggle()`.
 - Print the LED state.

Example output:

```
Blink 1: On  
Blink 2: Off  
Blink 3: On  
Blink 4: Off  
Blink 5: On  
Blink 6: Off
```

EXERCISE SET 3

Class Split Across Multiple Files

Exercise 2.5 Buzzer class

CODE

In this exercise, you will create a class `Buzzer` in the namespace `driver`. The class should be split across multiple files. Therefore, create the following directory structure:

```
Makefile
include/
  driver/
    buzzer.hpp
source/
  driver/
    buzzer.cpp
  main.cpp
```

Note:

- All method declarations shall be written in `driver/buzzer.hpp`.
- All method definitions shall be implemented in `driver/buzzer.cpp`.

In the header file `driver/buzzer.hpp`, the class shall be declared according to the structure below:

```
#pragma once

#include <stdint>

namespace driver
{
  class Buzzer final
  {
  public:

  private:
  };
} // namespace driver
```

Tasks

a) Add two private member variables:

- The first member variable shall:
 - Represent the pin number to which the buzzer is connected.
 - Have the type `std::uint8_t`.

- Be named `myPin`.
 - Be readable only after initialization.
 - The second member variable shall:
 - Represent the state of the buzzer (`true/false`).
 - Be named `myEnabled`.
 - Be `true` when the buzzer is enabled.
- b) Add a constructor in the header file that:
- Takes a pin number (`std::uint8_t`).
 - Takes an initial state (`bool`) with the default value `false`.
 - Is marked `explicit` and `noexcept`.
- c) Add a destructor in the header file:
- Mark it `noexcept`.
 - Implement it in `driver/buzzer.cpp` so that the output below is generated when the buzzer is destroyed:

```
Releasing resources allocated for buzzer at pin 8!
```

Note: In the example output above, it is assumed that the buzzer is connected to pin 8.

- d) Declare the following methods in the header file and implement them in `driver/buzzer.cpp`:
- The method `pin()` shall:
 - Return the pin number to which the buzzer is connected.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `isEnabled()` shall:
 - Return `true` if the buzzer is enabled, otherwise `false`.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `enable()` shall:
 - Set the buzzer state to `true`.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `disable()` shall:
 - Set the buzzer state to `false`.
 - Take no parameters.
 - Return no value.

- Be marked `noexcept`.
- The method `toggle()` shall:
 - Toggle the buzzer state.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
- e) Delete the following methods in the header file:
 - The default constructor.
 - The copy constructor.
 - The move constructor.
 - The copy assignment operator.
 - The move assignment operator.
- f) In `main.cpp`, create an instance of the class:

```
constexpr std::uint8_t buzzerPin{8U};  
driver::Buzzer buzzer{buzzerPin};
```

Test the class by:

- Printing the state.
- Calling `enable()`.
- Printing again.
- Calling `toggle()`.
- Printing again.

Use `std::printf()` from `<cstdio>` for the output.

Example output:

```
Initial state: Disabled  
After enable(): Enabled  
After toggle(): Disabled  
Releasing resources allocated for buzzer at pin 8!
```

EXERCISE SET 4

Timer Class

Exercise 2.6 Timer

CODE

In this exercise, you will create a class `Timer` in the namespace `driver`.

The class should be split across multiple files. Therefore, create a new project with the same layout as the previous exercise, containing the following files:

```
include/driver/timer.hpp
source/driver/timer.cpp
```

Note:

- All method declarations shall be written in `driver/timer.hpp`.
- All method definitions shall be implemented in `driver/timer.cpp`.

In the header file, the class shall be declared according to the structure below:

```
#pragma once

#include <cstdint>

namespace driver
{
    class Timer final
    {
    public:

    private:
    };
} // namespace driver
```

Tasks

a) Add four private member variables:

- The first member variable shall:
 - Be named `myTimeout_ms`.
 - Represent the timeout duration in milliseconds.
 - Have the type `std::uint32_t`.
 - Be readable only after initialization.
- The second member variable shall:
 - Be named `myCounter_ms`.
 - Represent the internal timer counter.

- Have the type `std::uint32_t`.
 - The third member variable shall:
 - Be named `myRunning`.
 - Represent whether the timer is active or not (`true/false`).
 - Be `true` when the timer is running.
 - The fourth member variable shall:
 - Be named `myInitialized`.
 - Represent whether the timer is initialized or not (`true/false`).
 - Be `true` when the timer is initialized.
- b) Add a constructor in the header file that:
- Takes a timeout value in milliseconds (`std::uint32_t`).
 - Uses an initialization list to initialize the member variables.
 - Initializes the counter to 0.
 - Initializes the timer as stopped.
 - Marks the timer as initialized if the timeout value is valid (> 0).
 - Is marked `explicit` and `noexcept`.
- c) Add a destructor in the header file:
- Mark it `noexcept`.
 - Implement it in `driver/timer.cpp` so that:
 - The timer is stopped if it was running.
 - If the timer was running and is stopped, the output below should be printed:

```
Stopping timer before deletion!
```

- d) Declare the following methods in the header file and implement them in `driver/timer.cpp`:
- The method `timeout_ms()` shall:
 - Return the configured timeout in milliseconds.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `isRunning()` shall:
 - Return `true` if the timer is running, otherwise `false`.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `isInitialized()` shall:
 - Return `true` if the timer is initialized, otherwise `false`.

- Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
 - The method `start()` shall:
 - Start the timer if the timer is initialized.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `stop()` shall:
 - Stop the timer if the timer is initialized.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `toggle()` shall:
 - Toggle the timer if the timer is initialized.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `tick()` shall:
 - Simulate that 1 millisecond has passed.
 - Increment the counter if the timer is running; otherwise do nothing.
 - Return no value.
 - Be marked `noexcept`.
 - The method `hasTimedOut()` shall:
 - Return `true` when the counter has reached the timeout value, otherwise `false`.
 - Reset the counter to 0 when a timeout occurs.
 - Be marked `noexcept` and `[[nodiscard]]`.
- e) Delete the following methods:
- The default constructor.
 - The copy constructor.
 - The move constructor.
 - The copy assignment operator.
 - The move assignment operator.
- f) In `main.cpp`, create an instance of the class:

```
constexpr std::uint32_t timeout_ms{1000U};  
driver::Timer timer{timeout_ms};
```

Test the class by:

- Starting the timer.
- Running a loop for 3000 iterations. In each iteration:
 - Call `tick()`.
 - Check `hasTimedOut()`.
 - Print a message if a timeout has occurred:

```
Timeout after 1000 ms!
```

Note: In the example output above, it is assumed that the timeout is set to 1000 milliseconds.

Use `std::printf()` from `<cstdio>` for the output above.

Example output:

```
Timeout after 1000 ms!  
Timeout after 1000 ms!  
Timeout after 1000 ms!  
Stopping timer before deletion!
```

Inheritance and Interfaces

One interface, many drivers, and code that does not care which one it has.

IN THIS CHAPTER

- Inheritance, for creating classes that build upon existing classes.
- Interfaces (abstract classes), for defining a common set of methods that different classes can implement in different ways.
- A complete example: one LED interface, implemented for two different microcontrollers and used by a single function that works with both.

This chapter is about object-oriented design for embedded systems. After it, you should be able to create classes that inherit from other classes, create interfaces, and understand how several different implementations can be used through the same interface. The chapter assumes basic familiarity with C++ classes, constructors and `const`, which is what Chapter 2 covered.

Read about inheritance in §3.1 and about interfaces in §3.2. §3.3 then walks through a complete program built around an interface. If you want to see how interfaces work “under the hood”, there is also an implementation of the same idea in C, from the previous embedded C course: https://github.com/qrtech-academy/embedded-c/blob/main/lectures/L10/interface_demo/README.md. The exercises in §3.5 are where you design an interface and its implementations yourself. Their solution is in the companion repository.

3.1 Inheritance in C++

Inheritance is a fundamental principle in C++ that makes it possible to create new classes that build upon existing classes. In this way, one can “reuse” all or parts of existing classes and thereby avoid writing the same code for several different classes.

In terms of terminology:

- A class that inherits from another is called a derived class (or subclass).
- The class that is inherited from is called a base class (or superclass).

Assume that a base class named `driver::Gpio` has been created in order to easily control a GPIO pin on a microcontroller:

```
namespace driver
{
class Gpio
{
public:
    explicit Gpio(std::uint8_t pin) noexcept;
    ~Gpio() noexcept;

    [[nodiscard]] bool read() const noexcept;
    void write(bool enable) noexcept;

    Gpio() = delete;
    Gpio(const Gpio&) = delete;
    Gpio(Gpio&&) = delete;
    Gpio& operator=(const Gpio&) = delete;
    Gpio& operator=(Gpio&&) = delete;

protected:
    // Add GPIO-specific methods for initialization and such here.
    void init() noexcept;

    /** The pin the GPIO is connected to. */
    std::uint8_t myPin;
};
} // namespace driver
```

Note that the keyword `protected` is used instead of `private` in this case:

- The reason for this is that we want potential derived classes to be able to access the parts of the class that would otherwise be private (provided that so-called public inheritance is used).
- The difference between private and protected members is therefore that derived classes can access protected members, but not private ones. However, neither protected nor private members are accessible otherwise.

3.1.1 Types of inheritance

There are three types of inheritance in C++:

- **Public inheritance:** The by far most common type. Public and protected members in the base class retain their access rights in the derived class. Used when the derived class is a specialization of the base class.

- **Private inheritance:** All members of the base class become private in the derived class. Used when the base class should only be used internally within the derived class.
- **Protected inheritance:** Public and protected members in the base class become protected in the derived class. It therefore works like private inheritance, with the difference that potential subclasses of the current derived class also gain internal access to the base class.

Below is an example of public inheritance, where a class named `driver::Led` inherits the GPIO functionality from the class `driver::Gpio`:

- The functions `write` and `read` from the base class `driver::Gpio` can therefore be used by instances of the class `driver::Led`, i.e. LEDs.
- A new method, `toggle`, has been added to toggle the LED:

```
namespace driver
{
class Led final : public Gpio
{
public:
    explicit Led(std::uint8_t pin) noexcept;
    ~Led() noexcept;

    void toggle() noexcept;

    Led() = delete;
    Led(const Led&) = delete;
    Led(Led&&) = delete;
    Led& operator=(const Led&) = delete;
    Led& operator=(Led&&) = delete;
};
} // namespace driver
```

Note that:

- The class `driver::Led` publicly inherits the class `driver::Gpio` by writing `: public Gpio` after the class name. Since both classes are in the same namespace, we do not need to use the prefix `driver` before `Gpio`.
- The keyword `final` has been added directly after the class name `driver::Led` so that this class cannot be inherited. As a rule of thumb, it is a good idea to add `final` if the class is not intended to be inherited. However, in some cases it may be desirable to inherit the implementation for testing purposes, and in that case this keyword must be omitted.

By inheriting the class `driver::Gpio`, we avoided having to add a member variable that stores the LED's pin number as well as routines for writing to or reading the LED output signal. Instead, we can easily create an LED via the constructor of the class `driver::Led` and then use the methods `write()` and `read()` to control the LED:

```
// Create an LED connected to pin 9.
driver::Led led1{9U};

// Enable the LED.
led1.write(true);
```

In the same way, we could have created a derived class named `driver::Button` in order to control a button.

In this case, a method has been added to enable/disable interrupts when the push button is pressed. A corresponding method for checking whether interrupts are enabled has also been added. Since the button should not be controllable, we ensure that the method `write()` is private; we cannot remove inherited methods, but we can change their visibility with a using-declaration (`using Gpio::write;`) in the private section.

```
namespace driver
{
class Button final : public Gpio
{
public:
    explicit Button(std::uint8_t pin) noexcept;
    ~Button() noexcept;

    void enableInterrupt(bool enable) noexcept;
    [[nodiscard]] bool isInterruptEnabled() const noexcept;

    Button() = delete;
    Button(const Button&) = delete;
    Button(Button&&) = delete;
    Button& operator=(const Button&) = delete;
    Button& operator=(Button&&) = delete;

private:
    using Gpio::write;
};
} // namespace driver
```

Through this class, we can then easily read the button input signal, enable interrupts, and so on:

```
// Create a button connected to pin 13.
driver::Button button1{13U};

// Enable interrupt at pressdown.
```

```
button1.enableInterrupt(true);

// Toggle led1 if the button is pressed.
if (button1.read()) { led1.toggle(); }
```

3.2 Interfaces in C++

An interface in C++ is an abstract base class that (normally) only contains pure virtual methods, i.e. methods that have no implementation. The purpose of an interface is to define a common set of methods that different classes can implement in their own way depending on their needs.

3.2.1 Advantages of interfaces

Using interfaces has several advantages:

- **Abstraction:** You can write code that works against an abstraction rather than a concrete class. This makes it easier to replace implementations without changing the rest of the code.
- **Replaceability:** Different implementations can be used depending on the need, for example different hardware drivers or test classes.
- **Reusability:** Code that uses interfaces becomes more general and can be reused in different contexts.
- **Testability:** By using interfaces, you can easily create mock or fake classes for unit testing.

3.2.2 Structure of an interface

Below the structure of an interface for timer circuits is shown. This interface is named `driver::timer::Interface`. To keep the code compact, no comments are used in this example:

```
namespace driver::timer
{
class Interface
{
public:
    virtual ~Interface() noexcept = default;
    virtual void start() noexcept = 0;
    virtual void stop() noexcept = 0;
    [[nodiscard]] virtual bool isRunning() const noexcept = 0;
    virtual void reset() noexcept = 0;
};
```

```
} // namespace driver::timer
```

Through the interface above, a timer can be started, stopped, and reset via the methods `start()`, `stop()`, and `reset()`. It is also possible to check whether the timer is active or not via the method `isRunning()`.

A few things to note:

- The destructor is marked as `virtual`:
 - This ensures that the correct destructor is called when an instance of a subclass is deleted through a pointer or reference to the interface.
 - This is crucial for avoiding memory leaks and for correctly cleaning up resources in both the derived class and the base class.
- The destructor is defined as `= default`:
 - This results in an empty, automatic implementation.
 - It is good practice to declare the destructor as `virtual` in an interface. Otherwise, objects of subclasses that are deleted through a pointer or reference to the interface may exhibit undefined behavior.
 - Since the interface has nothing to clean up, a default implementation is sufficient here.
- All methods that are meant to be overridden in subclasses are marked with `virtual` and end with `= 0`:
 - This makes them pure virtual methods without implementation in the interface.
- All virtual methods are here marked `noexcept`:
 - This forces all concrete implementations in the subclasses to not be able to throw exceptions.
 - This is advantageous in an embedded system, but if you do not want to enforce this it is fine to omit `noexcept` here.
- The method `isRunning()` is marked `[[nodiscard]]`:
 - This encourages callers to use the returned value by generating a compiler warning if it is discarded.
 - The attribute belongs to the declaration it is written on, not to the class. It is therefore **not** inherited by overriding methods, and a call made directly on a concrete subclass whose implementation omits it produces no warning.
 - Therefore, subclasses **should** repeat the `[[nodiscard]]` attribute on their implementations. This is done throughout this book.
 - Note also that compiler support for `[[nodiscard]]` on virtual methods varies. Some compilers, such as GCC, do not warn at all when the return value of a *virtual* call is discarded, not even when the call is made through the interface. Repeating the attribute on the override is what makes the warning appear where it can.

3.2.3 Structure of a concrete subclass

Below is an example of a concrete subclass `driver::timer::Atmega328p`, which represents a concrete implementation for timer circuits for the ATmega328P microcontroller:

```
namespace driver::timer
{
class Atmega328p final : public Interface
{
public:
    // Example constructor.
    explicit Atmega328p(std::uint16_t duration_ms) noexcept;

    // Overridden methods.
    ~Atmega328p() noexcept override;
    void start() noexcept override;
    void stop() noexcept override;
    [[nodiscard]] bool isRunning() const noexcept override;
    void reset() noexcept override;

    // Additional ATmega328P-specific methods (if any).

    // Deleted constructors and assignment operators.
    Atmega328p() = delete;
    Atmega328p(const Atmega328p&) = delete;
    Atmega328p(Atmega328p&&) = delete;
    Atmega328p& operator=(const Atmega328p&) = delete;
    Atmega328p& operator=(Atmega328p&&) = delete;

private:
    // Timer implementation details.
};
} // namespace driver::timer
```

Some things we saw earlier in the section on inheritance (§3.1) also appear here:

- The keyword `final` is used to ensure that the class `driver::timer::Atmega328p` cannot be inherited; this is the final implementation:
 - As a rule of thumb, this is good to do if the class is not intended to be inherited.
 - However, in some cases it may be desirable to inherit the implementation for testing, and in that case this keyword must be omitted.
- The class `driver::timer::Atmega328p` inherits the class `driver::timer::Interface` by writing `: public Interface` after the class name. Writing `public` means that everything

that is public (or protected) in the interface is also public (or protected) in this derived class.

Some new things to note:

- The overridden methods are marked with `override` precisely to indicate that these are concrete implementations of overridden virtual methods.
- The destructor is also marked with `override` to clearly show that it replaces the virtual destructor in the interface. It is the virtual destructor in the interface that ensures the correct destructor is called during polymorphic destruction; `override` makes the compiler check that this is the case, since it is an error if the base class destructor is not virtual.
- The constructor has nothing to do with the interface and is therefore neither marked `virtual` nor `override`.

3.2.4 Example of using interfaces

Below an interface `driver::led::Interface` is shown. This interface should be usable for various LEDs, for example from different microprocessors:

```
/**
 * @file LED driver interface.
 */
#pragma once

#include <cstdint>

namespace driver::led
{
/**
 * @brief LED driver interface.
 */
class Interface
{
public:
/**
 * @brief Destructor.
 */
virtual ~Interface() noexcept = default;

/**
 * @brief Get the pin the LED is connected to.
 *
 * @return The pin the LED is connected to.
 */
}
```

```

[[nodiscard]] virtual std::uint8_t pin() const noexcept = 0;

/**
 * @brief Check whether the LED is enabled.
 *
 * @return True if the LED is enabled, false otherwise.
 */
[[nodiscard]] virtual bool isEnabled() const noexcept = 0;

/**
 * @brief Enable/disable the LED.
 *
 * @param[in] enable True if the LED is to be enabled, false otherwise.
 */
virtual void setEnabled(bool enable) noexcept = 0;

/**
 * @brief Toggle the LED.
 */
virtual void toggle() noexcept = 0;
};
} // namespace driver::led

```

Below the subclass `driver::led::Atmega328p` is shown, which inherits the interface `driver::led::Interface` and is implemented to easily control LEDs connected to the ATmega328P microcontroller:

```

namespace driver::led
{
/**
 * @brief LED driver for ATmega328P.
 *
 * This class is non-copyable and non-movable.
 */
class Atmega328p final : public Interface
{
public:
/**
 * @brief Constructor.
 *
 * @param[in] pin The pin the LED is connected to.
 */
explicit Atmega328p(std::uint8_t pin) noexcept;

```

```

/**
 * @brief Destructor.
 */
~Atmega328p() noexcept override;

/**
 * @brief Get the pin the LED is connected to.
 *
 * @return The pin the LED is connected to.
 */
[[nodiscard]] std::uint8_t pin() const noexcept override;

/**
 * @brief Check whether the LED is enabled.
 *
 * @return True if the LED is enabled, false otherwise.
 */
[[nodiscard]] bool isEnabled() const noexcept override;

/**
 * @brief Enable/disable the LED.
 *
 * @param[in] enable True if the LED is to be enabled, false otherwise.
 */
void setEnabled(bool enable) noexcept override;

/**
 * @brief Toggle the LED.
 */
void toggle() noexcept override;

Atmega328p() = delete; // No default constructor.
Atmega328p(const Atmega328p&) = delete; // No copy constructor.
Atmega328p(Atmega328p&&) = delete; // No move constructor.
Atmega328p& operator=(const Atmega328p&) = delete; // No copy assignment.
Atmega328p& operator=(Atmega328p&&) = delete; // No move assignment.

private:
    /** The pin the LED is connected to. */
    std::uint8_t myPin;

    /** Indicate whether the LED is enabled. */
    bool myIsEnabled;
};

```

```
} // namespace driver::led
```

By using pointers or references to `driver::led::Interface`, you can write code that works with all subclasses. As an example, a function named `blinkLed()` is demonstrated below, which is used to blink a given LED. Note that:

- The input argument `led` consists of a reference to a `driver::led::Interface`.
- The LED can therefore be an instance of any arbitrary subclass.
- For example, this could be an instance of the previously demonstrated class `driver::led::Atmega328p`, or alternatively it could be an instance of a class for an LED on an ESP32-S3 processor or similar.

```
/**
 * @brief Blink the given LED.
 *
 * @param[in, out] led The LED to blink.
 * @param[in] blinkTimeMs The blink time in milliseconds.
 */
void blinkLed(driver::led::Interface& led, const std::uint16_t blinkTimeMs)
{
    // Toggle the LED, then delay the calling thread.
    led.toggle();
    std::this_thread::sleep_for(std::chrono::milliseconds(blinkTimeMs));

    // Toggle the LED again, then delay the calling thread.
    led.toggle();
    std::this_thread::sleep_for(std::chrono::milliseconds(blinkTimeMs));
}
```

When the function above is called, we can use one of the subclasses directly, for example our class `driver::led::Atmega328p`. Assume that we have implemented an LED connected to pin 9 on an Arduino Uno via an instance named `led1`:

```
driver::led::Atmega328p led1{9U};
```

We can blink this LED with a blink time of, for example, 1000 ms by calling the function `blinkLed()`. Since `driver::led::Atmega328p` is a subclass of `driver::led::Interface`, we can pass `led1` directly:

```
blinkLed(led1, 1000U);
```

Assume that we have also created a subclass `driver::led::Esp32s3` to implement LEDs for an ESP32-S3 processor, as shown below.

Note that:

- The implementation is slightly different this time; most notably, the user has the possibility to set the LED's initial state directly via a call to the constructor.
- This class also has a private method named `init()`.
- This is demonstrated to show that subclasses can be tailored according to specific needs:

```
namespace driver::led
{
    /**
     * @brief LED driver for ESP32-S3.
     *
     *      This class is non-copyable and non-movable.
     */
    class Esp32s3 final : public Interface
    {
    public:
        /**
         * @brief Constructor.
         *
         * @param[in] pin The pin the LED is connected to.
         * @param[in] initialState Initial state of the LED (default = off).
         */
        explicit Esp32s3(std::uint8_t pin, bool initialState = false) noexcept;

        /**
         * @brief Destructor.
         */
        ~Esp32s3() noexcept override;

        /**
         * @brief Get the pin the LED is connected to.
         *
         * @return The pin the LED is connected to.
         */
        [[nodiscard]] std::uint8_t pin() const noexcept override;

        /**
         * @brief Check whether the LED is enabled.
         *
         * @return True if the LED is enabled, false otherwise.
         */
        [[nodiscard]] bool isEnabled() const noexcept override;
    };
}
```

```

/**
 * @brief Enable/disable the LED.
 *
 * @param[in] enable True if the LED is to be enabled, false otherwise.
 */
void setEnabled(bool enable) noexcept override;

/**
 * @brief Toggle the LED.
 */
void toggle() noexcept override;

Esp32s3() = delete; // No default constructor.
Esp32s3(const Esp32s3&) = delete; // No copy constructor.
Esp32s3(Esp32s3&&) = delete; // No move constructor.
Esp32s3& operator=(const Esp32s3&) = delete; // No copy assignment.
Esp32s3& operator=(Esp32s3&&) = delete; // No move assignment.

private:
/**
 * @brief Initialize the LED.
 */
void init(bool initialState) noexcept;

/** The pin the LED is connected to. */
std::uint8_t myPin;

/** Indicate whether the LED is enabled. */
bool myIsEnabled;
};
} // namespace driver::led

```

Assume that we have implemented an LED connected to pin 20 on an ESP32-S3 processor via an instance named `led2`. We turn on the LED directly at startup:

```
driver::led::Esp32s3 led2{20U, true};
```

We can also blink this LED by calling the function `blinkLed()`, since the class `driver::led::Esp32s3` is a subclass of `driver::led::Interface`. For example, to blink this LED with a blink time of 500 milliseconds, the following call can be made:

```
blinkLed(led2, 500U);
```

The full example is in §3.3.

See also the following C implementation to understand how interfaces work “under the hood” (from the previous embedded C course): https://github.com/qrtech-academy/embedded-c/blob/main/lectures/L10/interface_demo/README.md.

3.3 Example: An LED Interface in C++

This is a demonstration of an interface driver::led::Interface, which is used to easily work with different types of LEDs, for example LEDs from different processors, via subclasses. It is the complete program behind §3.2.4, and it lives in lectures/L03/examples/cpp_interface.

3.3.1 Files

- driver/led/interface.hpp constitutes the interface itself in the form of the base class driver::led::Interface.
- driver/led/atmega328p.hpp constitutes an implementation of LEDs for the processor ATmega328P via the subclass driver::led::Atmega328p. The corresponding source code is source/driver/led/atmega328p.cpp.
- driver/led/esp32s3.hpp constitutes an implementation of LEDs for the processor ESP32-S3 via the subclass driver::led::Esp32s3. The corresponding source code is source/driver/led/esp32s3.cpp.
- In main.cpp, LEDs for each processor are created (via the previously mentioned subclasses) and blink at different frequencies:
 - This is achieved through a shared blink function that expects a reference to an LED interface (driver::led::Interface).
 - Since both subclasses inherit driver::led::Interface, this works perfectly.

The interface

```
lectures/L03/examples/cpp_interface/include/driver/led/interface.hpp
/**
 * @file LED driver interface.
 */
#pragma once

#include <cstdint>

namespace driver::led
{
  /**
   * @brief LED driver interface.
   */
  class Interface
```

```

{
public:
    /**
     * @brief Destructor.
     */
    virtual ~Interface() noexcept = default;

    /**
     * @brief Get the pin the LED is connected to.
     *
     * @return The pin the LED is connected to.
     */
    [[nodiscard]] virtual std::uint8_t pin() const noexcept = 0;

    /**
     * @brief Check whether the LED is enabled.
     *
     * @return True if the LED is enabled, false otherwise.
     */
    [[nodiscard]] virtual bool isEnabled() const noexcept = 0;

    /**
     * @brief Enable/disable the LED.
     *
     * @param[in] enable True if the LED is to be enabled, false otherwise.
     */
    virtual void setEnabled(bool enable) noexcept = 0;

    /**
     * @brief Toggle the LED.
     */
    virtual void toggle() noexcept = 0;
};
} // namespace driver::led

```

The ATmega328P implementation

```

lectures/L03/examples/cpp_interface/include/driver/led/atmega328p.hpp
/**
 * @file LED driver for ATmega328P.
 */
#pragma once

```

```

#include <stdint>

#include "driver/led/interface.hpp"

namespace driver::led
{
/**
 * @brief LED driver for ATmega328P.
 *
 * This class is non-copyable and non-movable.
 */
class Atmega328p final : public Interface
{
public:
/**
 * @brief Constructor.
 *
 * @param[in] pin The pin the LED is connected to.
 */
explicit Atmega328p(std::uint8_t pin) noexcept;

/**
 * @brief Destructor.
 */
~Atmega328p() noexcept override;

/**
 * @brief Get the pin the LED is connected to.
 *
 * @return The pin the LED is connected to.
 */
[[nodiscard]] std::uint8_t pin() const noexcept override;

/**
 * @brief Check whether the LED is enabled.
 *
 * @return True if the LED is enabled, false otherwise.
 */
[[nodiscard]] bool isEnabled() const noexcept override;

/**
 * @brief Enable/disable the LED.
 *
 * @param[in] enable True if the LED is to be enabled, false otherwise.

```

```

    */
    void setEnabled(bool enable) noexcept override;

    /**
     * @brief Toggle the LED.
     */
    void toggle() noexcept override;

    Atmega328p() = delete; // No default constructor.
    Atmega328p(const Atmega328p&) = delete; // No copy constructor.
    Atmega328p(Atmega328p&&) = delete; // No move constructor.
    Atmega328p& operator=(const Atmega328p&) = delete; // No copy assignment.
    Atmega328p& operator=(Atmega328p&&) = delete; // No move assignment.

private:
    /** The pin the LED is connected to. */
    std::uint8_t myPin;

    /** Indicate whether the LED is enabled. */
    bool myIsEnabled;
};
} // namespace driver::led

```

```

lectures/L03/examples/cpp_interface/source/driver/led/atmega328p.cpp
/**
 * @file LED driver implementation details for ATmega328P.
 */
#include <cstdint>
#include <iostream>

#include "driver/led/atmega328p.hpp"

namespace driver::led
{
    // -----
    Atmega328p::Atmega328p(const std::uint8_t pin) noexcept
        : myPin{pin}
        , myIsEnabled{false}
    {
        // Insert code to initialize the hardware here.
        std::cout << "Initializing LED connected to pin " << static_cast<int>(myPin)
            << " on processor ATmega328P!\n";
    }
}

```

```

// -----
Atmega328p::~Atmega328p() noexcept
{
    // Insert code to release allocated resources here.
    std::cout << "Deleting LED connected to pin " << static_cast<int>(myPin)
               << " on processor ATmega328P!\n";
}

// -----
std::uint8_t Atmega328p::pin() const noexcept { return myPin; }

// -----
bool Atmega328p::isEnabled() const noexcept { return myIsEnabled; }

// -----
void Atmega328p::setEnabled(const bool enable) noexcept
{
    // Insert code to enable the LED here.
    myIsEnabled = enable;
    const auto status{myIsEnabled ? "Enabling" : "Disabling"};
    std::cout << status << " LED connected to pin " << static_cast<int>(myPin) << "\n";
}

// -----
void Atmega328p::toggle() noexcept { setEnabled(!myIsEnabled); }

} // namespace driver::led

```

The ESP32-S3 implementation

```

lectures/L03/examples/cpp_interface/include/driver/led/esp32s3.hpp
/**
 * @file LED driver for ESP32-S3.
 */
#pragma once

#include <cstdint>

#include "driver/led/interface.hpp"

namespace driver::led

```

```

{
/**
 * @brief LED driver for ESP32-S3.
 *
 *      This class is non-copyable and non-movable.
 */
class Esp32s3 final : public Interface
{
public:
/**
 * @brief Constructor.
 *
 * @param[in] pin The pin the LED is connected to.
 * @param[in] initialState Initial state of the LED (default = off).
 */
explicit Esp32s3(std::uint8_t pin, bool initialState = false) noexcept;

/**
 * @brief Destructor.
 */
~Esp32s3() noexcept override;

/**
 * @brief Get the pin the LED is connected to.
 *
 * @return The pin the LED is connected to.
 */
[[nodiscard]] std::uint8_t pin() const noexcept override;

/**
 * @brief Check whether the LED is enabled.
 *
 * @return True if the LED is enabled, false otherwise.
 */
[[nodiscard]] bool isEnabled() const noexcept override;

/**
 * @brief Enable/disable the LED.
 *
 * @param[in] enable True if the LED is to be enabled, false otherwise.
 */
void setEnabled(bool enable) noexcept override;

/**

```

```

    * @brief Toggle the LED.
    */
    void toggle() noexcept override;

    Esp32s3() = delete; // No default constructor.
    Esp32s3(const Esp32s3&) = delete; // No copy constructor.
    Esp32s3(Esp32s3&&) = delete; // No move constructor.
    Esp32s3& operator=(const Esp32s3&) = delete; // No copy assignment.
    Esp32s3& operator=(Esp32s3&&) = delete; // No move assignment.

private:
    /**
     * @brief Initialize the LED.
     */
    void init(bool initialState) noexcept;

    /** The pin the LED is connected to. */
    std::uint8_t myPin;

    /** Indicate whether the LED is enabled. */
    bool myIsEnabled;
};
} // namespace driver::led

```

```

lectures/L03/examples/cpp_interface/source/driver/led/esp32s3.cpp
/**
 * @file LED driver implementation details for ESP32-S3.
 */
#include <stdint>
#include <iostream>

#include "driver/led/esp32s3.hpp"

namespace driver::led
{
    // -----
    Esp32s3::Esp32s3(const std::uint8_t pin, const bool initialState) noexcept
        : myPin{pin}
        , myIsEnabled{false}
    {
        init(initialState);
    }
}

```

```

// -----
Esp32s3::~Esp32s3() noexcept
{
    // Insert code to release allocated resources here.
    std::cout << "Deleting LED connected to pin " << static_cast<int>(myPin)
               << " on processor ESP32-S3!\n";
}

// -----
std::uint8_t Esp32s3::pin() const noexcept { return myPin; }

// -----
bool Esp32s3::isEnabled() const noexcept { return myIsEnabled; }

// -----
void Esp32s3::setEnabled(const bool enable) noexcept
{
    // Insert code to enable the LED here.
    myIsEnabled = enable;
    const auto status{myIsEnabled ? "Enabling" : "Disabling"};
    std::cout << status << " LED connected to pin " << static_cast<int>(myPin) << "\n";
}

// -----
void Esp32s3::toggle() noexcept { setEnabled(!myIsEnabled); }

// -----
void Esp32s3::init(const bool initialState) noexcept
{
    // Insert code to initialize the hardware here.
    std::cout << "Initializing LED connected to pin " << static_cast<int>(myPin)
               << " on processor ESP32-S3, starting value = " << initialState << "\n";
    setEnabled(initialState);
}
} // namespace driver::led

```

The program

```

lectures/L03/examples/cpp_interface/source/main.cpp
/**
 * @file Demonstration of interfaces for LEDs.

```

```

    */
#include <chrono>
#include <cstdlib>
#include <thread>

#include "driver/led/atmega328p.hpp"
#include "driver/led/esp32s3.hpp"

using namespace driver;

namespace
{
    // -----
    void delayMs(const std::uint16_t durationMs) noexcept
    {
        std::this_thread::sleep_for(std::chrono::milliseconds(durationMs));
    }

    // -----
    void blinkLed(driver::led::Interface& led, const std::uint16_t blinkTimeMs)
        noexcept
    {
        // Toggle the LED, then delay the calling thread.
        led.toggle();
        delayMs(blinkTimeMs);

        // Toggle the LED again, then delay the calling thread.
        led.toggle();
        delayMs(blinkTimeMs);
    }
} // namespace

/**
 * @brief Blink LEDs connected to different microprocessors.
 *
 * @return 0 on termination of the program.
 */
int main()
{
    // Connect LED1 to pin 9 of the ATmega328P processor.
    driver::led::Atmega328p led1{9U};

    // Connect LED2 to pin 20 of the ESP32-S3 processor, enable the LED on startup-
    .

```

```

driver::led::Esp32s3 led2{20U, true};

// Blink the LEDs ten times, then terminate the program.
constexpr std::uint16_t iterations{10U};

for (std::uint16_t i{}; i < iterations; ++i)
{
    blinkLed(led1, 1000U);
    blinkLed(led2, 500U);
}
return 0;
}

```

3.3.2 Compiling and running the program

The example is built with the Makefile from §1.1.5, extended with the three source files and the include directory:

```

lectures/L03/examples/cpp_interface/makefile
# Target application.
TARGET := main

# C++ compiler.
CXX_COMPILER := g++

# C++ compiler flags.
CXX_FLAGS := -Wall -Werror -std=c++17 -Iinclude

# Source files.
SRC_FILES := source/driver/led/atmega328p.cpp\
              source/driver/led/esp32s3.cpp\
              source/main.cpp\

# Build and run the application as default.
default: build run

# Build the application.
build:
    @$(CXX_COMPILER) $(SRC_FILES) -o $(TARGET) $(CXX_FLAGS)

# Run the application.
run:
    @./$(TARGET)

```

```
# Clean the application.  
clean:  
    @rm -f $(TARGET)  
  
.PHONY: default build run clean
```

Compile and run the program with the following command (in this directory):

```
make
```

You can also build the program without running it afterwards with the following command:

```
make build
```

You can run the program without compiling beforehand with the following command:

```
make run
```

You can also remove compiled files with the following command:

```
make clean
```

3.4 Review

What you should be able to do now

After this chapter, you should:

- Be able to create classes that inherit from other classes.
- Be able to create interfaces.
- Understand how several different implementations can be used through the same interface.

Questions to test yourself

You should be able to explain:

1. What does inheritance mean in C++?
2. What is an abstract class?
3. Why can it be useful to use an interface for a driver?

Looking ahead

Chapter 4 is about the factory pattern in C++, with raw pointers first and then with smart pointers.

3.5 Exercises

How to work these. These exercises reinforce the concepts from §3.1 and §3.2. Unlike the earlier chapters, all four sets build *one* program: an interface, two drivers implementing it, and a `main()` that uses both through the interface. Write it in `lectures/L03/exercises` in the companion repository, in the directory structure below, and build it with the Makefile from §1.1.5, listing the three source files and adding `-Iinclude` to the compiler flags.

How to check your work. Run `make test` from the repository root. Each set's tests switch on as soon as its files exist, so the interface can be checked before any driver is written, and they compare what the program prints with the output below, character for character; a set you have not started is reported as SKIP.

The solution is published in the companion repository, as one program in `lectures/L03/appendix/solutions`. Try each exercise yourself before looking at it.

Create the following directory structure:

```
Makefile
include/
  driver/
    serial/
      console.hpp
      interface.hpp
      stub.hpp
source/
  driver/
    serial/
      console.cpp
      stub.cpp
main.cpp
```

Note: When a class overrides a method that is marked `[[nodiscard]]` in the interface, repeat the attribute on the overriding method as well. The attribute is not inherited; see §3.2.2.

EXERCISE SET 1

Serial Interface

Exercise 3.1 Interface for serial communication

CODE

In this exercise you will design an interface `driver::serial::Interface` representing a generic serial communication driver.

Tasks

In `driver/serial/interface.hpp`, design a class named `driver::serial::Interface` that represents a generic serial driver.

All methods except the destructor shall be purely virtual (`= 0`).

a) Destructor

Add a destructor that is:

- Virtual.
- Marked `noexcept`.
- Implemented using `= default`.

b) Initialization status

Add a pure virtual method `isInitialized()` that:

- Indicates whether the driver has been initialized (`true/false`).
- Does not modify the object (`const`).
- Cannot throw exceptions.
- Generates a warning if the return value is discarded (`[[nodiscard]]`).

c) Transmitting data

Add a pure virtual method `write()` that:

- Transmits one byte of data.
- Takes a parameter of type `std::uint8_t`.
- Does not return a value.
- Cannot throw exceptions.

d) Receiving data

Add a pure virtual method `read()` that:

- Attempts to read one byte from the serial driver.
- Takes no parameters.
- Returns a `std::optional<std::uint8_t>` from `<optional>`: the received byte, or `std::nullopt` if no byte was received.
- Cannot throw exceptions.

- Generates a warning if the return value is discarded (`[[nodiscard]]`): a byte that is read and then discarded is lost.

EXERCISE SET 2

Stub Implementation

Exercise 3.2 Serial stub

CODE

In `driver/serial/stub.hpp`, implement a stub driver `driver::serial::Stub` for the serial interface.

A stub driver simulates hardware behaviour and is useful for testing without real hardware.

The class shall:

- Inherit from `driver::serial::Interface`.
- Be marked `final`.

Method definitions shall be placed in `driver/serial/stub.cpp`.

a) Member variables

Add two private member variables:

- The first member variable shall:
 - Store the most recently transmitted byte, until it is read.
 - Have the type `std::optional<std::uint8_t>`.
 - Be named `myLastByte`.
 - Be empty when no byte is available to read.
- The second member variable shall:
 - Indicate whether the driver is initialized.
 - Have the type `bool`.
 - Be named `myInitialized`.

b) Constructor

Add a default constructor that:

- Sets the driver as initialized.
- Leaves `myLastByte` empty, since no data is available to read.

c) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Copy constructor.
- Move constructor.
- Copy assignment operator.

- Move assignment operator.

d) Implement interface methods

Implement all methods required by the interface:

- The method `isInitialized()` shall:
 - Return the value stored in `myInitialized`.
- The method `write()` shall:
 - Do nothing if `myInitialized` is false.
 - Otherwise store the transmitted byte in `myLastByte`.
- The method `read()` shall:
 - Return the content of `myLastByte`, which is `std::nullopt` if no byte is available.
 - Leave `myLastByte` empty, so that a byte can only be read once.

e) Initialization method

Add an additional public method `setInitialized()` that is not part of the interface.

This method shall:

- Be used to simulate the initialization state.
- Take the initialization state as input.
- Discard the stored byte, leaving `myLastByte` empty, when the driver is set to uninitialized.
- Not return a value.

EXERCISE SET 3

Singleton Driver

Exercise 3.3 Console serial driver

CODE

In `driver/serial/console.hpp`, create a real driver implementation `driver::serial::Console` that writes transmitted bytes to the system console.

This driver shall follow the so-called singleton pattern, i.e. there is only one instance of the class.

The reason is that the system only has one console output device.

This class shall:

- Inherit from `driver::serial::Interface`.
- Be marked `final`.

Method definitions shall be placed in `driver/serial/console.cpp`.

a) Private constructor

Add a private default constructor that prevents users from directly creating objects of the class.

b) Private destructor

Add a private destructor, mark it `noexcept` and `default`.

c) Prevent multiple instances

Ensure that only one instance of the class can exist.

Delete the following functions (in the public section of the class):

- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

d) Singleton access method

Add a static public method `instance()` that returns a reference to the unique instance of the class.

Requirements:

- The method shall return the console driver as a reference to `driver::serial::Interface` (even though the instance itself is of type `driver::serial::Console`).
- The method cannot throw exceptions.
- In this method:
 - Create a local static instance of the driver (`static driver::serial::Console`).
 - Initialize the instance using the default constructor.

e) Implement interface methods

Implement all methods required by the interface:

- The method `isInitialized()` shall:
 - Always return `true`, since the console driver is always available.
- The method `write()` shall:
 - Cast the given byte to a character and print it to the console using `std::printf()` from `<cstdio>`.
- The method `read()` shall:
 - Always return `std::nullopt`, since the console does not support reading.

EXERCISE SET 4

Using the Interface

Exercise 3.4 Sending a message

CODE

a) The `sendMessage()` function

In source/main.cpp, create a function `sendMessage()` that sends a message using the serial interface.

The function shall:

- Take a reference to `driver::serial::Interface`.
- Transmit the string "Transmitting data with a serial driver!".
- Send the message one byte at a time.

b) Testing the console driver

In function `main()`, test the console driver:

- Obtain the singleton instance of the console driver.
- Pass it to the `sendMessage()` function.
- Verify that the program prints the following:

```
Transmitting data with a serial driver!
```

c) Testing the stub driver

In function `main()`, test the stub driver:

- Create an instance of the stub driver and pass it to the function `sendMessage()`.
- Verify that the last transmitted byte `!` is returned by the `read()` method, and that a second call to `read()` returns `std::nullopt`.

The Factory Pattern

Letting a factory choose the hardware, so the logic never has to.

IN THIS CHAPTER

- The motivation behind the factory pattern, specifically the Abstract Factory pattern.
- Implementing factories using raw and smart pointers.
- How system logic uses factories to create drivers.

This chapter is about what the factory pattern is and why it is used. A factory is what lets the same system logic run on real drivers or on stubs: the logic creates its drivers through the factory, depends only on their interfaces, and never names a concrete driver type. By the end of the chapter you should be able to read and follow a simple factory example, and say who owns the objects it creates.

It builds directly on Chapters 2 and 3. You need the classes and interfaces from those chapters, and the ideas of a stub and of system logic, which this chapter puts to work.

Work the chapter in order. §4.1 builds a factory that returns raw pointers, which makes plain what is allocated and who must delete it; §4.2 replaces those raw pointers with `std::unique_ptr` and arrives at a simplified Abstract Factory. The exercises in §4.4 build the same design for a serial driver, first with raw pointers and then with smart ones. Their solutions are in the companion repository.

4.1 A Simple Driver Factory with Raw Pointers

This section shows a simplified example of how a factory can be used to create drivers in an embedded system.

The purpose is to demonstrate:

- What a factory is.
- Why we use it.
- How it makes the system logic independent of hardware.

4.1.1 Raw pointers and dynamic memory allocation

In this example, so-called raw pointers are used to clarify what happens. Raw pointers are obtained when we use the operators `new` and `delete` to allocate and free objects respectively:

- These operators serve a similar function to `malloc()` and `free()` in C, but in C++ `new` also calls the object's constructor and `delete` its destructor.
- Just as when using `malloc()` and `free()`, we are responsible for freeing allocated resources ourselves.
- In modern C++, smart pointers are used instead, which ensure that resources are released via the pointer's destructor when it goes out of scope, preventing memory leaks.
- Smart pointers are discussed in §4.2, where these raw pointers are replaced with `std::unique_ptr`.

4.1.2 Background

Assume that we have implemented the following drivers as classes:

- GPIO: `driver::gpio::Interface` with the following concrete implementations:
 - `driver::gpio::Esp32s3`: GPIO driver for the ESP32-S3 microcontroller.
 - `driver::gpio::Stub`: GPIO stub driver.

Assume that we have created the following class `app::logic::Logic` to handle the system logic:

- The system logic uses an LED and a button, where the LED is toggled on the rising edge of the push button.
- Interfaces are used so that the hardware used can be selected generically, i.e. the concrete types are chosen by the user.
- The method `run()` is used to execute the system logic continuously.

```
namespace app::logic
{
class Logic final
{
public:
    explicit Logic(driver::gpio::Interface& led,
                  driver::gpio::Interface& button) noexcept
        : myLed{led}
        , myButton{button}
    {}

    ~Logic() noexcept
    {
        myLed.write(false);
    }
};
}
```

```

    }

    void run() noexcept
    {
        bool buttonPrev{false};

        while (1)
        {
            const bool buttonCurrent{myButton.read()};
            if (buttonCurrent && !buttonPrev)
            {
                myLed.toggle();
            }
            buttonPrev = buttonCurrent;
        }
    }

    Logic() = delete;
    Logic(const Logic&) = delete;
    Logic(Logic&&) = delete;
    Logic& operator=(const Logic&) = delete;
    Logic& operator=(Logic&&) = delete;

private:
    driver::gpio::Interface& myLed;
    driver::gpio::Interface& myButton;
};
} // namespace app::logic

```

The example above can be implemented by creating drivers of any concrete types before the system logic is created, as shown below:

```

int main()
{
    constexpr std::uint8_t ledPin{2U};
    constexpr std::uint8_t buttonPin{3U};

    // Create ESP32-S3 drivers (ignoring data direction for simplicity).
    driver::gpio::Esp32s3 led{ledPin};
    driver::gpio::Esp32s3 button{buttonPin};

    // Create and run system logic.
    app::logic::Logic logic{led, button};
}

```

```

    logic.run();
}

```

The method above is very efficient when dynamic memory allocation should be limited.

However, it is often desirable to create multiple drivers of the same type, for example drivers for a specific microcontroller. For this reason, it is common to implement classes that create such instances. Such a class is called a **factory**, and its sole purpose is to create objects.

Assume that the following factories have been implemented:

- `driver::factory::Interface` with the following concrete implementations:
 - `driver::factory::Esp32s3`: Creates ESP32-S3 instances.
 - `driver::factory::Stub`: Creates stub instances.

Assume that the constructor for `app::logic::Logic` had taken a reference to an arbitrary factory:

```

explicit Logic(driver::factory::Interface& factory,
               const std::uint8_t ledPin,
               const std::uint8_t buttonPin) noexcept;

```

Assuming that the member variables had been implemented as pointers instead of references:

```

driver::gpio::Interface* myLed;
driver::gpio::Interface* myButton;

```

and the factory had created raw pointers, the instances could be created as shown below:

```

explicit Logic(driver::factory::Interface& factory,
               const std::uint8_t ledPin,
               const std::uint8_t buttonPin) noexcept
    : myLed{factory.gpio(ledPin)}
    , myButton{factory.gpio(buttonPin)}
{}

```

As an example, `Esp32s3` instances could be created by passing a `driver::factory::Esp32s3` to the system logic when it is created:

```

constexpr std::uint8_t ledPin{2U};
constexpr std::uint8_t buttonPin{3U};

// Create ESP32-S3 factory.
driver::factory::Esp32s3 factory{};

```

```
// Create and run system logic.
app::logic::Logic logic{factory, ledPin, buttonPin};
logic.run();
```

This moves the creation of drivers away from the system logic. Instead, the factory is responsible for selecting and creating the correct concrete implementation. The system logic can therefore operate on interfaces without knowing the underlying hardware.

4.1.3 Dynamic memory allocation in the factory

When instances are created in this way, they normally need to be allocated dynamically, unless the factory instead returns a reference to a singleton class.

This can be done using either:

- Raw pointers (`new` and `delete`).
- Smart pointers (`std::make_unique()`).

Instead of allocating an instance of `driver::gpio::Esp32s3` dynamically ourselves:

```
auto* gpio = new driver::gpio::Esp32s3(pin);
```

we let a factory perform this task:

```
auto* gpio = factory.gpio(pin);
```

This means that:

- The system logic does not know which concrete class is used.
- It only knows that it receives an object that satisfies a certain interface.

4.1.4 Why do we use a factory?

In many embedded systems, a factory plays a role similar to a hardware abstraction layer (HAL) or a board support package (BSP); it centralizes the creation of hardware-specific objects.

In an embedded system we often want to be able to:

Situation	What we want to do
Real hardware	Use real drivers
Testing/simulation	Use stubs
New platform	Replace drivers

If the system logic creates drivers itself:

```
auto* led = new driver::gpio::Esp32s3(pin);
```

then:

- The logic is tied to ESP32-S3.
- The system cannot be tested without hardware.

With a factory:

```
auto* led = factory.gpio(pin);
```

we can switch between:

```
driver::factory::Esp32s3 factory{};
```

and:

```
driver::factory::Stub factory{};
```

without changing the system logic.

This is called *dependency injection via factory*.

4.1.5 Overview of the architecture in a typical embedded system

Assume that we have an embedded system where the software is divided into four layers:



The system logic therefore does not know whether it is running on real hardware or on stubs. It only knows about the interfaces.

4.1.6 Example implementation using a factory

Step 1: Interface for GPIO

We create a simple GPIO interface in a file `driver/gpio/interface.hpp`, as shown below:

```
#pragma once

namespace driver::gpio
{
class Interface
{
public:
    virtual ~Interface() noexcept = default;
    [[nodiscard]] virtual bool read() const noexcept = 0;
    virtual void write(bool state) noexcept = 0;
    virtual void toggle() noexcept = 0;
};
} // namespace driver::gpio
```

Step 2: Real driver

We create a GPIO driver for the ESP32-S3 in a file `driver/gpio/esp32s3.hpp`, as shown below:

```
#pragma once

#include <cstdint>

#include "driver/gpio/interface.hpp"

namespace driver::gpio
{
class Esp32s3 final : public Interface
{
public:
    explicit Esp32s3(const std::uint8_t pin) noexcept
        : myPin{pin}
    {
        // ESP-specific initialization here!
    }

    ~Esp32s3() noexcept override
    {
        // ESP-specific cleanup here!
    }
};
}
```

```

    }

    [[nodiscard]] bool read() const noexcept override
    {
        // ESP-specific code here!
        return false;
    }

    void write(const bool state) noexcept override
    {
        // ESP-specific code here!
        (void) (state);
    }

    void toggle() noexcept override
    {
        // ESP-specific code here!
    }

    Esp32s3() = delete;
    Esp32s3(const Esp32s3&) = delete;
    Esp32s3(Esp32s3&&) = delete;
    Esp32s3& operator=(const Esp32s3&) = delete;
    Esp32s3& operator=(Esp32s3&&) = delete;

private:
    const std::uint8_t myPin;
};
} // namespace driver::gpio

```

Step 3: Stub

We create a GPIO stub in a file `driver/gpio/stub.hpp`, as shown below:

```

#pragma once

#include <cstdint>

#include "driver/gpio/interface.hpp"

namespace driver::gpio
{
    class Stub final : public Interface
    {

```

```

public:
    Stub() noexcept
        : myState{false}
    {}
    ~Stub() noexcept override = default;

    [[nodiscard]] bool read() const noexcept override { return myState; }
    void write(const bool state) noexcept override { myState = state; }
    void toggle() noexcept override { myState = !myState; }

    Stub(const Stub&) = delete;
    Stub(Stub&&) = delete;
    Stub& operator=(const Stub&) = delete;
    Stub& operator=(Stub&&) = delete;

private:
    bool myState;
};
} // namespace driver::gpio

```

Step 4: Factory interface

We create a factory interface in a file `driver/factory/interface.hpp` in order to implement separate factories:

- A factory for creating instances for ESP32-S3.
- A factory for creating stub instances.

```

#pragma once

#include <cstdint>

namespace driver
{
    /** GPIO driver interface. */
    namespace gpio { class Interface; }
} // namespace driver

namespace driver::factory
{
    class Interface
    {
    public:
        virtual ~Interface() noexcept = default;
    };
}

```

```

    [[nodiscard]] virtual gpio::Interface* gpio(std::uint8_t pin) noexcept = 0;
};
} // namespace driver::factory

```

Note that `gpio()` is marked `[[nodiscard]]`: it allocates a new object with `new` and hands ownership to the caller. Discarding the returned pointer would lose the only reference to that object, leaking it — there would be no way to delete it later.

Step 5: Real factory

We then create an ESP32-S3 factory in a file `driver/factory/esp32s3.hpp` in order to construct ESP32 instances:

```

#pragma once

#include <cstdint>

#include "driver/factory/interface.hpp"
#include "driver/gpio/esp32s3.hpp"

namespace driver::factory
{
    class Esp32s3 final : public Interface
    {
    public:
        Esp32s3() noexcept = default;
        ~Esp32s3() noexcept override = default;

        [[nodiscard]] gpio::Interface* gpio(const std::uint8_t pin) noexcept override
        {
            return new gpio::Esp32s3(pin);
        }

        Esp32s3(const Esp32s3&) = delete;
        Esp32s3(Esp32s3&&) = delete;
        Esp32s3& operator=(const Esp32s3&) = delete;
        Esp32s3& operator=(Esp32s3&&) = delete;
    };
} // namespace driver::factory

```

Step 6: Stub factory

We also create a stub factory in a file `driver/factory/stub.hpp` in order to construct stub instances:

```

#pragma once

#include <cstdint>

#include "driver/factory/interface.hpp"
#include "driver/gpio/stub.hpp"

namespace driver::factory
{
class Stub final : public Interface
{
public:
    Stub() noexcept = default;
    ~Stub() noexcept override = default;

    [[nodiscard]] gpio::Interface* gpio(const std::uint8_t pin) noexcept override
    {
        // Ignore the pin number since it is not used by the stub.
        (void) (pin);
        return new gpio::Stub();
    }

    Stub(const Stub&) = delete;
    Stub(Stub&&) = delete;
    Stub& operator=(const Stub&) = delete;
    Stub& operator=(Stub&&) = delete;
};
} // namespace driver::factory

```

Step 7: System logic using a factory

We then create a logic class that uses a given factory to create instances, in a file `app/logic/logic.hpp`:

- In the constructor we allocate memory for the GPIO instances `myLed` and `myButton` via the factory.
- In the destructor we free the allocated resources, i.e. the GPIO instances — since we have allocated memory using raw pointers we are responsible for doing this ourselves.
- In the method `run()`, `myLed` is toggled on the rising edge of the push button.
- Because `myLed` and `myButton` are pointers, the arrow operator `->` is used to call their methods.
- The copy and move operations are deleted. This is particularly important here, because `Logic` owns two raw pointers and deletes them in its destructor. Had we not deleted them, the compiler would have generated a copy constructor that copies the two *pointer*

values, so a copy would refer to the exact same two GPIO objects. Both objects would then delete them when destroyed, which means each GPIO object would be deleted twice; a **double deletion**, which is undefined behavior, would occur. The same applies to the copy assignment operator.

Note: This is a general rule, and it is sometimes referred to as the *rule of three*: a class that needs a destructor almost always needs its copy operations written or deleted as well. For a class such as `Logic`, which represents one system with one set of drivers, deleting them is the right choice.

In this example, the system logic owns the driver objects. The pointers returned by the factory are therefore owned by `Logic`, which means that the system logic is also responsible for deleting them in the destructor:

```
#pragma once

#include <stdint>

#include "driver/factory/interface.hpp"
#include "driver/gpio/interface.hpp"

namespace app::logic
{
class Logic final
{
public:
    explicit Logic(driver::factory::Interface& factory,
                  const std::uint8_t ledPin, const std::uint8_t buttonPin)
        noexcept
        : myLed{factory.gpio(ledPin)}
        , myButton{factory.gpio(buttonPin)}
    {}

    ~Logic() noexcept
    {
        delete myLed;
        delete myButton;
    }

    void run() noexcept
    {
        bool buttonPrev{false};

        while (1)
        {
```

```

        // Read the current button input.
        const bool buttonCurrent{myButton->read()};

        // Toggle the LED on pressdown (rising edge).
        if (buttonCurrent && !buttonPrev)
        {
            myLed->toggle();
        }
        // Store the current button input for next comparison.
        buttonPrev = buttonCurrent;
    }
}

// Deleting the copy operations is essential here: a copy would hold the same ↵
↵ two pointers,
// and both objects would delete the same GPIO instances (double deletion).
Logic()                = delete; // No default constructor.
Logic(const Logic&)     = delete; // No copy constructor.
Logic(Logic&&)          = delete; // No move constructor.
Logic& operator=(const Logic&) = delete; // No copy assignment.
Logic& operator=(Logic&&)    = delete; // No move assignment.

private:
    driver::gpio::Interface* myLed;
    driver::gpio::Interface* myButton;
};
} // namespace app::logic

```

Step 8: *main*

In the function `main`, we create a factory and pass it to the system logic to create the objects.

Below is an example of how a factory could be used to run the system logic on an ESP32-S3 by passing an instance of `driver::factory::Esp32s3` to the system logic, followed by calling the method `run()`:

```

#include <cstdint>

#include "driver/factory/esp32s3.hpp"
#include "app/logic/logic.hpp"

int main()
{
    constexpr std::uint8_t ledPin{2U};
    constexpr std::uint8_t buttonPin{3U};

```

```

// Create system logic and initialize the system.
driver::factory::Esp32s3 factory{};
app::logic::Logic logic{factory, ledPin, buttonPin};

// Run the system continuously.
logic.run();
return 0;
}

```

To run with stub drivers, it is sufficient to switch to the stub factory `driver::factory::Stub`, as shown below. This is well suited for testing and will be demonstrated in a later QAcademy course on software testing:

```

#include <cstdint>

#include "driver/factory/stub.hpp"
#include "app/logic/logic.hpp"

int main()
{
    constexpr std::uint8_t ledPin{2U};
    constexpr std::uint8_t buttonPin{3U};

    // Create system logic and initialize the system.
    driver::factory::Stub factory{};
    app::logic::Logic logic{factory, ledPin, buttonPin};

    // Run the system continuously.
    logic.run();
    return 0;
}

```

4.2 Driver Factory with Smart Pointers (Abstract Factory)

In this section we further develop the factory example from §4.1.

We replace raw pointers with smart pointers and use a simplified form of the **Abstract Factory pattern**.

The purpose is to:

- Avoid manual memory management.
- Clarify ownership.
- Make the system safer and more robust.

4.2.1 What is Abstract Factory?

Abstract Factory is a design pattern where:

- A common factory interface defines how objects are created.
- Multiple concrete factories implement this interface.
- Each factory creates drivers for a specific platform or environment.

In our case:

Factory	What it creates
Esp32s3	Real hardware drivers (for ESP32-S3)
Stub	Simulated drivers for testing

The system logic only uses the factory interface and does not know which concrete factory is used.

4.2.2 Why do we switch to smart pointers?

In the previous example with raw pointers, the factory created objects using the `new` operator:

```
return new gpio::Esp32s3(pin);
```

The system logic had to delete the objects manually using the `delete` operator:

```
delete myLed;
```

Using raw pointers introduces several problems:

- It is easy to forget to call `delete`, which leads to memory leaks.
- Ownership of the object is unclear.
- Manual memory management makes programs harder to write correctly.

A modern C++ solution is to use `std::unique_ptr` from the `<memory>` header instead of raw pointers.

A `std::unique_ptr`:

- Represents exclusive ownership.
- Deletes the object automatically.
- Cannot be copied, only moved.

The `std::unique_ptr` returned by the factory transfers ownership of the created object to the caller.

When the `std::unique_ptr` goes out of scope, the object is automatically destroyed.

If we use smart pointers, the factory can instead create objects by calling the `std::make_unique()` function from `<memory>`:

```
return std::make_unique<gpio::Esp32s3>(pin);
```

std::make_unique():

- Creates the object.
- Returns a std::unique_ptr.
- Ensures safe and clear construction.

4.2.3 Exceptions and noexcept

In C++, memory allocation can fail. When we use new or std::make_unique(), an exception (std::bad_alloc) may be thrown if the system cannot allocate memory.

In embedded systems, exceptions are often not used, or a predictable behavior is preferred when errors occur. Therefore noexcept is still often used in driver and factory interfaces.

Example:

```
virtual std::unique_ptr<gpio::Interface> gpio(std::uint8_t pin) noexcept = 0;
```

If a function marked with noexcept nevertheless throws an exception, the program terminates immediately.

This provides clear and predictable error behavior. In many embedded systems this is desirable because:

- Running out of memory is often a critical error.
- The system cannot safely continue anyway.

For this reason, noexcept is used in our driver and factory interfaces. If a memory error still occurs, the program will terminate, providing clear and predictable error behavior.

4.2.4 Example implementation using Abstract Factory

Here we use a simplified form of the Abstract Factory pattern.

Step 1: Factory interface

```
#pragma once

#include <stdint>
#include <memory>

namespace driver
{
    /** GPIO driver interface. */
    namespace gpio { class Interface; }
} // namespace driver
```

```

namespace driver::factory
{
class Interface
{
public:
    virtual ~Interface() noexcept = default;

    [[nodiscard]] virtual std::unique_ptr<gpio::Interface> gpio(std::uint8_t pin) noexcept = 0;
};
} // namespace driver::factory

```

The factory now returns a `std::unique_ptr` instead of a raw pointer.

Note that `gpio()` is marked `[[nodiscard]]`: discarding the returned `std::unique_ptr` would destroy the newly created object immediately, since nothing would be left holding onto it. This is exactly the “factory function returning an owning resource” case mentioned back in §1.2.13.

Step 2: Real factory (for ESP32-S3)

```

#pragma once

#include <memory>

#include "driver/factory/interface.hpp"
#include "driver/gpio/esp32s3.hpp"

namespace driver::factory
{
class Esp32s3 final : public Interface
{
public:
    Esp32s3() noexcept = default;
    ~Esp32s3() noexcept override = default;

    [[nodiscard]] std::unique_ptr<gpio::Interface> gpio(std::uint8_t pin) noexcept override
    {
        return std::make_unique<gpio::Esp32s3>(pin);
    }

    Esp32s3(const Esp32s3&) = delete;

```

```

    Esp32s3(Esp32s3&&)                = delete;
    Esp32s3& operator=(const Esp32s3&) = delete;
    Esp32s3& operator=(Esp32s3&&)     = delete;
};
} // namespace driver::factory

```

Step 3: Stub factory

```

#pragma once

#include <cstdint>
#include <memory>

#include "driver/factory/interface.hpp"
#include "driver/gpio/stub.hpp"

namespace driver::factory
{
    class Stub final : public Interface
    {
    public:
        Stub() noexcept = default;
        ~Stub() noexcept override = default;

        [[nodiscard]] std::unique_ptr<gpio::Interface> gpio(std::uint8_t pin) noexcept
        override
        {
            (void)(pin);
            return std::make_unique<gpio::Stub>();
        }

        Stub(const Stub&)                = delete;
        Stub(Stub&&)                     = delete;
        Stub& operator=(const Stub&)     = delete;
        Stub& operator=(Stub&&)          = delete;
    };
} // namespace driver::factory

```

Step 4: System logic with `unique_ptr`

```

#pragma once

#include <stdint>
#include <memory>

#include "driver/factory/interface.hpp"
#include "driver/gpio/interface.hpp"

namespace app::logic
{
class Logic final
{
public:
    explicit Logic(driver::factory::Interface& factory,
                  std::uint8_t ledPin,
                  std::uint8_t buttonPin) noexcept
        : myLed{factory.gpio(ledPin)}
        , myButton{factory.gpio(buttonPin)}
    {}

    void run() noexcept
    {
        bool buttonPrev{false};

        while (1)
        {
            // Read the current button input.
            const bool buttonCurrent{myButton->read()};

            // Toggle the LED on button press (rising edge).
            if (buttonCurrent && !buttonPrev)
            {
                myLed->toggle();
            }
            // Store the current button input for next comparison.
            buttonPrev = buttonCurrent;
        }
    }

    Logic() = delete; // No default constructor.
    Logic(const Logic&) = delete; // No copy constructor.
    Logic(Logic&&) = delete; // No move constructor.

```

```

    Logic& operator=(const Logic&) = delete; // No copy assignment.
    Logic& operator=(Logic&)      = delete; // No move assignment.

private:
    std::unique_ptr<driver::gpio::Interface> myLed;
    std::unique_ptr<driver::gpio::Interface> myButton;
};
} // namespace app::logic

```

Note:

- No destructor is needed.
- The memory is released automatically.

Step 5: main

Just as in the factory example with raw pointers, we create a factory in the main function and then pass it to the system logic to create objects.

Below is an example of how a factory can be used to run the system logic on ESP32-S3 by passing an instance of `driver::factory::Esp32s3` to the system logic and then calling `run()`:

```

#include <cstdint>

#include "driver/factory/esp32s3.hpp"
#include "app/logic/logic.hpp"

int main()
{
    constexpr std::uint8_t ledPin{2U};
    constexpr std::uint8_t buttonPin{3U};

    // Create system logic and initialize the system.
    driver::factory::Esp32s3 factory{};
    app::logic::Logic logic{factory, ledPin, buttonPin};

    // Run the system continuously.
    logic.run();
    return 0;
}

```

As shown in the example factory with raw pointers, it is sufficient to switch to the stub factory `driver::factory::Stub` in order to run with stub drivers:

```

#include <cstdint>

#include "driver/factory/stub.hpp"
#include "app/logic/logic.hpp"

int main()
{
    constexpr std::uint8_t ledPin{2U};
    constexpr std::uint8_t buttonPin{3U};

    // Create system logic and initialize the system.
    driver::factory::Stub factory{};
    app::logic::Logic logic{factory, ledPin, buttonPin};

    // Run the system continuously.
    logic.run();
    return 0;
}

```

4.2.5 Summary

In this version we use:

- **Interfaces** for all drivers.
- **Abstract Factory** to create the correct type of drivers.
- **std::unique_ptr** for safe ownership.

Advantages:

- No manual delete.
- No memory leaks.
- Clear ownership.
- The same system logic works:
 - On real hardware.
 - In simulation.

4.3 Review

What you should be able to do now

After this chapter, you should:

- Understand what the factory pattern is and why it is used.
- Understand how a factory enables switching between real drivers and stubs.
- Understand how the system logic can create drivers via a factory.
- Be able to read and follow a simple factory example.

Questions to test yourself

1. What is the purpose of the factory pattern?
2. What does it mean that the system logic only depends on driver interfaces and not on concrete driver types?
3. Who is responsible for deleting driver objects when factories return raw pointers?
4. What is the difference between raw pointers and `std::unique_ptr`?
5. Why is `std::unique_ptr` suitable in the factory pattern?

Looking ahead

Chapter 5 is about templates: generic programming for embedded systems without runtime overhead.

4.4 Exercises

How to work these. These exercises reinforce the concepts from §4.1 and §4.2. Sets 1 to 3 build one program with a raw-pointer factory; Set 4 turns the same program into one with a smart-pointer factory. Write the first in `lectures/L04/exercises/raw_factory` and the second in `lectures/L04/exercises/smart_factory`, in the companion repository, each with the layout below, and build and run them with the Makefile from §1.1.5, which already supports the include and source directories.

How to check your work. Run `make test` from the repository root. A set's tests switch on as soon as its files exist, and check the names, signatures and deleted operations exactly; they compare the ESP32-S3 driver's output with the example in Exercise 4.3 character for character, and count what your logic class creates and deletes. A set you have not started is reported as SKIP.

The solutions are published in the companion repository, under `lectures/L04/appendix/solutions`: `raw_factory` for Sets 1 to 3 and `smart_factory` for Set 4, with the answers to the reflection questions in the README beside them. Try each exercise yourself before looking at them.

Create the following directory structure:

```

Makefile
include/
  app/
    logic/
      logic.hpp
  driver/
    factory/
      esp32s3.hpp
      interface.hpp
      stub.hpp
    serial/
      esp32s3.hpp
      interface.hpp
      stub.hpp
source/
  main.cpp

```

Note: When a class overrides a method that is marked `[[nodiscard]]` in the interface, repeat the attribute on the overriding method as well. The attribute is not inherited; see the structure of an interface in §3.2.

EXERCISE SET 1

Serial Interface and Drivers

Exercise 4.1 Serial interface

CODE

In this exercise you will design an interface `driver::serial::Interface` representing a generic serial driver.

Tasks

Design a class named `driver::serial::Interface` that represents a generic serial communication driver.

All methods except the destructor shall be pure virtual (`= 0`).

a) Destructor

Add a destructor that is:

- Virtual.
- Marked `noexcept`.
- Implemented using `= default`.

b) Initialization status

Add a pure virtual method `isInitialized()` that:

- Indicates whether the driver has been initialized (`true/false`).
- Does not modify the object (`const`).
- Cannot throw exceptions.
- Generates a warning if the return value is discarded (`[[nodiscard]]`).

c) Transmitting data

Add a pure virtual method `write()` that:

- Transmits one byte of data.
- Takes a parameter of type `std::uint8_t`.
- Does not return a value.
- Cannot throw exceptions.

d) Receiving data

Add a pure virtual method `read()` that:

- Attempts to read one byte from the serial driver.
- Takes no parameters.
- Returns a `std::optional<std::uint8_t>` from `<optional>`: the received byte, or `std::nullopt` if no byte was received.
- Cannot throw exceptions.
- Generates a warning if the return value is discarded (`[[nodiscard]]`): a byte that is read and then discarded is lost.

Exercise 4.2 Stub serial driver

CODE

In this exercise you will implement a stub driver `driver::serial::Stub`.

A stub driver simulates hardware behavior and is useful for testing without real hardware. The class shall:

- Inherit from `driver::serial::Interface`.
- Be marked `final`.

a) Member variables

Add two private member variables:

- The first member variable shall:
 - Store the most recently transmitted byte, until it is read.
 - Have the type `std::optional<std::uint8_t>`.
 - Be named `myLastByte`.
 - Be empty when no byte is available to read.

- The second member variable shall:
 - Indicate whether the driver is initialized.
 - Have the type `bool`.
 - Be named `myInitialized`.

b) Constructor

Add a default constructor that:

- Sets the driver as initialized.
- Leaves `myLastByte` empty, since no data is available to read.
- Is marked `noexcept`.

c) Destructor

Add a destructor that is:

- Public.
- Marked `noexcept`.
- Implemented using `= default`.

d) Implement interface methods

Implement all methods required by the interface:

- The method `isInitialized()` shall return the value stored in `myInitialized`.
- The method `write()` shall:
 - Do nothing if `myInitialized` is `false`.
 - Otherwise store the transmitted byte in `myLastByte`.
- The method `read()` shall:
 - Return `std::nullopt` if `myInitialized` is `false`, leaving `myLastByte` as it is.
 - Otherwise return the content of `myLastByte`, which is `std::nullopt` if no byte is available, and leave `myLastByte` empty, so that a byte can only be read once.

e) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

Exercise 4.3 Serial driver for ESP32-S3

CODE

In this exercise you will create a hardware-specific serial driver `driver::serial::Esp32s3`.

This class represents a serial driver for the ESP32-S3 microcontroller.

The class shall:

- Inherit from `driver::serial::Interface`.
- Be marked `final`.

a) Member variables

Add two private member variables:

- The first shall store the transmit pin number.
- The second shall store the receive pin number.
- Both shall have the type `std::uint8_t` and be marked `const`.
- They shall be named `myTxPin` and `myRxPin`.

b) Constructor

Add an explicit constructor that:

- Takes a transmit pin number.
- Takes a receive pin number.
- Stores them in the corresponding member variables.
- Is marked `noexcept`.

c) Destructor

Add a destructor that is:

- Public.
- Marked `noexcept`.
- Declared with `override`.

d) Interface methods

Declare and implement the required interface methods.

For this exercise, it is sufficient to use placeholder implementations:

- `isInitialized()` may always return `true`.
- `write()` shall print the transmitted byte in hexadecimal format. For example, transmitting byte `0xFF` on TX pin 17 shall print `Transmitting byte 0xFF via TX pin 17!`.
- `read()` may always return `std::nullopt`.

e) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Default constructor.
- Copy constructor.

- Move constructor.
- Copy assignment operator.
- Move assignment operator.

EXERCISE SET 2

Factory Interface with Raw Pointers

Exercise 4.4 Factory interface

CODE

In this exercise you will design a factory interface `driver::factory::Interface`.

The purpose of this factory is to create serial drivers without exposing the concrete driver type to the application logic.

Tasks

Design a class named `driver::factory::Interface`.

a) Destructor

Add a destructor that is:

- Virtual.
- Marked `noexcept`.
- Implemented using `= default`.

b) Factory method

Add a pure virtual method `serial()` that:

- Creates a serial driver.
- Takes a transmit pin number of type `std::uint8_t`.
- Takes a receive pin number of type `std::uint8_t`.
- Returns a pointer to `driver::serial::Interface`.
- Is marked `noexcept`.
- Generates a warning if the return value is discarded (`[[nodiscard]]`).

Exercise 4.5 ESP32-S3 factory

CODE

In this exercise you will implement `driver::factory::Esp32s3`.

This factory shall create real drivers for ESP32-S3.

The class shall:

- Inherit from `driver::factory::Interface`.
- Be marked `final`.

a) Constructor and destructor

Add:

- A default constructor marked `noexcept`.
- A destructor marked `noexcept` and `override`.

b) Factory method

Implement the method `serial()` so that it:

- Takes a transmit pin number and a receive pin number as input.
- Dynamically allocates a `driver::serial::Esp32s3`.
- Returns the object as a pointer to `driver::serial::Interface`.

Use the operator `new`.

c) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

Exercise 4.6 Stub factory`CODE`

In this exercise you will implement `driver::factory::Stub`.

This factory shall create stub serial drivers.

The class shall:

- Inherit from `driver::factory::Interface`.
- Be marked `final`.

a) Constructor and destructor

Add:

- A default constructor marked `noexcept`.
- A destructor marked `noexcept` and `override`.

b) Factory method

Implement the method `serial()` so that it:

- Takes a transmit pin number and a receive pin number as input.
- Ignores both pin numbers.
- Dynamically allocates a `driver::serial::Stub`.
- Returns the object as a pointer to `driver::serial::Interface`.

Use the operator `new`.

c) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

EXERCISE SET 3**Application Logic with Raw Pointers****Exercise 4.7 Logic class using a factory****CODE**

In this exercise you will implement a logic class `app::logic::Logic` that uses a factory to create its serial driver.

The application logic shall:

- Own one serial driver.
- Periodically transmit a message.
- Attempt to read one received byte.

a) Member variable

Add a private member variable `mySerial` of type `driver::serial::Interface*`.

b) Constructor

Add a constructor that:

- Takes a reference to `driver::factory::Interface`.
- Takes a transmit pin number of type `std::uint8_t`.
- Takes a receive pin number of type `std::uint8_t`.
- Is marked `noexcept`.

Inside the constructor, use the factory to create the serial driver.

c) Destructor

Add a destructor that:

- Is marked `noexcept`.
- Deletes `mySerial`.

d) run() method

Add a method `run()` that:

- Is marked `noexcept`.
- Runs continuously.

- Sends an incrementing byte value (0–255). After reaching 255, the value wraps around to 0.
- Attempts to read one byte and prints it if one was received.
- Repeats forever.
- Delays execution by 100 ms at the end of each loop iteration:

```
#include <chrono>
#include <thread>

constexpr std::uint8_t sleep_ms{100U};

std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));
```

Use a local variable of type `std::uint8_t` to store the transmitted byte:

```
std::uint8_t txByte{};
```

The received byte needs no variable declared in advance: it is returned by `read()` as a `std::optional<std::uint8_t>`.

e) Disable copy and move semantics

Delete the following functions (in the public section of the class):

- Default constructor.
- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

Exercise 4.8 Using the raw-pointer factory in main

CODE

In this exercise you will test your application using both factories.

a) ESP32-S3 factory

In `main.cpp`:

- Create an instance of `driver::factory::Esp32s3`.
- Create an instance of `app::logic::Logic` using that factory.
- Pass suitable pin numbers, for example 17U and 18U.
- Call `run()`.

b) Stub factory

Modify `main.cpp` so that `driver::factory::Stub` is used instead of `driver::factory::Esp32s3`.

c) Reflection

Answer the following questions:

- What code in `Logic` had to change when switching from the ESP32-S3 factory to the stub factory?
- What code in `main()` had to change?

EXERCISE SET 4**Factory Interface with Smart Pointers**

Exercise 4.9 Updating the factory interface**CODE**

In this exercise you will modernize the factory interface by replacing raw pointers with `std::unique_ptr`.

Tasks

Update `driver::factory::Interface` so that the method `serial()`:

- Returns `std::unique_ptr<driver::serial::Interface>`.
- Still takes a transmit pin number of type `std::uint8_t`.
- Still takes a receive pin number of type `std::uint8_t`.
- Is marked `noexcept`.
- Is still marked `[[nodiscard]]`: discarding the returned `std::unique_ptr` would destroy the newly created object immediately.

Also include the header `<memory>` where needed.

Exercise 4.10 Updating the concrete factories**CODE**

In this exercise you will update both concrete factories to use smart pointers.

a) ESP32-S3 factory

Modify `driver::factory::Esp32s3` so that:

- The method `serial()` returns `std::unique_ptr<driver::serial::Interface>`.
- The object is created using `std::make_unique<driver::serial::Esp32s3>(txPin, rxPin)`.

b) Stub factory

Modify `driver::factory::Stub` so that:

- The method `serial()` returns `std::unique_ptr<driver::serial::Interface>`.
- The object is created using `std::make_unique<driver::serial::Stub>()`.
- The pin numbers are ignored.

Exercise 4.11 Updating the logic class

CODE

In this exercise you will update `app::logic::Logic` to use smart pointers.

a) Member variable

Replace the raw pointer member `mySerial` with a `std::unique_ptr<driver::serial::Interface>`.

b) Constructor

Keep the constructor parameters the same as before, but initialize the member variable using the factory's smart-pointer return value.

c) Destructor

Remove the destructor entirely.

d) run() method

Keep the behavior the same as before:

- Send an incrementing byte value (0–255), wrapping around to 0 after 255.
- Attempt to read one byte and print it if received.
- Delay execution by 100 ms at the end of each loop iteration.
- Repeat forever.

Remember that member access through a smart pointer still uses the arrow operator `->`.

Templates

Generic code the compiler writes for you, and what it costs in flash.

IN THIS CHAPTER

- Motivation for templates in embedded systems.
- Function templates.
- Template instantiation (compile-time code generation).
- Class templates.
- Code size considerations in embedded systems.
- Introduction to type traits and parameter packs.

This chapter is about what templates are and why they are used. You will see how function templates enable generic code, and how class templates can be used for reusable components such as buffers and frames. Templates are instantiated at compile time and generate concrete types and functions, and that is exactly what makes them both free at run time and a cost in code size, so the chapter is as much about when templates are appropriate in an embedded system as about how to write them. By the end you should be able to read and follow simple template-based code, and use type traits for basic compile-time checks.

The chapter assumes basic knowledge of functions and classes in C++ (Chapter 2), an understanding of interfaces and system architecture (Chapters 3 and 4), and a basic understanding of compilation. Function templates and type traits were first met in §1.2.12 and §1.2.15; this chapter takes them further.

Work the chapter in order. §5.1 introduces function templates, §5.2 type traits and constraints, and §5.3 class templates. The exercises in §5.5 are where you write the code yourself; their solutions are in the companion repository.

5.1 Function Templates

Templates allow functions to operate on multiple data types without duplicating code.

In embedded systems, templates are often used to implement generic utilities without run-time overhead, while still allowing the compiler to optimize the code for each specific type.

5.1.1 Basic example

Consider a simple function template that adds two values:

```
/**
 * @brief Add two numbers.
 *
 * @tparam T The numeric type.
 *
 * @param[in] x The first value.
 * @param[in] y The second value.
 *
 * @return The sum of the two values.
 */
template<typename T>
constexpr T add(const T x, const T y) noexcept
{
    return x + y;
}
```

The function can be used with different types:

```
const auto sum1 = add(1, 2);
const auto sum2 = add(1.0, 2.0);
```

The compiler generates a separate version of the function for each type.

5.1.2 Embedded example: bit manipulation

In embedded systems, templates are often used for low-level utilities.

The example below sets a bit in a register of arbitrary integral type:

```
#include <cstdint>

/**
 * @brief Set bit in register.
 *
 * @tparam T The register type.
 *
 * @param[in, out] reg Register to write to.
```

```

    * @param[in] bit The bit to set.
    */
    template<typename T>
    constexpr void set(T& reg, const std::uint8_t bit) noexcept
    {
        reg |= (static_cast<T>(1U) << bit);
    }

```

Example usage:

```

std::uint8_t reg{};
set(reg, 1U);
set(reg, 2U);

```

This allows the same function to be reused for different register sizes.

5.1.3 Template instantiation

Templates are instantiated at compile time.

For example:

```

std::uint8_t reg1{};
std::uint32_t reg2{};

set(reg1, 1U);
set(reg2, 12U);

```

The compiler generates two separate functions:

```

set(std::uint8_t&, std::uint8_t)
set(std::uint32_t&, std::uint8_t)

```

Each version is compiled independently.

5.1.4 Code size considerations

Templates are often described as zero-cost abstractions:

- No run-time overhead.
- Code is specialized for each type.

However, in embedded systems this also means:

- Multiple instantiations → increased binary size.
- More types → more generated code.

Therefore:

- Use templates when they provide clear value.
- Avoid unnecessary instantiations in resource-constrained systems.

5.1.5 Function overloading vs templates

C++ supports function overloading:

```
constexpr void set(std::uint8_t& reg, std::uint8_t bit) noexcept;
constexpr void set(std::uint32_t& reg, std::uint8_t bit) noexcept;
```

Templates provide a more scalable solution:

```
template<typename T>
constexpr void set(T& reg, std::uint8_t bit) noexcept;
```

5.1.6 Summary

- Function templates enable generic and reusable code.
- Templates are instantiated at compile time.
- Each type results in a separate generated function.
- Templates provide no run-time overhead, but may increase code size.
- Templates are especially useful for:
 - Bit manipulation.
 - Utility functions.
 - Hardware abstraction helpers.

5.2 Type Traits and Constraints

Type traits allow programs to inspect and reason about types at compile time.

They are provided in the C++ standard library via the header:

```
#include <type_traits>
```

Type traits are commonly used together with templates to create safer and more robust code, for instance to prevent misuse of low-level hardware APIs (e.g. restricting register operations to integral types).

5.2.1 Common type traits

Some commonly used type traits include:

- `std::is_integral` — checks if a type is an integer type.
- `std::is_arithmetic` — checks if a type is numeric (integer or floating point).
- `std::is_unsigned` — checks if a type is an unsigned integer.
- `std::is_floating_point` — checks if a type is a floating-point type.

Each type trait provides a boolean value:

```
std::is_integral<int>::value    // true
std::is_integral<double>::value // false
```

Since C++17, a shorthand form is also available:

```
std::is_integral_v<int> // true
```

5.2.2 Compile-time constraints with `static_assert()`

Type traits are often used together with `static_assert()` to enforce constraints at compile time.

Example:

```
#include <type_traits>

/**
 * @brief Add two numbers.
 *
 * @tparam T The numeric type. Must be arithmetic.
 */
template<typename T>
constexpr T add(const T x, const T y) noexcept
{
    static_assert(std::is_arithmetic<T>::value, "Cannot add values of non-↵
    ↵ arithmetic types!");
    return x + y;
}
```

If the function is used incorrectly:

```
add("Hello", "World");
```

The `static_assert()` fails and the compiler stops with the error message given above. Without it, the compiler would still reject the call, since two pointers cannot be added, but with a less readable error from inside the function body.

5.2.3 Why compile-time checks matter in embedded systems

In embedded systems:

- Errors should be detected as early as possible.
- Debugging run-time issues can be difficult.
- Safety and robustness are critical.

Compile-time checks provide:

- Early error detection.
- Clear diagnostics.
- Zero run-time overhead.

5.2.4 Creating custom type traits

If no suitable type trait exists, we can create our own.

A type trait is typically implemented as a template struct that exposes a boolean constant named value. We set the constant value to false by default for a given type T:

```
template<typename T>
struct isUnsigned
{
    static constexpr bool value{false};
};
```

We then set the constant value to true for unsigned types, such as `std::uint8_t` and `std::uint32_t`:

```
#include <cstdint>

template<>
struct isUnsigned<std::uint8_t>
{
    static constexpr bool value{true};
};

template<>
struct isUnsigned<std::uint16_t>
{
    static constexpr bool value{true};
};

template<>
struct isUnsigned<std::uint32_t>
```

```
{
    static constexpr bool value{true};
};
```

Usage:

```
// Generate a compiler error if T isn't of unsigned type:
static_assert(isUnsigned<T>::value, "T must be unsigned!");
```

5.2.5 Variable templates (C++17)

For convenience, we can define a variable template:

```
template<typename T>
inline constexpr bool isUnsigned_v{isUnsigned<T>::value};
```

This allows:

```
// Generate a compiler error if T isn't of unsigned type:
static_assert(isUnsigned_v<T>, "T must be unsigned!");
```

Note: In practice, the standard library already provides many useful traits (e.g. `std::is_unsigned`). Custom traits are mainly used when no suitable standard trait exists.

5.2.6 Summary

- Type traits allow inspection of types at compile time.
- `static_assert()` enables compile-time validation.
- Compile-time checks improve safety and robustness.
- Custom type traits can be created when needed.
- These tools are essential for building safe template-based APIs.

5.3 Class Templates

Class templates allow us to define generic data structures, where the type is determined at compile time.

This enables reusable and efficient code without run-time overhead.

5.3.1 Example: a generic container

In this section, we implement a simple container similar to `std::vector`:

```
container::Vector<T>
```

The purpose is to:

- Understand how class templates work.
- Understand how memory management affects embedded systems.
- Use compile-time constraints for safety.
- See compile-time polymorphism in practice.

5.3.2 What is a class template?

A class template is a blueprint for a class where the type is not fixed until compilation.

Example:

```
namespace container
{
    template<typename T>
    class Vector
    {
    public:
        Vector() noexcept
            : myData{nullptr}
            , mySize{}
        {}
    private:
        T* myData;
        std::size_t mySize;
    };
} // namespace container
```

When used:

```
container::Vector<int> a{};
container::Vector<double> b{};
```

The compiler generates two separate types:

```
namespace container
{
    // Vector specialization for int.
    class Vector<int>
    {
    public:
        Vector() noexcept
```

```

        : myData{nullptr}
        , mySize{}
    {}
private:
    int* myData;
    std::size_t mySize;
};

// Vector specialization for double.
class Vector<double>
{
public:
    Vector() noexcept
        : myData{nullptr}
        , mySize{}
    {}
private:
    double* myData;
    std::size_t mySize;
};
} // namespace container

```

This is compile-time polymorphism.

5.3.3 Example implementation

Below is a simplified implementation of a vector-like container.

Key features:

- Dynamic resizing.
- Push-back functionality.
- Range-based iteration support.

```

#pragma once

#include <cstddef>
#include <cstdlib>
#include <type_traits>

namespace container
{
    template<typename T>
    class Vector
    {

```

```

// Generate a compiler error if the given type isn't trivially copyable or ↵
↵ destructible,
// since std::realloc() and std::free() are used in this implementation.
static_assert(std::is_trivially_copyable<T>::value && std::is_
↵ is_trivially_destructible<T>::value,
    "Vector type T must be of trivial type!");
public:
    Vector() noexcept
        : myData{nullptr}
        , mySize{}
    {
        // Initialized empty vector.
    }

    template<typename... Args>
    Vector(const Args& ...args) noexcept
        : Vector()
    {
        // Try to resize the vector to fit the given number of arguments.
        // Copy the arguments to the vector on success.
        if (resize(sizeof...(args)))
        {
            std::size_t i{};
            ((myData[i++] = args), ...);
        }
    }

    ~Vector() noexcept
    {
        // Release allocated resources before deletion.
        clear();
    }

    T& operator[](const std::size_t index) noexcept
    {
        // Return a reference to the element at given index.
        return myData[index];
    }

    const T& operator[](const std::size_t index) const noexcept
    {
        // Return a read-only reference to the element at given index.
        return myData[index];
    }

```

```

T* begin() noexcept
{
    // Return a pointer to the beginning of the data field.
    return myData;
}
const T* begin() const noexcept
{
    // Return a read-only pointer to the beginning of the data field.
    return myData;
}

T* end() noexcept { return nullptr != myData ? myData + mySize : nullptr; }
const T* end() const noexcept { return nullptr != myData ? myData + mySize :
↳ nullptr; }

[[nodiscard]] const T* data() const noexcept
{
    // Return a read-only pointer to the data field.
    return myData;
}

[[nodiscard]] std::size_t size() const noexcept
{
    // Return the size of the vector.
    return mySize;
}

[[nodiscard]] bool empty() const noexcept
{
    // Return true if the vector is empty.
    return 0U == mySize;
}

void clear() noexcept
{
    // Free allocated resources, then reset the vector parameters.
    std::free(myData);
    myData = nullptr;
    mySize = 0U;
}

[[nodiscard]] bool resize(const std::size_t size) noexcept
{
    // Clear the vector if the requested size is 0.

```

```

        if (0U == size)
        {
            clear();
            return true;
        }

        // Try to reallocate the data field to the requested size, return false on
        ↪ failure.
        T* data{static_cast<T*>(std::realloc(myData, sizeof(T) * size))};
        if (nullptr == data) { return false; }
        myData = data;

        // Initialize new elements (if any).
        // Update the vector size, then return true to indicate success.
        for (std::size_t i{mySize}; i < size; ++i) { myData[i] = {}; }
        mySize = size;
        return true;
    }

[[nodiscard]] bool pushBack(const T& element) noexcept
{
    const std::size_t newSize{mySize + 1U};
    // Try to reallocate the data field to fit one more element, return false ↪
    ↪ on failure.
    T* data{static_cast<T*>(std::realloc(myData, sizeof(T) * newSize))};
    if (nullptr == data) { return false; }

    // Push the new element to the back of the vector, then return true to ↪
    ↪ indicate success.
    myData = data;
    myData[mySize++] = element;
    return true;
}

Vector(const Vector&)           = delete; // No copy constructor.
Vector(Vector&&)                = delete; // No move constructor.
Vector& operator=(const Vector&) = delete; // No copy assignment.
Vector& operator=(Vector&&)      = delete; // No move assignment.

private:
    /** Pointer to dynamically allocated data field. */
    T* myData;

    /** Size of the data field in number of elements. */

```

```

        std::size_t mySize;
    };
} // namespace container

```

The above implementation means that the class template `container::Vector` behaves in many ways like `std::vector`, including initialization with an argument list:

```

container::Vector<int> vec1{1, 3, 5, 7, 9};
container::Vector<double> vec2{0.5, 1.5, 2.5, 3.5, 4.5};

```

5.3.4 Placement of the implementation in header files

Since `container::Vector` is a template, this affects how the implementation must be placed in the source code.

The compiler must see the full definition of a template when it instantiates it, for example when we write:

```

container::Vector<int>

```

This means that both the class declaration and the method implementations must be visible where the template is used.

There are two common solutions.

a) Everything in the same header file

The most common approach is to place both declaration and implementation in the same header file, for example:

```

include/container/vector.hpp

```

Contains:

```

template<typename T>
class Vector { ... };

template<typename T>
bool Vector<T>::pushBack(const T& element)
{
    ...
}

```

This works because the compiler sees all code when the template is instantiated.

b) Header + implementation header

In larger projects, it is common to separate declaration and implementation into two files, but include the implementation at the end of the header file.

Structure:

```
include/container/vector.hpp
include/container/impl/vector_impl.hpp
```

And at the end of `vector.hpp`:

```
#include "container/impl/vector_impl.hpp"
```

This gives the same effect as option a), but provides better structure and readability.

Why doesn't a .cpp file work?

If the implementation is placed in a regular `.cpp` file, the compiler cannot see the code when the template is instantiated in other files. This often leads to linker errors such as:

```
undefined reference to ...
```

Templates must therefore be implemented in header files (directly or indirectly via `include`).

5.3.5 Embedded perspective

In embedded systems, we need to consider the following:

Heap usage

- Dynamic allocation may be forbidden.
- Fragmentation can occur.

Binary size

- Each instantiation of `Vector<T>` generates new code.
- `Vector<int> ≠ Vector<double>`.

Exception handling

- In many embedded systems, exceptions are disabled.
- Therefore, `noexcept` is used.

5.3.6 Summary

In this section, we have seen:

- How to create a class template.
- How to handle manual memory allocation.

- How copy/move semantics can be handled or restricted.
- Why template implementations must reside in header files.
- How templates affect embedded systems.

`container::Vector<T>` is a clear example of:

- Compile-time polymorphism.
- Generic programming.
- Performance-optimized C++.
- Full control over memory and lifecycle.

5.4 Review

What you should be able to do now

After this chapter, you should:

- Understand what templates are and why they are used.
- Understand how function templates enable generic code.
- Understand how class templates can be used for reusable components (e.g. buffers, frames).
- Understand that templates are instantiated at compile time and generate concrete types and functions.
- Understand how templates affect code size in embedded systems.
- Understand when templates are appropriate in embedded systems and when they are not.
- Be able to read and follow simple template-based code.
- Understand the purpose of type traits for basic compile-time checks.

Questions to test yourself

1. What problem do templates solve in C++?
2. What is the difference between a function template and a class template?
3. What does it mean that templates are instantiated at compile time, and what does the compiler generate?
4. Why can templates increase code size in embedded systems?
5. Give an example where a template is more suitable than an interface.
6. Why must template implementations typically be placed in header files?

Looking ahead

Chapter 6 is about concurrency concepts in modern C++:

- Threads.
- Mutexes.
- Atomic variables.
- Lock guards.
- Synchronization primitives.
- Thread-safe design.

5.5 Exercises

How to work these. These exercises practice the concepts from §5.1 to §5.3. Sets 1 and 3 are one program each: write each in its own directory under `lectures/L05/exercises` in the companion repository, with the file names its README gives, and build and run it with the Makefile from §1.1.3 (or the one from §1.1.5, for a program with an include directory).

How to check your work. Run `make test` from the repository root. A set's tests switch on as soon as its files exist, and compare your program's output with the expected output each exercise gives, character for character; a set you have not started is reported as SKIP.

The solutions are published in the companion repository, under `lectures/L05/appendix/solutions`: `function_template` for Set 1 and `class_template` for Set 3, with the answers to the reflection questions in the README beside them. Set 2 has no program; its answers are the subject of §5.2.2 and §5.2.3. Try each exercise yourself before looking at them.

EXERCISE SET 1

Function Templates

Exercise 5.1 Clear bit

CODE

Create a function template to clear a bit in a register:

```
/**
 * @brief Clear bit in a register.
 *
 * @tparam T Register type. Must be integral.
 *
 * @param[in, out] reg Destination register.
 * @param[in] bit Bit to clear.
 */
template<typename T>
```

```
constexpr void clear(T& reg, const std::uint8_t bit) noexcept;
```

Tasks

- a) Use `static_assert()` with `std::is_integral<T>::value` and the following error message:

```
Cannot perform bit operation with non-integral type!
```

- b) Clear the selected bit.
c) Try the `clear()` function with the following code:

```
#include <bitset>
#include <cstdint>
#include <iostream>

int main()
{
    std::uint8_t reg{0xFFU};

    // Clear bit 2 => reg becomes 0b11111011.
    clear(reg, 2U);
    std::cout << "Register content after clearing bit 2: 0b" << std::bitset<8U>(reg) << "\n";
    return 0;
}
```

Expected output:

```
Register content after clearing bit 2: 0b11111011
```

Exercise 5.2 Toggle bits

[CODE](#)

Create the following function template to toggle one or several bits in a register:

```
/**
 * @brief Toggle bits in a register.
 *
 * @tparam T Register type. Must be integral.
 * @tparam Bits Parameter pack of bits.
 *
 * @param[in, out] reg Destination register.
 * @param[in] bits Bits to toggle.
```

```

*/
template<typename T, typename... Bits>
constexpr void toggle(T& reg, const Bits... bits) noexcept;

```

Tasks

- a) Use a parameter pack.
- b) Ensure that T is an integral type using `static_assert()`.
- c) Iterate over the bits (for example using `{bits...}`).
- d) Toggle the specified bits.
- e) Try the `toggle()` function with the following code snippet:

```

#include <bitset>
#include <cstdint>
#include <iostream>

int main()
{
    std::uint8_t reg{0xFFU};

    // Clear bit 2 => reg becomes 0b11111011.
    clear(reg, 2U);
    std::cout << "Register content after clearing bit 2: 0b" << std::bitset<8U>(reg) << "\n";

    // Toggle bits 0-3 => reg becomes 0b11110100.
    toggle(reg, 0U, 1U, 2U, 3U);
    std::cout << "Register content after toggling bits 0-3: 0b" << std::bitset<8U>(reg) << "\n";
    return 0;
}

```

Expected output:

```

Register content after clearing bit 2: 0b11111011
Register content after toggling bits 0-3: 0b11110100

```

EXERCISE SET 2

Function Template Concepts

Exercise 5.3 Template constraints

REFLECTION

- Explain why `static_assert()` is useful.
- What happens if the constraint fails?
- Why use compile-time checks instead of runtime?

EXERCISE SET 3

Class Template Specialization

Exercise 5.4 Timer driver template

CODE

In this exercise, you will create a timer driver that supports multiple implementations using template specialization in a file `driver/timer/timer.hpp`.

The goal is to select the timer implementation at compile time based on a template parameter.

The timer type shall be selected using the following enumeration class:

```
namespace driver::timer
{
    /**
     * @brief Enumeration of supported timers.
     */
    enum class Type : std::uint8_t
    {
        Stm32, ///< STM32 timer.
        Stub,  ///< Stub timer.
    };
} // namespace driver::timer
```

The timer shall be declared according to the following structure:

```
namespace driver::timer
{
    /**
     * @brief Timer implementation.
     *
     * @tparam T Timer type.
     */
```

```

template<Type T>
class Timer;

} // namespace driver::timer

```

Note: `final` belongs on the definition of the class, not on this declaration. Written as `class Timer final;`, the compiler would read `final` as the name of a variable of type `Timer`.

The primary template shall represent a stub timer implementation. A specialization shall later be created for `Type::Stm32`.

Part I: Stub timer

a) Implement the primary template as a stub timer:

```

/**
 * @brief Timer implementation.
 *
 * @tparam T Timer type (default = stub timer).
 */
template<Type T = Type::Stub>
class Timer final
{
};

```

b) Add four private member variables:

- The first member variable shall:
 - Be named `myTimeout_ms`.
 - Represent the timeout duration in milliseconds.
 - Have the type `std::uint16_t`.
 - Be readable only after initialization.
- The second member variable shall:
 - Be named `myCounter_ms`.
 - Represent the internal timer counter.
 - Have the type `std::uint16_t`.
- The third member variable shall:
 - Be named `myRunning`.
 - Represent whether the timer is active or not (`true/false`).
 - Be `true` when the timer is running.
- The fourth member variable shall:
 - Be named `myInitialized`.

- Represent whether the timer is initialized or not (`true/false`).
- Be `true` when the timer is initialized.

c) Add a constructor that:

- Takes a timeout value in milliseconds (`std::uint16_t`).
- Uses an initialization list to initialize the member variables.
- Initializes the counter to 0.
- Initializes the timer as stopped.
- Marks the timer as initialized if the timeout value is valid (> 0).
- Is marked `explicit` and `noexcept`.
- If the timeout is valid, prints:

```
Created stub timer with timeout 1000 ms!
```

where 1000 shall be replaced by the configured timeout value.

- If the timeout is 0, prints:

```
Failed to initialize stub timer: invalid timeout 0 ms!
```

d) Add a destructor that:

- Is marked `noexcept`.
- Stops the timer if it is currently running.
- Prints `Destroying stub timer!`.

e) Define the following methods:

- The method `timeout_ms()` shall:
 - Return the configured timeout in milliseconds.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
- The method `isRunning()` shall:
 - Return `true` if the timer is running; otherwise `false`.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
- The method `isInitialized()` shall:
 - Return `true` if the timer is initialized; otherwise `false`.
 - Take no parameters.
 - Be marked `const`, `noexcept`, and `[[nodiscard]]`.
- The method `start()` shall:
 - Start the timer if the timer is initialized.

- Print `Starting stub timer!`.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `stop()` shall:
 - Stop the timer if the timer is initialized.
 - Print `Stopping stub timer!`.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `toggle()` shall:
 - Toggle the timer if the timer is initialized.
 - Print `Toggling stub timer!`.
 - Take no parameters.
 - Return no value.
 - Be marked `noexcept`.
 - The method `tick()` shall:
 - Simulate that 1 millisecond has passed.
 - Increment the counter if the timer is running; otherwise do nothing.
 - Return no value.
 - Be marked `noexcept`.
 - The method `hasTimedOut()` shall:
 - Return `true` when the counter has reached the timeout value; otherwise `false`.
 - Reset the counter to 0 when a timeout occurs.
 - Be marked `noexcept` and `[[nodiscard]]`.
- f) Delete the following methods:
- The default constructor.
 - The copy constructor.
 - The move constructor.
 - The copy assignment operator.
 - The move assignment operator.

Part II: STM32 timer

- a) Create a specialization for microcontroller STM32:

```

/**
 * @brief STM32 timer implementation.
 */
template<>
class Timer<Type::Stm32> final
{
};

```

b) Implement the STM32 timer with the same public interface as the primary template:

- Implement the same methods, but adjust the prints, for instance:

```

Starting STM32 timer!
Stopping STM32 timer!
Destroying STM32 timer!

```

- Delete the default constructor as was done for the stub implementation.
- Prohibit copy and move semantics as was done for the stub implementation.

Part III: Testing the timers

Test the timers using the following test program in `main.cpp`:

```

#include <chrono>
#include <cstdint>
#include <thread>

#include "driver/timer/timer.hpp"

using namespace driver;

namespace
{
// -----
template<timer::Type T>
constexpr const char* timerType() noexcept
{
    if constexpr (T == timer::Type::Stub) { return "Stub"; }
    else if (T == timer::Type::Stm32) { return "STM32"; }
    return "Unknown";
}

// -----
template<timer::Type T>
void tickTimer(timer::Timer<T>& timer) noexcept

```

```

{
    constexpr const char* type{timerType<T>{}};
    timer.tick();

    if (timer.hasTimedOut())
    {
        std::printf("%s timer has timed out after %u ms!\n", type, timer.
        timeout_ms());
    }
}

// -----
void delay_ms(const std::uint16_t ms) noexcept
{
    std::this_thread::sleep_for(std::chrono::milliseconds(ms));
}
} // namespace

// -----
int main()
{
    timer::Timer timer1{500U};
    timer::Timer<timer::Type::Stm32> timer2{1500U};
    timer1.start();
    timer2.start();

    constexpr std::uint16_t iterationCount{2000U};

    for (std::uint16_t i{}; i < iterationCount; ++i)
    {
        tickTimer(timer1);
        tickTimer(timer2);
        delay_ms(1U);
    }
    return 0;
}

```

Expected output:

```

Created stub timer with timeout 500 ms!
Created STM32 timer with timeout 1500 ms!
Starting stub timer!
Starting STM32 timer!

```

```

Stub timer has timed out after 500 ms!
Stub timer has timed out after 500 ms!
Stub timer has timed out after 500 ms!
STM32 timer has timed out after 1500 ms!
Stub timer has timed out after 500 ms!
Stopping STM32 timer!
Destroying STM32 timer!
Stopping stub timer!
Destroying stub timer!

```

Exercise 5.5 Reflection

REFLECTION

- a) Why is the timer type selected at compile time?
- b) What is a template specialization?
- c) What advantages does this design provide compared to using inheritance and virtual functions?
- d) When would inheritance be a better choice?
- e) What happens if the specialization for `Type::Stm32` is removed and a `Timer<Type::Stm32>` instance is created?
- f) Why might a template specialization be preferred over a virtual interface in a small microcontroller system?

Multithreading and Synchronization

Threads, and the tools that stop them corrupting what they share.

IN THIS CHAPTER

- Introduction to multithreading in C++ (`std::thread`).
- Concurrency vs. true parallelism — single-core vs. multi-core.
- Scheduling: time-slicing vs. cooperative.
- Data races and why they are dangerous.
- Critical sections using `std::mutex` and `std::lock_guard`.
- The difference between `std::mutex` and `std::atomic`.
- Signalling between threads using `std::condition_variable`.
- Launching background tasks with `std::future` and `std::async`.
- Priority inversion — a common pitfall in embedded systems.

This chapter is about running code in more than one thread, and about the tools C++ provides to keep those threads from corrupting the data they share. It explains what a thread is and how one is created with `std::thread`, what separates concurrency from parallelism, and how a scheduler decides which thread runs. It then shows why a data race leads to *undefined behavior*, how `std::mutex` and `std::lock_guard` protect a critical section, when `std::atomic` is sufficient and when a mutex is required, how `std::condition_variable` lets one thread wait for another without busy-waiting, and how `std::async` and `std::future` launch a task and collect its result. It ends with priority inversion, and why `std::mutex` does not solve it on an RTOS.

The chapter assumes a good knowledge of C++ from the chapters before it (classes, references and RAII) and a basic understanding of program flow and object lifetime. Read §6.1 first; the exercises in §6.3 then build a series of small threaded programs, ending with a thread-safe driver. Their solutions are in the companion repository.

6.1 Multithreading and Synchronization in C++

This section introduces the fundamentals of concurrency in C++, and extends into more advanced topics useful for embedded systems with multiple cores or an RTOS:

- `std::thread`: Threads for parallel execution of code in C++.
- `std::mutex` combined with `std::lock_guard`: Protection of shared data (critical sections).
- `std::atomic`: How to share individual variables between threads without a mutex.
- The practical differences between atomic variables and mutexes.
- Scheduling: how the OS decides which thread runs and when.
- `std::future` and `std::async`: Higher-level asynchronous task launching.
- `std::condition_variable`: Efficient signalling between threads.
- Priority inversion: A subtle but dangerous bug in mixed-priority systems.

6.1.1 Concurrency vs. true parallelism

Modern systems can execute multiple tasks simultaneously or appear to do so, depending on the hardware and OS. These two concepts are known as concurrency and parallelism:

- **Concurrency** means multiple tasks make progress over time; they may not run at exactly the same instant.
- **Parallelism** means they genuinely run simultaneously.

The distinction becomes clear when looking at single-core vs. multi-core hardware:

- On a **single-core MCU** the scheduler interleaves threads one at a time (concurrency).
- On a **multi-core SoC** (e.g. ARM Cortex-A53 quad-core) threads truly run at the same time (parallelism).

`std::thread` works the same way in both cases; the hardware and OS decide.

6.1.2 What is a thread?

A thread is an independent execution flow managed by the OS. Each thread has its own:

- **Program counter**: Tracks which instruction executes next.
- **Stack**: Holds local variables and return addresses for that thread's call chain.
- **Register state**: The CPU registers saved and restored on every context switch.

What threads share is everything else in the process: global variables, the heap, file handles, and any other objects explicitly passed between them. This is what makes shared-data bugs possible.

In C++, a thread is created using:

```
std::thread t{function, argument};
```

Note: The `join()` method must be called to wait for a thread to finish before the program exits:

```
std::thread t1{function1, argument1};
std::thread t2{function2, argument2};

t1.join();
t2.join();
```

Alternatively, `detach()` can be used to allow a thread to run independently of the other threads, but this is not recommended in the context of this book.

Note: Function `std::ref()` must be used when passing references to a thread, since `std::thread` copies its arguments by default:

```
std::uint32_t counter{};
std::thread t1{function1, std::ref(counter)};
t1.join();
```

To pass a constant reference, use `std::cref()` instead:

```
std::uint32_t counter{};
std::thread t2{function2, std::cref(counter)};
t2.join();
```

6.1.3 Scheduling: time-slicing vs. cooperative

The OS switches between threads in one of two ways:

- **Preemptive (time-slice):** The OS forcibly switches the running thread after a fixed time quantum (e.g. 1 ms). The thread has no say; a timer interrupt triggers the context switch.
- **Cooperative:** A thread runs until it voluntarily yields. One badly-behaved thread can starve the rest.

You can hint at cooperative behaviour from C++ by invoking `std::this_thread::yield()` as shown below, but you cannot control the scheduler directly. Notably, `std::thread` provides no way to set thread priority. For that, you must use the platform's native thread handle via `std::thread::native_handle()` and call the platform API (e.g. `pthread_setschedparam()` on POSIX, `SetThreadPriority()` on Windows).

```
#include <chrono>
#include <cstdint>
#include <thread>

void task() noexcept
```

```

{
    constexpr std::uint8_t sleep_ms{50};

    // Run thread continuously.
    while (true)
    {
        // Do some work in this thread.
        doSomeWork();

        // Give hint to give up the CPU now (cooperative behaviour).
        std::this_thread::yield();

        // Sleep for at least 5 ms.
        std::this_thread::sleep_for(std::chrono::milliseconds(sleep_ms));
    }
}

```

Below is an example that demonstrates how you can set the priority and policy of a given thread `t1` in a Linux system:

- `native_handle()` returns a `pthread_t` on Linux, enabling access to the full POSIX thread API.
- `SCHED_FIFO` is a real-time scheduling policy; it requires elevated privileges on Linux (run with `sudo`).
- `t1` is assigned `SCHED_FIFO` with priority 99, which is the highest `SCHED_FIFO` priority on Linux (the valid range is 1–99; `sched_get_priority_max(SCHED_FIFO)` returns the maximum on any POSIX system).

```

int main()
{
    // Create thread running function task().
    std::thread t1{task};

    // Get the native handle of the thread (pthread in Linux).
    auto handle = t1.native_handle();

    // Set thread policy and priority via the handle.
    constexpr int maxPrio{99};
    sched_param sched{maxPrio};
    pthread_setschedparam(handle, SCHED_FIFO, &sched);

    t1.join();
    return 0;
}

```

```
}
```

6.1.4 What is a data race?

A data race occurs when:

- Two threads access the same memory location.
- At least one access is a write operation.
- No synchronization mechanism is used.

Result:

- Leads to undefined behavior, even if the program appears to work during testing.
- The program may work correctly until it suddenly does not.

6.1.5 Mutexes and critical sections

A mutex is a lock that makes it easy to protect critical sections, i.e., regions containing resources shared between threads.

A mutex is implemented using the `std::mutex` type from the `<mutex>` header:

```
std::mutex mutex{};
```

A mutex can be used to protect resources shared between threads:

- A given mutex is shared between the threads.
- Shared resources are protected by locking the mutex using `lock()`. This call blocks the current thread until the resource becomes available.
- Shared resources are made available to other threads by unlocking the mutex using `unlock()`.
- A mutex can protect multiple related variables simultaneously.

The example below demonstrates how a mutex is shared between two functions running in separate threads. Each thread temporarily gains exclusive access to the shared resources:

- The threads take turns using `std::cout` for terminal output.
- A mutex is used to protect `std::cout` in each thread:
 - A thread reserves `std::cout` as soon as it becomes available by locking the mutex.
 - While `std::cout` is reserved, other threads must wait.
 - Once access is granted, the thread performs its output operation.
 - After the output is complete, the mutex is unlocked to make `std::cout` available to the other thread.

Note: These examples intentionally omit a stop mechanism and are used for illustration purposes only.

```

std::mutex mutex{};

// Function running in thread 1.
void thread1Func()
{
    while (true)
    {
        // Lock mutex before entering critical region.
        mutex.lock();
        {
            // Critical region - Feel free to access shared resources here.
            // Note: Terminal print is a resource shared between the threads.
            std::cout << "Thread1 currently has access to the terminal!\n";
        }
        // Unlock the mutex to make shared data available for the other thread.
        mutex.unlock();
    }
}

// Function running in thread 2.
void thread2Func()
{
    while (true)
    {
        // Lock mutex before entering critical region.
        mutex.lock();
        {
            // Critical region - Feel free to access shared resources here.
            // Note: Terminal print is a resource shared between the threads.
            std::cout << "Thread2 currently has access to the terminal!\n";
        }
        // Unlock the mutex to make shared data available for other threads.
        mutex.unlock();
    }
}

```

We can also use `std::lock_guard<T>` from `<mutex>` to automatically lock and unlock a mutex:

```
std::lock_guard<std::mutex> lock{mutex};
```

This works as follows:

- The mutex is automatically locked when the lock guard is created.
- The mutex is automatically unlocked when the lock guard is destroyed, i.e., when it goes out of scope.

The code snippet above can be modified to use a lock guard, as shown below:

```
std::mutex mutex{};

// Function running in thread 1.
void thread1Func()
{
    while (true)
    {
        {
            // Lock mutex before entering critical region.
            std::lock_guard<std::mutex> lock{mutex};

            // Critical region - Feel free to access shared resources here.
            // Note: Terminal print is a resource shared between the threads.
            std::cout << "Thread1 currently has access to the terminal!\n";
        }
    }
}

// Function running in thread 2.
void thread2Func()
{
    while (true)
    {
        {
            // Lock mutex before entering critical region.
            std::lock_guard<std::mutex> lock{mutex};

            // Critical region - Feel free to access shared resources here.
            // Note: Terminal print is a resource shared between the threads.
            std::cout << "Thread2 currently has access to the terminal!\n";
        }
    }
}
```

6.1.6 std::atomic

Individual variables shared between threads, such as stop flags, can be made atomic instead of being protected by a mutex. This simplifies the implementation because no mutex needs to be created, locked, or unlocked:

- Atomic variables are implemented using the `std::atomic` type from the `<atomic>` header.
- For example, an atomic boolean variable named `stopFlag` can be implemented as:

```
std::atomic<bool> stopFlag{false};
```

To modify and read the stop flag, the methods `store()` and `load()` can be used:

```
// Set the stop flag.
stopFlag.store(true);

// Check the stop flag.
const bool stop{stopFlag.load()};
```

Atomic stop flags are commonly used to stop multiple threads simultaneously.

Prefer `std::atomic` over a volatile variable; volatile gives no thread-safety guarantees in C++.

Note:

- An atomic variable only protects a single variable.
- If multiple variables must be updated together, a mutex is required, even if each individual variable is atomic.
- A mutex also provides memory synchronization between threads, ensuring that writes performed within a critical section become visible to other threads once the lock is released.

6.1.7 Example: mutex with lock guard and atomic stop flag

The example below demonstrates a typical embedded scenario:

- A transmitter thread (TX) periodically produces messages.
- A receiver thread (RX) reads and consumes new data.
- A third thread stops program execution after a specified timeout.

```
#include <atomic>
#include <chrono>
#include <cstdint>
#include <cstdlib>
#include <functional>
#include <iostream>
#include <mutex>
#include <thread>

namespace
{
```

```

/**
 * @brief Shared memory used by TX and RX threads.
 *
 *      Access must be protected by mutex to avoid data races.
 */
struct SharedMem
{
    /** Shared data. */
    std::uint16_t data{};

    /** Indicate that new data is available (protected by a mutex, hence not ↵
    ↵ atomic). */
    bool newData{false};
};

/** Mutex to protect critical regions. */
std::mutex mutex{};

// -----
void wait_ms(const std::size_t ms) noexcept
{
    // Sleep the calling thread for the given duration.
    std::this_thread::sleep_for(std::chrono::milliseconds(ms));
}

// -----
void txThread(SharedMem& shared, const std::atomic<bool>& stop,
              const std::size_t txInterval_ms) noexcept
{
    std::uint16_t counter{};

    // Run the TX loop as long as the stop flag isn't set.
    while (!stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.
        {
            std::lock_guard<std::mutex> lock{mutex};

            // Produce new data and mark it as available.
            shared.data = counter++;
            shared.newData = true;
            std::cout << "TX: Produced " << shared.data << "!\n";
        }
        wait_ms(txInterval_ms);
    }
}

```

```

    }
}

// -----
void rxThread(SharedMem& shared, const std::atomic<bool>& stop,
              const std::size_t rxInterval_ms) noexcept
{
    // Run the RX loop as long as the stop flag isn't set.
    while (!stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.
        {
            std::lock_guard<std::mutex> lock{mutex};

            // Consume new data if available.
            if (shared.newData)
            {
                std::cout << "RX: Consumed " << shared.data << "!\n";
                shared.newData = false;
            }
        }
        wait_ms(rxInterval_ms);
    }
}

// -----
void stopThread(std::atomic<bool>& stop, const std::size_t timeout_ms) noexcept
{
    // Wait for timeout, then signal all threads to stop.
    wait_ms(timeout_ms);
    stop.store(true);
}

} // namespace

// -----
int main()
{
    constexpr std::size_t txInterval_ms{1000U};
    constexpr std::size_t rxInterval_ms{100U};
    constexpr std::size_t threadTimeout_ms{10000U};

    // Shared memory structure.
    SharedMem sharedMem{};

```

```

// Atomic stop flag shared between threads.
// Atomic is sufficient here, since it protects a single variable.
std::atomic<bool> stopFlag{false};

// Create and run threads during ten seconds to simulate communication.
std::thread t1{txThread, std::ref(sharedMem), std::cref(stopFlag), ↵
    ↵ txInterval_ms};
std::thread t2{rxThread, std::ref(sharedMem), std::cref(stopFlag), ↵
    ↵ rxInterval_ms};
std::thread t3{stopThread, std::ref(stopFlag), threadTimeout_ms};

// Synchronize the threads.
t1.join();
t2.join();
t3.join();
return 0;
}

```

6.1.8 Why is a mutex needed?

SharedMem contains resources shared between the TX and RX threads that belong together:

- data: Data shared between the threads.
- newData: Indicates that new data is available.

If TX and RX execute simultaneously without a mutex, RX may read the data while it is being updated. A mutex ensures that both fields are updated completely before another thread is allowed to read them.

6.1.9 std::future and std::async

A std::future represents the result of an asynchronous operation, i.e., the return value from a task running in another thread. std::async is the simplest way to launch such a task; it returns a std::future bound to its result. No manual thread management is needed, making it convenient for one-shot background computations.

std::async accepts an optional launch policy as its first argument:

- std::launch::async: The task runs immediately in a new thread.
- std::launch::deferred: The task is postponed and runs synchronously on the calling thread the first time future.get() or future.wait() is called. No new thread is created.

If no policy is specified, the implementation is free to choose either, which can lead to surprising behaviour. Prefer being explicit:

```

// Always run in a new thread.
auto asyncFuture = std::async(std::launch::async, computeChecksum, frame, ↵
    ↵ headerLen);

// Run on the calling thread when deferredFuture.get() is called.
auto deferredFuture = std::async(std::launch::deferred, computeChecksum, frame, ↵
    ↵ headerLen);

```

Below is an example of a checksum computation of a frame being performed while printing the frame content:

```

#include <cstdint>
#include <cstdint>
#include <cstdio>
#include <future>

namespace
{
// -----
[[nodiscard]] std::uint16_t computeChecksum(const std::uint8_t* buf, const std::size_t ↵
    ↵ bufLen) noexcept
{
    // Check buffer, return 0 if invalid.
    if ((nullptr == buf) || (0U == bufLen)) { return 0U; }
    std::uint16_t chk{};

    // Compute the checksum by adding each byte of the buffer.
    for (std::size_t i{}; i < bufLen; ++i)
    {
        chk += buf[i];
    }
    return chk;
}

// -----
void printBuf(const std::uint8_t* buf, const std::size_t bufLen) noexcept
{
    if ((nullptr == buf) || (0U == bufLen)) { return; }
    std::printf("-----\n");
    ↵ -----\n";
    for (std::size_t i{}; i < bufLen; ++i) { std::printf("0x%02X ", buf[i]); }
    std::printf("\n-----\n\n");
    ↵ -----\n\n");
}

```

```

} // namespace

// -----
int main()
{
    constexpr std::size_t headerLen{8U};
    constexpr std::size_t chkLen{2U};
    constexpr std::size_t frameLen{headerLen + chkLen};
    constexpr std::size_t chkOffset1{frameLen - 2U};
    constexpr std::size_t chkOffset2{frameLen - 1U};
    constexpr std::size_t shift{8U};

    // Create a frame without checksum.
    std::uint8_t frame[frameLen]{0xA5U, 0xF7U, 0x00U, 0x02U, 0x25U, 0x01U, 0xFAU, ↵
    ↵ 0xABU};

    // Compute the checksum asynchronously; print the frame during the computation.↵
    ↵
    auto future = std::async(std::launch::async, computeChecksum, frame, headerLen↵
    ↵ );
    printBuf(frame, headerLen);

    // Add the checksum to the frame, then print the frame again.
    const auto chk = future.get();
    frame[chkOffset1] = static_cast<std::uint8_t>(chk >> shift);
    frame[chkOffset2] = static_cast<std::uint8_t>(chk);
    printBuf(frame, frameLen);
    return 0;
}

```

6.1.10 `std::condition_variable`: signalling between threads

A condition variable lets a thread sleep until another signals it; no busy-waiting or polling required:

- Condition variables are appropriate when one thread produces data and another consumes it.
- In C++, a condition variable is implemented as shown below:

```

#include <condition_variable>

std::condition_variable condition{};

```

The example below demonstrates a typical embedded scenario:

- A transmitter thread (TX) periodically produces data and pushes it onto a shared queue.
- A receiver thread (RX) waits for new data using a condition variable and pops it from the queue.
- A third thread stops program execution after a specified timeout by setting a stop flag and notifying all waiting threads.

```
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdint>
#include <cstdio>
#include <functional>
#include <mutex>
#include <queue>
#include <thread>

namespace
{
    /**
     * @brief Shared data structure.
     */
    struct Shared
    {
        /** Mutex to protect access to queue. */
        std::mutex mutex{};

        /** Stop flag to signal all threads to terminate. */
        std::atomic<bool> stop{false};

        /** Condition variable to notify rxThread when new data is available. */
        std::condition_variable condition{};

        /** Queue of data shared between txThread and rxThread. */
        std::queue<std::uint32_t> queue{};
    };

    // -----
    [[nodiscard]] bool isDataAvailable(const Shared& shared) noexcept
    {
        // Return true if data is available or the stop flag is set.
        return !shared.queue.empty() || shared.stop.load();
    }
}
```

```

// -----
void stopThread(Shared& shared, const std::uint16_t duration_ms) noexcept
{
    // Wait for the given duration to pass before setting the stop flag.
    std::this_thread::sleep_for(std::chrono::milliseconds(duration_ms));

    // Set stop flag while holding the mutex, then wake up all threads for clean ↵
    ↵ termination.
    {
        std::lock_guard<std::mutex> lock{shared.mutex};
        shared.stop.store(true);
    }
    shared.condition.notify_all();
}

// -----
void txThread(Shared& shared) noexcept
{
    constexpr std::uint8_t period_ms{100U};
    std::uint32_t counter{};

    // Run thread as long as the stop flag is not set.
    while (!shared.stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.
        {
            // Add number to queue.
            std::lock_guard<std::mutex> lock{shared.mutex};
            shared.queue.push(counter);
            std::printf("Pushed number %u to shared queue!\n", counter++);
        }
        // Notify the RX thread that data is available.
        shared.condition.notify_one();
        std::this_thread::sleep_for(std::chrono::milliseconds(period_ms));
    }
}

// -----
void rxThread(Shared& shared) noexcept
{
    // Run thread as long as the stop flag is not set.
    while (!shared.stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.

```

```

    {
        // Wait for data to be available.
        std::unique_lock<std::mutex> lock{shared.mutex};
        auto predicate = [&shared] { return isDataAvailable(shared); };
        shared.condition.wait(lock, predicate);

        // Check if stop flag was set during the wait, terminate thread if so
        ← true.
        if (shared.stop.load()) { break; }

        // Pop value from the queue when available.
        if (!shared.queue.empty())
        {
            const auto num = shared.queue.front();
            shared.queue.pop();
            std::printf("Popped number %u from shared queue!\n", num);
        }
    }
}
} // namespace

// -----
int main()
{
    constexpr std::uint16_t duration_ms{2000U};

    // Create shared data structure.
    Shared shared{};

    // Create threads.
    std::thread t1{stopThread, std::ref(shared), duration_ms};
    std::thread t2{txThread, std::ref(shared)};
    std::thread t3{rxThread, std::ref(shared)};

    // Synchronize threads.
    t1.join();
    t2.join();
    t3.join();
    return 0;
}

```

Notes:

- `std::unique_lock` is used here instead of `std::lock_guard` because `condition.wait()` needs to temporarily release the mutex while the thread is sleeping.
- A lambda is used to wrap `isDataAvailable` together with the shared state it depends on, since `condition.wait()` requires a predicate callable with no arguments. The lambda captures shared by reference (`[&shared]`), so it can pass it into `isDataAvailable(shared)` each time `wait()` re-checks the condition, without copying the Shared struct (which wouldn't even compile, since it contains a non-copyable `std::atomic`).
- If `stopThread` only set `stop = true` without calling `notify_all()`, the RX thread could get stuck indefinitely inside `condition.wait()`: once `stop` is set, TX exits its loop and stops pushing values to the queue and calling `notify_one()`, so nothing would wake the RX thread up. Calling `notify_all()` ensures any thread sleeping in `wait()` is woken up. The `|| shared.stop.load()` clause in `isDataAvailable` then makes the predicate return true, allowing `wait()` to return so the thread can detect the stop flag and exit cleanly.
- `stopThread` sets the stop flag while holding the mutex, even though `stop` is atomic. `wait()` checks the predicate and goes to sleep while holding the mutex, so a change made under the same mutex cannot land in between. Without the lock, `stop` could be set, and `notify_all()` called, just after RX found the predicate false but before it went to sleep; the notification would then be lost, and RX would sleep forever. Any variable a predicate depends on must be modified while holding the mutex, atomic or not.

You can also invoke non-static member functions as thread entry points, rather than passing the shared struct by reference into free functions, as shown below:

```
#include <atomic>
#include <chrono>
#include <condition_variable>
#include <cstdint>
#include <cstdio>
#include <functional>
#include <mutex>
#include <queue>
#include <thread>

namespace
{
    /**
     * @brief Shared data structure.
     */
    struct Shared
    {
        /** Mutex to protect access to queue. */
```

```

std::mutex mutex{};

/** Stop flag to signal all threads to terminate. */
std::atomic<bool> stop{false};

/** Condition variable to notify rxThread when new data is available. */
std::condition_variable condition{};

/** Queue of data shared between txThread and rxThread. */
std::queue<std::uint32_t> queue{};

// -----
void stopThread(const std::uint16_t duration_ms) noexcept
{
    // Wait for the given duration to pass before setting the stop flag.
    std::this_thread::sleep_for(std::chrono::milliseconds(duration_ms));

    // Set stop flag while holding the mutex, then wake up all threads for ↵
    ↵ clean termination.
    {
        std::lock_guard<std::mutex> lock{mutex};
        stop.store(true);
    }
    condition.notify_all();
}

// -----
void txThread() noexcept
{
    constexpr std::uint8_t period_ms{100U};
    std::uint32_t counter{};

    // Run thread as long as the stop flag is not set.
    while (!stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.
        {
            // Add number to queue.
            std::lock_guard<std::mutex> lock{mutex};
            queue.push(counter);
            std::printf("Pushed number %u to shared queue!\n", counter++);
        }
        // Notify the RX thread that data is available.
        condition.notify_one();
    }
}

```

```

        std::this_thread::sleep_for(std::chrono::milliseconds(period_ms));
    }
}

// -----
void rxThread() noexcept
{
    // Run thread as long as the stop flag is not set.
    while (!stop.load())
    {
        // Enter critical section, lock the mutex to protect shared memory.
        {
            // Wait for data to be available.
            std::unique_lock<std::mutex> lock{mutex};
            auto predicate = [this] { return isDataAvailable(); };
            condition.wait(lock, predicate);

            // Check if stop flag was set during the wait, terminate thread if-
            true.
            if (stop.load()) { break; }

            // Pop value from the queue when available.
            if (!queue.empty())
            {
                const auto num = queue.front();
                queue.pop();
                std::printf("Popped number %u from shared queue!\n", num);
            }
        }
    }
}

private:
// -----
[[nodiscard]] bool isDataAvailable() noexcept
{
    // Return true if data is available or the stop flag is set.
    return !queue.empty() || stop.load();
}

};
} // namespace

// -----

```

```

int main()
{
    constexpr std::uint16_t duration_ms{2000U};

    // Create shared data structure.
    Shared shared{};

    // Create threads.
    std::thread t1{&Shared::stopThread, &shared, duration_ms};
    std::thread t2{&Shared::txThread, &shared};
    std::thread t3{&Shared::rxThread, &shared};

    // Synchronize threads.
    t1.join();
    t2.join();
    t3.join();
    return 0;
}

```

Notes:

- Since `isDataAvailable` and the thread functions now live inside `Shared`, they have implicit access to all of `Shared`'s members through `this`. The lambda therefore captures `[this]` instead of `[&shared]`, giving it access to the whole enclosing object rather than one external reference. This also means that it can reach `isDataAvailable()` even though it's private.
- The same stuck-wait concern applies here as in the free-function version above: `notify_all()` in `stopThread` is essential, and so is setting `stop` while holding the mutex. Without either, the RX thread could remain blocked in `condition.wait()` indefinitely once TX stops pushing new values.

6.1.11 Priority inversion

Priority inversion is a scheduling bug that can occur when threads of different priorities share a mutex. Here is what happens, step by step:

1. A **low-priority thread** locks a mutex and begins a long operation.
2. A **high-priority thread** wakes up and tries to lock the same mutex. Since the mutex is locked by the low-priority thread, the high-priority thread blocks and waits.
3. A **medium-priority thread** wakes up and preempts (forcibly takes the CPU from) the low-priority thread, because it has higher priority. This thread does not use the mutex at all; it is doing its own completely unrelated work. The scheduler simply gives it the CPU because it outranks the currently running low-priority thread.

4. The medium-priority thread runs to completion, preventing the low-priority thread from finishing and releasing the mutex.
5. The high-priority thread remains blocked, not because of its own work, but because the medium-priority thread delayed the lock release.

Only once the medium-priority thread has finished can the low-priority thread run again, complete its operation, and release the mutex; only then can the high-priority thread lock it and run. The high-priority task ends up waiting on the medium-priority one; an inversion of the intended scheduling order.

Note that the preemption in step 3 is a normal scheduler decision:

- The OS sees a higher-priority thread become runnable and gives it the CPU. The mutex plays no role there.
- The problem is that `std::mutex` has no way to prevent the inversion once it starts; it does not support **priority inheritance**, a feature where the mutex temporarily boosts the low-priority thread's priority (to match the blocked high-priority thread) so it can finish and release the lock before any medium-priority thread cuts in. On an RTOS, this can be addressed with a priority-inheritance mutex (see the notes below).

Priority inversion can be illustrated as shown in Figure 6.1.

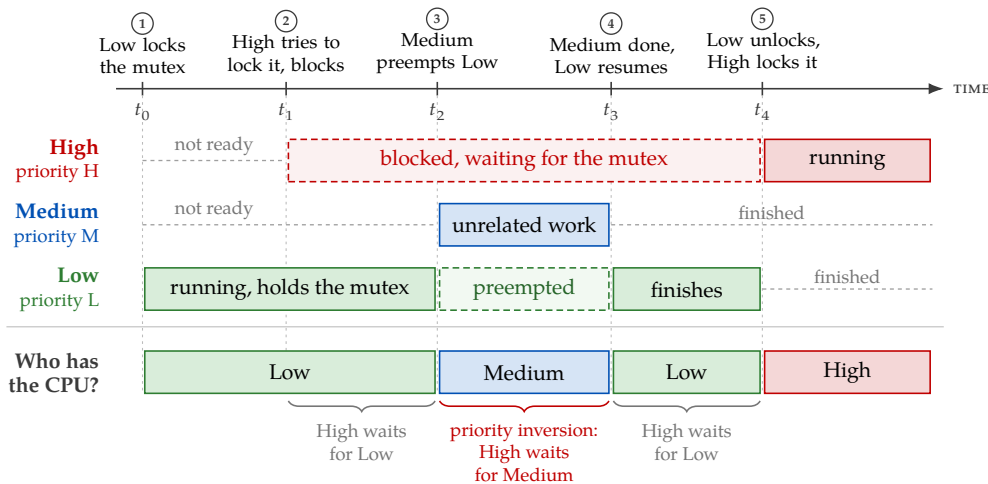


Figure 6.1 Priority inversion on one CPU. High blocks on the mutex Low holds (event 2), which is expected; then Medium, which never touches the mutex, preempts Low (event 3) and runs to completion. Only when Medium is done can Low finish and unlock (event 5), so High waits for the whole of Medium's work as well as Low's critical section.

The following code shows a conceptual example of the same scenario:

```

// Mutex used to protect access of shared resources.
std::mutex mutex{};

// -----
void lowPriorityTask() noexcept
{
    // Holds the mutex during a long operation, blocking highPriorityTask.
    std::lock_guard<std::mutex> lock{mutex};
    runLongOperation();
}

// -----
void mediumPriorityTask() noexcept
{
    // Preempts lowPriorityTask, preventing it from releasing the mutex.
    runCpuIntensiveWork();
}

// -----
void highPriorityTask() noexcept
{
    // Blocks until lowPriorityTask releases the mutex.
    std::lock_guard<std::mutex> lock{mutex};
    resetWatchdog();
}

```

Notes:

- Priority inversion cannot be reliably demonstrated with standard C++ on a desktop OS. `std::thread` assigns no priority by default, so the scheduler treats all threads equally. The example above is a conceptual illustration only.
- If you are mixing thread priorities on an RTOS, replace `std::mutex` with the platform's native priority-inheritance mutex. `std::thread` itself is not the problem.

6.1.12 Rule of thumb

- `std::atomic` is suitable for individual variables, such as stop flags.
- `std::mutex` is required when multiple resources, such as variables, are shared between threads.
- `std::condition_variable` replaces polling loops when a thread must wait for an event produced by another thread.
- `std::async` / `std::future` is convenient for one-shot background tasks where you need the result later.

- Be aware of **priority inversion** whenever threads of mixed priority share a mutex.

6.1.13 Summary

In this section, we have seen:

- The difference between concurrency and true parallelism.
- How the scheduler decides which thread runs; time-slicing vs. cooperative.
- How to create threads using `std::thread`.
- How to protect shared data using `std::mutex`.
- How to use `std::atomic` correctly.
- The difference between atomic variables and mutexes.
- How to signal between threads efficiently using `std::condition_variable`.
- How to launch background tasks using `std::async` and `std::future`.
- What priority inversion is and why it matters in embedded systems.

These concepts form the foundation of safe parallel programming in modern C++.

6.2 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what a thread is and how it is created using `std::thread`.
- Explain the difference between concurrency and parallelism.
- Explain preemptive and cooperative scheduling.
- Explain what a data race is and why it leads to *undefined behavior*.
- Use `std::mutex` and `std::lock_guard` correctly.
- Explain when `std::atomic` is sufficient and when `std::mutex` is required.
- Use `std::condition_variable` to signal between threads without busy-waiting.
- Launch asynchronous tasks using `std::async` and retrieve results via `std::future`.
- Explain what priority inversion is and why `std::mutex` does not solve it on an RTOS.

Questions to test yourself

You should be able to explain:

1. What is the difference between a mutex and an atomic?
2. What is a data race?
3. What is the difference between concurrency and parallelism?
4. When would you use a `condition_variable` instead of polling in a loop?
5. What is priority inversion, and why should you be careful with `std::mutex` on an RTOS?

Looking ahead

This is the last chapter, and with it the book's main text is complete. What remains is to work the exercises in §6.3 and compare your programs with the solutions in the companion repository. After that, the two written papers in Appendices A to E are there to test, on paper and without a compiler, how much of all six chapters you can reconstruct on your own.

6.3 Exercises

How to work these. These exercises reinforce the concepts from §6.1. Each set names the file or directory to work in: write it in its own directory under `lectures/L06/exercises` in the companion repository, with the file names its README gives. Build it with the Makefile from §1.1.3, and add `-pthread` to the compiler flags (`CXX_FLAGS := -Wall -Werror -std=c++17 -pthread`). GCC requires it for programs that use `std::thread`; without it, the program may fail to link, or fail at runtime, on systems with a C library older than glibc 2.34.

How to check your work. Run `make test` from the repository root. A set's tests switch on as soon as its files exist; a set you have not started is reported as `SKIP`. Each file grows through its set, and the tests check where it ends up. Threads run in whatever order the scheduler picks, so the tests check what a correct program guarantees, such as exact counts and threads that stop when asked, rather than the order of its output.

The solutions are published in the companion repository, one program per set under `lectures/L06/appendix/solutions` (Sets 4 and 5 share one), with the answers to the reflection questions in the README beside them. Try each exercise yourself before looking at them.

EXERCISE SET 1

Threads and Atomic Stop Flag

Create a file named `exercise1.cpp` to work in for this exercise set.

Exercise 6.1 Creating a worker thread

CODE

In this exercise you will create a simple worker function that runs in a separate thread.

Tasks

In `exercise1.cpp`, create a function named `workerThread()`.

The function shall:

- Take a `std::uint8_t` named `printCount`: The number of times to print.
- Take a `std::uint16_t` named `printSpeed_ms`: The delay in milliseconds between each print.
- Return `void`.
- Print "Worker thread running!" `printCount` times.

- Sleep for `printSpeed_ms` milliseconds between each print using `std::this_thread::sleep_for()` with `std::chrono::milliseconds`.

Exercise 6.2 Starting and joining the thread

CODE

In `exercisel.cpp`:

- Create a `std::thread` that runs `workerThread()`, passing 50 as `printCount` and 5000 as `printSpeed_ms`.
- Print "Main thread running!" from `main()`.
- Call `join()` on the worker thread before returning from `main()`.

Reflection

- What happens if `join()` is not called?
- Which thread prints first, the main thread or the worker thread?
- Is the print order guaranteed?

Exercise 6.3 Atomic stop flag

CODE

In this exercise you will modify the worker thread so that it runs until a stop flag is set.

Tasks

Create an atomic boolean variable in `main()`:

```
std::atomic<bool> stop{false};
```

Modify `workerThread()` so that it:

- Replaces the `printCount` parameter with a `const std::atomic<bool>&` named `stop`, placed after `printSpeed_ms`.
- Runs as long as `stop.load()` is `false`.
- Prints "Worker thread running!" once every `printSpeed_ms` milliseconds.

In `main()`:

- Start the worker thread.
- Let the main thread sleep for 3000 ms.
- Set the stop flag using `stop.store(true)`.
- Join the worker thread.

Reflection

- Why is `std::atomic<bool>` suitable for this stop flag?
- Would a normal `bool` be safe here?
- Why is `load()` used when reading the flag?
- Why is `store()` used when writing the flag?

EXERCISE SET 2

Shared Counter with Data Race

Create a file named `exercise2.cpp` to work in for this exercise set.

Exercise 6.4 Shared counter without synchronization

CODE

In this exercise you will intentionally create a data race.

Tasks

Create a shared counter variable in `main()`:

```
std::uint32_t counter{};
```

Create a function named `incrementCounter()`. The function shall:

- Take a reference to the counter.
- Take a number of iterations (`std::uint32_t`).
- Increment the counter once per iteration.
- Be marked `noexcept`.

In `main()`:

- Create two threads that both call `incrementCounter()`.
- Let each thread increment the counter 1000000 times.
- Join both threads.
- Print the final counter value.

Reflection

- What final value do you expect?
- Do you always get the expected value?
- Why is this program unsafe?
- Which condition makes this a data race?

Exercise 6.5 Protecting the counter with a mutex

CODE

In this exercise you will fix the data race using `std::mutex`.

Tasks

Create a mutex in `main()`:

```
std::mutex mutex{};
```

Modify `incrementCounter()` so that it:

- Takes a reference to the mutex.
- Locks the mutex before incrementing the counter.

- Uses `std::lock_guard<std::mutex>`.
- Unlocks automatically when the lock guard goes out of scope.

Reflection

- What final counter value do you get now?
- Why does the mutex solve the problem?
- Why is `std::lock_guard` safer than manually calling `lock()` and `unlock()`?

Exercise 6.6 Atomic counter

CODE

In this exercise you will replace the mutex-protected counter with an atomic counter.

Tasks

Replace the counter with:

```
std::atomic<std::uint32_t> counter{};
```

Modify `incrementCounter()` so that it:

- Takes a reference to the atomic counter.
- Increments it once per iteration.

Reflection

- What final counter value do you get?
- Why is `std::atomic<std::uint32_t>` sufficient in this exercise?
- Would an atomic counter still be sufficient if several related variables had to be updated together?

EXERCISE SET 3

Shared Data Protected by Mutex

Create a file named `exercise3.cpp` to work in for this exercise set.

Exercise 6.7 Shared memory structure

CODE

In this exercise you will create shared data that must be protected by a mutex.

Tasks

In `exercise3.cpp`, create a structure named `SharedMem` with the following members:

- `data`: A 16-bit unsigned value.
- `newData`: True if new data is available, false otherwise.

The structure represents data shared between a transmitter thread and a receiver thread.

Exercise 6.8 Transmitter thread

CODE

In `exercise3.cpp`, create a function named `txThread()`.

The function shall:

- Take a reference to `SharedMem`.
- Take a reference to `std::mutex`.
- Take a reference to `const std::atomic<bool>` named `stop`.
- Run until the stop flag is set.
- Produce an incrementing `std::uint16_t` value.
- Store the value in `shared.data`.
- Set `shared.newData` to `true`.
- Sleep for 1000 ms between transmissions.

The update of `shared.data` and `shared.newData` shall be protected using a lock guard.

Exercise 6.9 Receiver thread

CODE

In `exercise3.cpp`, create a function named `rxThread()`.

The function shall:

- Take a reference to `SharedMem`.
- Take a reference to `std::mutex`.
- Take a reference to `const std::atomic<bool>` named `stop`.
- Run until the stop flag is set.
- Check whether new data is available.
- Print the received data if `shared.newData` is `true`.
- Set `shared.newData` to `false` after consuming the data.
- Sleep for 100 ms between receive attempts.

The check and update of `shared.data` and `shared.newData` shall be protected by the same mutex.

Exercise 6.10 Running TX and RX threads

CODE

In `exercise3.cpp`:

- Create one `SharedMem` object.
- Create one `std::mutex`.
- Create one atomic stop flag.
- Start one TX thread.
- Start one RX thread.
- Let the main thread sleep for 10000 ms.
- Set the stop flag.
- Join both threads.

Reflection

- Why must data and newData be protected by the same mutex?
- Why is newData not atomic in this design?
- What could happen if TX updated data and newData without locking the mutex?
- Why is the stop flag atomic instead of mutex-protected?
- Could `std::atomic<SharedMem>` be used instead of a mutex? Why or why not?

EXERCISE SET 4**Thread-Safe Stub Driver**

Create the following directory structure to work in for this exercise set:

```

Makefile
include/
  driver/
    counter/
      interface.hpp
      stub.hpp
source/
  main.cpp

```

Note: When a class overrides a method that is marked `[[nodiscard]]` in the interface, repeat the attribute on the overriding method as well. The attribute is not inherited; see §3.2.

Exercise 6.11 Counter driver interface**CODE**

In this exercise you will design an interface named `driver::counter::Interface`.

The interface represents a generic counter driver.

Tasks

Design a class named `driver::counter::Interface`.

All methods except the destructor shall be pure virtual (`= 0`).

a) Destructor

Add a destructor that is:

- Virtual.
- Marked `noexcept`.
- Implemented using `= default`.

b) Checking initialization state

Add a pure virtual method `isInitialized()` that:

- Returns `true` if the driver is initialized, `false` otherwise.
- Does not modify the object.
- Is marked `noexcept` and `[[nodiscard]]`.

c) Reading the counter

Add a pure virtual method `value()` that:

- Returns a `std::optional<std::uint32_t>` from `<optional>`: the current counter value, or `std::nullopt` if the driver has no value to give.
- Does not modify the object.
- Is marked `noexcept` and `[[nodiscard]]`.

d) Incrementing the counter

Add a pure virtual method `increment()` that:

- Increments the counter by one.
- Does not return a value.
- Is marked `noexcept`.

e) Resetting the counter

Add a pure virtual method `reset()` that:

- Resets the counter to zero.
- Does not return a value.
- Is marked `noexcept`.

Exercise 6.12 Thread-safe stub counter

CODE

In this exercise you will implement `driver::counter::Stub`.

The class shall:

- Inherit from `driver::counter::Interface`.
- Be marked `final`.
- Be safe to use from multiple threads (via a mutex).

a) Member variables

Add two private member variables:

- `myMutex`:
 - Type: `std::mutex`.
 - Protects access to `myValue` below.
 - Must be marked `mutable` because `value()` is a `const` method but still needs to lock the mutex.

- `myValue`:
 - Type: `std::uint32_t`.
 - Stores the counter value.

b) Constructor

Add a default constructor that:

- Initializes the counter to zero.
- Is marked `noexcept`.

c) Destructor

Add a destructor that is:

- Public.
- Marked `noexcept`.
- Declared with `override`.
- Implemented using `= default`.

d) Implement interface methods

Implement all methods required by the interface:

- `isInitialized()` shall return `true`.
- `value()` shall:
 - Lock the mutex using `std::lock_guard<std::mutex>`.
 - Return `myValue`.
- `increment()` shall:
 - Lock the mutex using `std::lock_guard<std::mutex>`.
 - Increment `myValue`.
- `reset()` shall:
 - Lock the mutex using `std::lock_guard<std::mutex>`.
 - Set `myValue` to zero.

e) Disable copy and move semantics

Delete the following functions in the public section of the class:

- Copy constructor.
- Move constructor.
- Copy assignment operator.
- Move assignment operator.

Exercise 6.13 Testing the thread-safe stub

CODE

Create a function named `counterThread()`.

The function shall:

- Take a reference to `driver::counter::Interface`.
- Take a `std::uint32_t` named `iterations`, which is the number of times to call `increment()`.
- Be marked `noexcept`.

In `main.cpp`:

- Create one `driver::counter::Stub`.
- Start two threads, both running `counterThread()` — the first with 100U iterations, the second with 200U.
- Join both threads.
- Print the final counter value using `value()`, after checking that a value was returned.

Reflection

- Why does `value()` need to lock the mutex even though it only reads the value?
- Why is `myMutex` marked `mutable`?
- What would happen if `increment()` did not lock the mutex?
- Could `std::atomic<std::uint32_t>` be used instead in this specific class?
- Why might a mutex still be preferable in a more complex driver?

EXERCISE SET 5

Atomic State and Mutex-Protected Data**Exercise 6.14** Driver state

CODE

In this exercise you will extend the counter stub with an initialization state.

Tasks

Add a private member variable to `driver::counter::Stub`:

```
std::atomic<bool> myInitialized;
```

Initialize it to `true` in the constructor.

Exercise 6.15 Initialization methods

CODE

Add a public method `setInitialized()` specific to the stub. The method shall:

- Take a `bool` named `initialized`.
- Write to `myInitialized` using `store()`.
- Be marked `noexcept`.

Override `isInitialized()` from the interface so that it:

- Reads `myInitialized` using `load()`.
- Returns the current initialization state instead of always returning `true`.

Exercise 6.16 Guarding counter operations

CODE

Modify `increment()`, `value()`, and `reset()`:

- `increment()` shall do nothing if the driver is not initialized.
- `value()` shall return `std::nullopt` if the driver is not initialized.
- `reset()` shall do nothing if the driver is not initialized.

The counter value itself shall still be protected by the mutex.

Reflection

- Why is `std::nullopt` a better answer from `value()` than `0U` when the driver is not initialized?
- Why is `myInitialized` suitable as an atomic variable?
- Why is `myValue` still protected by a mutex?
- Would it be safe to make both variables atomic if future requirements added more related state?
- What is the main design difference between a thread-safe flag and thread-safe shared data?
- What could happen if `setInitialized(false)` is called between the initialization check and the mutex lock in `increment()`? How would you fix this?

EXERCISE SET 6**Condition Variable**

Create a file named `exercise6.cpp` by copying `exercise3.cpp`. All modifications in this exercise set are made to that file.

Exercise 6.17 Replacing polling with a condition variable

CODE

In Exercise 6.9 the receiver thread checks for new data by polling in a loop with a 100 ms sleep. In this exercise you will replace that pattern with a `std::condition_variable` so the receiver wakes up immediately when data is available.

Tasks

Add a `std::condition_variable` named `cv` alongside the existing mutex.

Create a helper function named `hasNewData()` to use as the wake-up predicate. The function shall:

- Take a `const SharedMem&`.
- Take a `const std::atomic<bool>&` named `stop`.
- Return `true` if `shared.newData` is `true` or the stop flag is set, `false` otherwise.
- Be marked `noexcept` and `[[nodiscard]]`.

Modify `txThread()` so that it calls `cv.notify_one()` after releasing the lock guard that protects the data update.

Modify `rxThread()` so that instead of sleeping it:

- Acquires a `std::unique_lock<std::mutex>`.
- Calls `cv.wait()` with the lock and a predicate that calls `hasNewData()`.

Note: The predicate is passed as a lambda; the `[&]` syntax captures `shared` and `stop` by reference so the predicate can access both. Lambdas are covered in §6.1.10.

In `main()`, set the stop flag while holding the mutex (in a scope of its own with a `std::lock_guard<std::mutex>`), then call `cv.notify_all()` to wake the receiver thread so it can exit cleanly.

Note: The stop flag is atomic, but it is also part of the predicate, and a variable the predicate depends on must be modified while holding the mutex. Otherwise the flag could be set, and `notify_all()` called, just after the receiver found the predicate `false` but before it went to sleep. That notification would be lost, and the receiver would never wake up.

Reflection

- Why is `std::unique_lock` required here instead of `std::lock_guard`?
- What is a spurious wake-up, and how does the predicate protect against it?
- Why must the stop flag also be checked inside the predicate?
- How does this design differ from the polling approach in Exercise 6.9?
- What would happen if `notify_all()` was not called after setting the stop flag?
- Why is `notify_all()` used here instead of `notify_one()`?

EXERCISE SET 7

std::async and std::future

Create a file named `exercise7.cpp` to work in for this exercise set.

Exercise 6.18 Offloading a computation

CODE

In this exercise you will use `std::async` to run a validation in the background while the main thread continues working.

Tasks

Create a function named `validateFirmware()`. The function shall:

- Take a `const std::uint8_t*` named `buf` and a `std::uint16_t` named `bufLen`.
- Return `false` if `buf` is `nullptr` or `bufLen` is `0`.
- Sleep for `2000 ms` to simulate a slow flash read before validating.
- Iterate over all bytes in `buf`.
- Return `false` if any byte has the value `0xFFU` (unprogrammed flash).
- Return `true` if the image is valid.
- Be marked `noexcept` and `[[nodiscard]]`.

In `main()`:

- Create a buffer with some test data.
- Launch `validateFirmware()` asynchronously using `std::async` with `std::launch::async`, storing the result in a `std::future<bool>`.
- Print "Booting system while validating firmware..." while validation is running.
- Retrieve the result using `future.get()` and print whether the firmware is valid or not.

Reflection

- What does `std::launch::async` guarantee compared to omitting the launch policy?
- At what point does the main thread block?
- What would happen if `future.get()` was called immediately after `std::async`?

Exercise 6.19 Checking whether the result is ready

CODE

Modify `main()` so that instead of calling `future.get()` directly, it polls in a loop:

- Each iteration shall call `future.wait_for()` with `std::chrono::milliseconds(200U)` and compare the result against `std::future_status::ready`.
- As long as the result is not ready, print "Still waiting for firmware validation..."
- Once the loop exits, retrieve the result using `future.get()`.

Reflection

- How does `wait_for()` differ from `get()`?
- Is it safe to call `get()` after `wait_for()` returns `std::future_status::ready`?

The Self-Assessment Papers

This appendix and the four after it hold two complete three-hour papers for *Modern Embedded C++*, with worked solutions. Appendices [B](#) and [C](#) are the papers, questions only; Appendices [D](#) and [E](#) are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. What you get instead is the exercise sets after every chapter, each with a published solution, and the questions in every chapter's review.

These papers do not replace that. **They are here for you to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them, and neither make `build` nor make `format-check` knows they exist.

What they check is something the exercises cannot. Every exercise is done at a keyboard, with the chapter open beside you and a compiler ready to say no. These papers ask what you can reconstruct with none of that in front of you — which of the rules you have absorbed and which you have merely been looking up.

Take one after the last chapter. Both papers draw on all six chapters, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood.

The two papers

Both cover the whole book, Chapters 1 to 6, at the same weighting. They share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

Question	Topic	Chapters	Marks
1	From C to C++: linkage, keywords, vocabulary	1	12
2	Structs, drivers and object lifetime	1, 2	12
3	Classes: what belongs to the type	2	12
4	Copy, move, and what the compiler writes	2	13
5	Inheritance, interfaces and polymorphism	3	13
6	The factory, and who owns what	4	13
7	Templates and compile-time programming	5	12
8	Threads and synchronization	6	13
			100

Both papers mix theory with code, because the subject does. Roughly half the marks are for prose — what a keyword guarantees, why an interface is worth its vtable, what a data race is — and roughly half are for reading or writing C++ on paper.

The two papers differ in which half they lean on:

- **Paper A leans towards code that is put in front of you.** Seven of its questions hand you a snippet that compiles, passes a smoke test, and is wrong: a software timer with four defects against the book's rules, a driver missing `explicit`, a class whose copy constructor shares a heap block, an interface without a virtual destructor, a factory whose product is leaked, a bit-clearing template that wipes the upper half of a 64-bit register, and a producer/consumer that holds a mutex and still races. Naming the defect is half the marks; saying what it does to the running program is the other half.
- **Paper B leans towards code you write and design decisions you justify.** It asks for a C driver rewritten as a C++ class, an enumeration class and its validity check, a move constructor, a serial interface with every convention §3.2 imposes on one, a stub factory returning `std::unique_ptr`, and a fixed-capacity container with its size in the type.

Neither paper asks anything of a compiler that a careful reader could not answer from the chapters. Where a question turns on a rule that is easy to state and easy to get backwards — which `const` goes in the header and which in the source file, what a user-declared destructor does to the implicit move operations, why `std::move` moves nothing — the solution says so at length, because that is where the marks and the understanding both are.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **C++17 throughout.** Standard headers may be assumed included unless a question asks for them.

- **Code is marked on semantics, not syntax.** A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `virtual` on a destructor costs everything.
- **The book's house style is expected** where a question asks for it: camelCase, type names with a leading capital, `my` on private members and `our` on private statics, a leading capital on a static `constexpr` member, brace initialization, `noexcept` on driver methods, `[[nodiscard]]` on queries and `repeated` on overriding methods, and `copy` and `move` deleted on anything that stands for a unique hardware resource.
- **"State the consequence" means the consequence.** Naming a defect earns half the marks; saying what it does to the running program earns the other half. "This is wrong" scores nothing.
- **Where a question asks for a stated number of defects,** listing more is not penalised, but only that many are marked, so the strongest answers go first.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write C++ for a living, so each carries the reasoning rather than the answer alone. Marks are shown per part.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis. Several questions feed the next part, so an error should be followed through rather than penalised twice.
- **The named traps are worth full marks on their own.** A number of parts exist entirely to see whether a candidate walks into a specific mistake: a shallow copy constructor, a move assignment that forgets to release what it already owns, a non-virtual destructor in an interface, a discarded owning pointer, `1U << bit` on a 64-bit register, a mutex that one of the two threads never locks, a `std::thread` that is neither joined nor detached. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.
- **The discussion parts are not decoration.** "State why" is where the book's actual content is, and a paper marked only on whether the code compiles would pass a candidate who has understood none of it.

Self-Assessment Paper A

Time: 3 hours. **Closed book.** No compiler and no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all six chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix [D](#).

Rubric

C++17 is assumed throughout. Standard headers may be assumed included unless a question asks you to name them.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `virtual` on a destructor costs everything.

Where a question says “in the course’s style”, it means the conventions used in the chapters:

- camelCase for functions and methods, type names with a leading capital.
- Private member variables prefixed `my`, private static members prefixed `our`.
- `static constexpr` members named with a leading capital, e.g. `MaxInstances`.
- Brace initialization, `noexcept` on driver methods, `[[nodiscard]]` on queries and repeated on overriding methods.
- Copy and move operations deleted on a class that represents a unique hardware resource.

Where a question says “state the consequence”, naming the defect earns half the marks and saying what it does to the running program earns the other half. “This is wrong” scores nothing.

Where a question asks you to find defects, the number of defects is stated. Listing more than the stated number is not penalised, but only the stated number is marked, so put your strongest answers first.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	12	12	13	13	13	12	13

Question 1 Crossing the line between C and C++

12 MARKS

(a) A silicon vendor ships a header `hal_gpio.h` and a precompiled library, both built with a C compiler. Your application is C++ and calls `halGpioInit()`. The code compiles, and the linker then reports:

```
undefined reference to `halGpioInit()'
```

Explain the mechanism that produces this error. Write the minimal change to the vendor's header that fixes it while keeping the header usable from C, and state why that change has to be guarded. (4 marks)

(b) State three concrete advantages of `constexpr` over `#define` for a constant such as a clock frequency, and give one thing a `constexpr` value can be used for that a runtime `const` variable cannot. (3 marks)

(c) State what `noexcept` guarantees, what happens at runtime if a `noexcept` function throws anyway, and give two reasons why exceptions are commonly disabled entirely in an embedded build. (3 marks)

(d) Chapter 1 states that a function which may allocate dynamically should **not** be marked `noexcept`. The factory interface in Chapter 4 returns `std::make_unique<...>` and is marked `noexcept`. Reconcile the two, and state exactly which runtime behavior the Chapter 4 choice selects. (2 marks)

Question 2 Object lifetime, and a driver that nearly works

12 MARKS

(a) Consider the program below.

```
#include <cstdint>
#include <cstdio>

namespace driver
{
    struct Gpio
    {
        explicit Gpio(const std::uint8_t pin) noexcept
```

```

        : myPin{pin}
    {
        std::printf("Init %u\n", myPin);
    }

    ~Gpio() noexcept { std::printf("Release %u\n", myPin); }

private:
    const std::uint8_t myPin;
};
} // namespace driver

int main()
{
    driver::Gpio led{9U};
    {
        driver::Gpio button{13U};
        driver::Gpio buzzer{7U};
        std::printf("Inner\n");
    }
    std::printf("Outer\n");
    return 0;
}

```

Write the exact output, in order. Then state the two rules about destructors that your answer depends on. (4 marks)

(b) The software timer below compiles and passes a smoke test. It contains **four** defects against the course's own rules. Identify each one, state the consequence, and give the corrected line. (6 marks)

```

namespace driver
{
    struct Timer
    {
        Timer(const std::uint16_t timeout_ms) noexcept
            : myCounter_ms{0U}
            , myTimeout_ms{timeout_ms}
            , myRunning{false}
        {}

        ~Timer() noexcept = default;

        void start() noexcept { myRunning = true; }
    }
}

```

```

void stop() noexcept { myRunning = false; }

std::uint16_t timeout_ms() noexcept { return myTimeout_ms; }

[[nodiscard]] bool elapsed() noexcept
{
    if (!myRunning) { return false; }
    if (++myCounter_ms < myTimeout_ms) { return false; }
    myCounter_ms = 0U;
    return true;
}

private:
    const std::uint16_t myTimeout_ms;
    std::uint16_t myCounter_ms;
    bool myRunning;
};
} // namespace driver

```

- (c) State what RAII means, and give one embedded-specific example of hardware state that a destructor should undo. (2 marks)

Question 3 Classes, and what belongs to the type

12 MARKS

- (a) State the only formal difference between a struct and a class in C++. Then state the convention this book uses to decide which of the two to reach for. (2 marks)
- (b) Consider the following, which compiles and runs:

```

class Buzzer final
{
public:
    Buzzer(const std::uint16_t frequency_hz) noexcept
        : myFrequency_hz{frequency_hz}
    {}

    [[nodiscard]] std::uint16_t frequency_hz() const noexcept { return ↵
        ↵ myFrequency_hz; }

private:
    const std::uint16_t myFrequency_hz;
};

```

```

void playAlarm(const Buzzer buzzer) noexcept
{
    std::printf("Alarm at %u Hz\n", buzzer.frequency_hz());
}

int main()
{
    playAlarm(4000U);
    return 0;
}

```

State what the compiler does at the call to `playAlarm`, why this is a hazard rather than a convenience, and the one-word change that makes the call a compile error. (3 marks)

(c) A `Gpio` class is to gain two shared values: the maximum number of GPIO instances the MCU supports, and a count of how many `Gpio` objects currently exist.

Declare both in the course's style, state where each is defined and why the two differ on that point, and state what a static method cannot access and why. (4 marks)

(d) State what `final` does when applied to a class. Give one reason a driver class should normally have it, and one concrete reason you might deliberately leave it off. (3 marks)

Question 4 The class that owns something

13 MARKS

The class below is used to hold a receive buffer.

```

namespace util
{
class Buffer final
{
public:
    explicit Buffer(const std::size_t size) noexcept
        : myData{new std::uint8_t[size]{}},
          mySize{size}
    {}

    ~Buffer() noexcept { delete[] myData; }

    Buffer(const Buffer& other) noexcept
        : myData{other.myData},
          mySize{other.mySize}
    {}

    Buffer& operator=(Buffer&& other) noexcept

```

```

    {
        myData = other.myData;
        mySize = other.mySize;
        return *this;
    }

    [[nodiscard]] std::size_t size() const noexcept { return mySize; }

private:
    std::uint8_t* myData;
    std::size_t mySize;
};
} // namespace util

```

- (a) Name the four special member functions concerned with copying and moving, and write the signature of each for Buffer. (3 marks)
- (b) The copy constructor and the move assignment operator are both wrong. For each, state the defect and describe precisely what happens to the heap when the following runs. (6 marks)

```

util::Buffer a{64U};
util::Buffer b{a};

util::Buffer c{32U};
c = std::move(a);

```

- (c) Rewrite the move assignment operator correctly. (2 marks)
- (d) State what `std::move()` actually does. It is not what its name suggests. (2 marks)

Question 5 Interfaces, and the keyword that is missing

13 MARKS

```

namespace driver::sensor
{
    /**
     * @brief Temperature sensor interface.
     */
    class Interface
    {
    public:
        ~Interface() noexcept = default;

        virtual void start() noexcept
            = 0;
    };
}

```

```

    [[nodiscard]] virtual std::uint16_t read() const noexcept = 0;
};

/**
 * @brief Temperature sensor driver for the BME280.
 */
class Bme280 final : public Interface
{
public:
    explicit Bme280(const std::uint8_t address) noexcept
        : myAddress{address}
        , mySamples{new std::uint16_t[SampleCount]{} }
    {}

    ~Bme280() noexcept { delete[] mySamples; }

    void start() noexcept override {}
    [[nodiscard]] std::uint16_t read() const noexcept override { return mySamples->[0U]; }

private:
    /** Number of samples held by the driver. */
    static constexpr std::size_t SampleCount{16U};

    const std::uint8_t myAddress;
    std::uint16_t* mySamples;
};
} // namespace driver::sensor

void run(driver::sensor::Interface* sensor) noexcept
{
    sensor->start();
    delete sensor;
}

```

- (a) The code above compiles without error. State exactly what happens when `run()` is called with a pointer to a heap-allocated `Bme280`, and why. (4 marks)
- (b) Give the single-word change that fixes it. Then state what the author could have written on **one other line** so that the compiler would have rejected the broken version outright. (3 marks)
- (c) A colleague implements the sensor for another chip and writes the `read` method as:

```
[[nodiscard]] std::uint16_t read() noexcept override { return 0U; }
```

State what the compiler reports, and state what it would have reported had `override` been left off. (3 marks)

(d) Give two advantages of programming against `driver::sensor::Interface` rather than against a concrete driver type, and state what a call through the interface costs at runtime compared with a direct call. (3 marks)

Question 6 The factory, and who owns what

13 MARKS

(a) State the purpose of the factory pattern in an embedded system. Explain what it means that the system logic depends only on interfaces, and name what has to change in a program to move it from real drivers to stubs. (4 marks)

(b) The program below uses the raw-pointer factory from Chapter 4. It contains **two** ownership defects. Identify each and state the consequence. (5 marks)

```
int main()
{
    driver::factory::Esp32s3 factory{};

    // Spare status LED on pin 4.
    factory.gpio(4U);

    app::logic::Logic logic{factory, 2U, 3U};
    app::logic::Logic backup{logic};

    logic.run();
    return 0;
}
```

This version of `Logic` stores its two drivers as `driver::gpio::Interface*`, obtains them from the factory in its constructor, and deletes them in its destructor. Its public section declares the constructor, the destructor and `run()`, and nothing else.

(c) The factory is changed to return `std::unique_ptr<driver::gpio::Interface>`. State what happens to `Logic`'s destructor, and explain why one of the two defects in (b) becomes a compile error rather than a runtime fault. (4 marks)

Question 7 Templates

12 MARKS

(a) A colleague puts a function template's declaration in `bit_util.hpp` and its definition in `bit_util.cpp`, exactly as they would for an ordinary function. The build fails at link time with undefined reference. Explain why, and give the two arrangements the book recommends instead. (4 marks)

(b) The bit-clearing helper below has been in the codebase for a year and is used on 8-, 16- and 32-bit registers without trouble. Its first use on a 64-bit register corrupts the register.

```
template<typename T>
constexpr void clear(T& reg, const std::uint8_t bit) noexcept
{
    static_assert(std::is_integral<T>::value,
        "Cannot perform bit operation with non-integral type!");
    reg &= ~(1U << bit);
}
```

Given `std::uint64_t reg{0xFFFFFFFFFFFFFFFFU}` and a call `clear(reg, 2U)`, give the exact value of `reg` afterwards and explain how it got there. Name the second, separate failure this expression has for any bit of 32 or more. Then give the corrected body. (5 marks)

(c) With the corrected template in place, the following appears in one translation unit:

```
std::uint8_t reg1{};
std::uint32_t reg2{};
std::uint8_t reg3{};

clear(reg1, 0U);
clear(reg2, 17U);
clear(reg3, 3U);
```

State how many versions of `clear()` the compiler generates and name them. State what this implies for a resource-constrained target, and name one situation where a template is the wrong tool. (3 marks)

Question 8 Two threads and a shared struct

13 MARKS

```
namespace
{
    struct Shared
    {
        std::uint16_t data{};
        bool newData{false};
    };
}
```

```

};

Shared shared{};
std::mutex mutex{};
bool stop{false};

// -----
void txThread(const std::uint16_t count) noexcept
{
    for (std::uint16_t i{}; i < count; ++i)
    {
        shared.data    = i;
        shared.newData = true;
        std::this_thread::sleep_for(std::chrono::milliseconds(10U));
    }
    stop = true;
}

// -----
void rxThread() noexcept
{
    while (!stop)
    {
        std::lock_guard<std::mutex> lock{mutex};
        if (shared.newData)
        {
            std::printf("RX: %u\n", shared.data);
            shared.newData = false;
        }
    }
}
} // namespace

// -----
int main()
{
    std::thread t1{txThread, 100U};
    std::thread t2{rxThread};
    t1.detach();
    return 0;
}

```

(a) The program contains a mutex, and it is still not thread-safe. Identify **three** synchronization defects, and for each state the consequence. (6 marks)

- (b) State what happens at the closing brace of `main()`, and why. Name the two legal ways to end a `std::thread`'s lifetime and state which one this program should use. (3 marks)
- (c) Rewrite `rxThread()` so that it waits on a `std::condition_variable` instead of spinning. Give the predicate, state why `std::unique_lock` must be used rather than `std::lock_guard`, and state what the writer of the stop flag must additionally do so that the receiver can never be left asleep forever. (4 marks)

Self-Assessment Paper B

Time: 3 hours. **Closed book.** No compiler and no reference material. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all six chapters, so **it is meant to be taken once you have finished the book**. The model answers are in [Appendix E](#).

Rubric

C++17 is assumed throughout. Standard headers may be assumed included unless a question asks you to name them.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `virtual` on a destructor costs everything.

Where a question says “in the book’s style”, it means the conventions used in the chapters:

- camelCase for functions and methods, type names with a leading capital.
- Private member variables prefixed `my`, private static members prefixed `our`.
- `static constexpr` members named with a leading capital, e.g. `MaxInstances`.
- Brace initialization, `noexcept` on driver methods, `[[nodiscard]]` on queries and repeated on overriding methods.
- Copy and move operations deleted on a class that represents a unique hardware resource.

Where a question says “state the consequence”, naming the defect earns half the marks and saying what it does to the running program earns the other half. “This is wrong” scores nothing.

Where a question asks you to write a class, documentation comments are not required and are not marked. Write the code.

Question	1	2	3	4	5	6	7	8
Marks	12	12	12	13	13	13	12	13

Question 1 The vocabulary**12 MARKS**

- (a) State what difference the extension `.h` or `.hpp` makes to the compiler, and what actually decides whether a file is compiled as C or as C++. State what each extension signals by convention, and say which of the two a vendor's register-definition header should carry, with the reason. (3 marks)
- (b) State what an anonymous namespace does and name the C construct it replaces. Then state the purpose of a nested namespace such as `driver::gpio`, and write it in both the pre-C++17 form and the compact modern form. (3 marks)
- (c) Give three reasons references are usually preferred over pointers when a function must modify its argument. Then name one thing a pointer can do that a reference cannot, and give an example from the book where that matters. (3 marks)
- (d) Complete the function template below so that it sets every bit in the pack, using a C++17 fold expression rather than a loop. State what the compiler generates for the call `set(reg, 1U, 2U, 3U)`. (3 marks)

```
template<typename T, typename... Bits>
constexpr void set(T& reg, const Bits... bits) noexcept
{
    static_assert(std::is_integral<T>::value,
        "Failed to set bit in register: T must be of integral type!");
    // Your code here.
}
```

Question 2 The same driver, twice**12 MARKS**

Below is a PWM driver as it would be written in C.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

typedef struct
{
    uint8_t pin;
    uint16_t period_ms;
    bool enabled;
} pwm_t;

void pwm_init(pwm_t* self, const uint8_t pin, const uint16_t period_ms)
{
```

```

    if (NULL == self) { return; }
    self->pin      = pin;
    self->period_ms = period_ms;
    self->enabled  = false;
}

void pwm_set_enabled(pwm_t* self, const bool enabled)
{
    if (NULL == self) { return; }
    self->enabled = enabled;
}

bool pwm_is_enabled(const pwm_t* self)
{
    return NULL != self ? self->enabled : false;
}

```

(a) Rewrite it as a single-header C++ driver in the book's style. It shall:

- live in namespace `driver::pwm`, as a class named `Pwm` that cannot be inherited from;
- replace `pwm_init` with a constructor that cannot be used for implicit conversions and whose period defaults to 200;
- keep the pin and the period unchangeable after construction;
- expose `pin()`, `period_ms()`, `isEnabled()` and `setEnabled()`, with the query methods marked so that ignoring their result warns;
- be impossible to copy or move.

(6 marks)

(b) Name **four** distinct failure modes that exist in the C version and cannot occur in your C++ version, and state for each what removes it.

(4 marks)

(c) Your header and the vendor's C register header sit in the same project. State which extension each should carry, and what must be added to the vendor's header before your driver can call into it.

(2 marks)

Question 3 Enumerations, files, and lifetime

12 MARKS

(a) Write the enumeration class `Direction` from Chapter 2 in namespace `driver::gpio`, with the enumerators `Input`, `InputPullup` and `Output`, a fixed underlying type, and whatever is needed to support the validity check below. Then write that check.

```
[[nodiscard]] constexpr bool isDirectionValid(Direction direction) noexcept;
```

Give three reasons to prefer an enumeration class over a plain enum, and state what fixing the underlying type buys on an embedded target. (5 marks)

(b) A class is split into `driver/gpio/gpio.hpp` and `driver/gpio/gpio.cpp`. For each of the following, state whether it belongs in the header, in the source file, or in both, and why:

`explicit`, a default argument, `[[nodiscard]]`, the trailing `const` on a query method, `noexcept`, `= default` and `= delete`. (4 marks)

(c) State when the constructor runs and when the destructor runs for each of:

1. a local object inside a function;
2. an object at namespace scope;
3. an object created with `new`.

For the second, name the hazard this creates in an embedded program. (3 marks)

Question 4 What the compiler writes for you

13 MARKS

(a) The `Gpio` class from Chapter 2 declares its pin as `const std::uint8_t myPin;`. A copy constructor that makes a complete copy can be written for this class; a copy assignment operator that does so cannot. Explain why, and state what the compiler does about the copy assignment operator when you do not declare one. (4 marks)

(b) A class `Buffer` owns a heap block through two members:

```
std::uint8_t* myData;
std::size_t mySize;
```

Write its move constructor. Then state the one thing a move assignment operator must do that a move constructor need not, and explain why the constructor is exempt. (4 marks)

(c) State what `= default` and `= delete` each mean. Give the block of operations a driver class representing a unique hardware resource should delete, and the design reason.

Then answer this: writing `~Gpio() noexcept = default;` is often described as costing nothing, because the compiler would have generated the same destructor anyway. State what it nevertheless changes about the rest of the class. (5 marks)

Question 5 Designing against an abstraction

13 MARKS

(a) Write the interface `driver::serial::Interface` in the book's style. It shall support initializing the port with a baud rate, writing a null-terminated string, and querying whether the port has been initialized.

Then state, for four of the conventions §3.2 applies to an interface, what each is and why it is there. (5 marks)

(b) State the difference between private and protected members. Name the three kinds of inheritance in C++, state what each does to the access level of inherited members, and state which one a concrete driver uses to implement an interface, and why. (4 marks)

(c) A colleague writes:

```
void sendGreeting(driver::serial::Interface serial) noexcept
{
    serial.write("Hello!\n");
}
```

State what happens when this is compiled, and why. Then state what *would* have happened had the base class not been abstract, and name the mechanism. Give the corrected signature, and state one reason not to make it const. (4 marks)

Question 6 Factories

13 MARKS

(a) Draw or describe the dependency graph of the four layers in the Chapter 4 example: system logic, factory interface, concrete factories, concrete drivers. State which dependency is the one that makes the system testable without hardware, and why. (3 marks)

(b) Write the stub factory `driver::factory::Stub` for the smart-pointer version of the factory interface

```
[[nodiscard]] virtual std::unique_ptr<gpio::Interface> gpio(std::uint8_t pin) →
    noexcept = 0;
```

including whatever is needed to keep the compiler quiet about the unused pin. (4 marks)

(c) Give four differences between a raw `gpio::Interface*` and a `std::unique_ptr<gpio::Interface>` as the return type of `gpio()`. State where `std::move` becomes necessary once the logic class stores one, and what the source pointer holds afterwards. (4 marks)

(d) Name one cost the factory pattern imposes on an embedded system, and describe the alternative shown in Chapter 4 that avoids it entirely. (2 marks)

Question 7 A container with its size in the type**12 MARKS**

(a) Write a class template `container::Array<T, Size>` in namespace `container`, in a single header, providing:

- a default constructor;
- `push(const T& element)`, returning `false` if the array is already full;
- `size()` and `capacity()`;
- `operator[]` in both a modifiable and a read-only form;
- a compile-time check rejecting a size of zero, with a message.

The class shall not allocate memory.

(6 marks)

(b) State why this class cannot have its method definitions placed in `array.cpp`, and what the build would report if you tried. Then state how many distinct types the following three declarations produce, with the reason, and what that means for the size of the binary.

(4 marks)

```
container::Array<std::uint8_t, 8U> a{};
container::Array<std::uint8_t, 16U> b{};
container::Array<std::uint16_t, 8U> c{};
```

(c) State one advantage `container::Array<T, Size>` has over the heap-based `container::Vector<T>` from §5.3 on a target where dynamic allocation is forbidden, and one thing you give up by putting the capacity in the type.

*(2 marks)***Question 8 Sharing state between threads****13 MARKS**

(a) State the three conditions that together constitute a data race. State what the standard says the program then does, and why that is a stronger statement than “the value may be wrong”.

(3 marks)

(b) The TX and RX threads in Chapter 6 share this structure under a mutex:

```
struct SharedMem
{
    std::uint16_t data{};
    bool newData{false};
};
```

A colleague deletes the mutex and writes instead:

```
struct SharedMem
{
```

```
std::atomic<std::uint16_t> data{};
std::atomic<bool> newData{false};
};
```

State whether the data race is gone. State whether the program is correct, and justify your answer with a specific interleaving. Then state the two things a mutex provides that a pair of atomics does not. *(4 marks)*

(c) Name the two launch policies of `std::async` and state what each does. State what happens when no policy is passed, and describe the failure this produces in code written as “start the work, do something else, collect the result later”. *(3 marks)*

(d) Describe priority inversion as a sequence of events involving three threads. State why `std::mutex` cannot prevent it, and name what does. *(3 marks)*

APPENDIX D

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing semicolons, missing includes and misplaced braces cost nothing.

Question 1 Crossing the line between C and C++ 12 MARKS

(a) 4 MARKS

The mechanism is name mangling. C++ supports function overloading, so several functions may share a name and differ only in their parameter lists. The compiler therefore encodes the parameter types into the symbol it emits, turning `halGpioInit()` into something like `_Z11halGpioInitv`. C has no overloading and emits the name unchanged.

The vendor's library was compiled by a C compiler and exports `halGpioInit()`. Your translation unit was compiled by a C++ compiler, saw an ordinary C++ declaration, and emitted a call to the mangled name. The two do not match, so the linker finds nothing. The error message is itself the clue: the linker prints the missing symbol demangled, `halGpioInit()` with parentheses, which only happens when the symbol was a C++ one. (2 marks)

The fix. Tell the C++ compiler to use C linkage for those declarations:

```
#ifdef __cplusplus
extern "C" {
#endif

void halGpioInit(void);
void halGpioWrite(uint8_t pin, bool state);

#ifdef __cplusplus
}
#endif
```

(1 mark)

Why the guard. `extern "C"` is C++-only syntax. A C compiler reading the unguarded header would fail on it. `__cplusplus` is defined only when the file is compiled as C++, so the C compiler sees the preprocessor strip both blocks and is left with plain declarations.

(1 mark)

Credit either of these two additions: that `extern "C"` changes only linkage and does not make C++-only constructs legal, so the contents must remain valid C; or that a header written this way is exactly the kind that keeps the `.h` extension rather than `.hpp`, because it is meant to be compiled as both.

(b) 3 MARKS

Three advantages, any three of:

- **Type safety.** `constexpr std::uint32_t cpuFrequency{16000000U}` has a type the compiler checks at every use. `#define CPU_FREQUENCY 16000000` is a textual substitution with no type at all, and the error it eventually produces names the expansion, not the macro.

- **Scoping.** A `constexpr` obeys namespaces, class scope and block scope, so two modules may each have their own `bufLen`. A macro is invisible to all of that: it is live from its `#define` to the end of the translation unit and collides with anything of the same name, including member variables.
- **Debuggability and diagnostics.** The value exists in the symbol table as a named entity of known type. A macro has been erased before the compiler proper ever sees it.

(1 mark for any two, 2 for all three)

What a runtime `const` cannot do. A `constexpr` value may be used anywhere a *constant expression* is required: an array bound, a non-type template argument, a `static_assert` condition, a case label. A `const` variable whose initializer is not itself a constant expression, for example `const std::uint32_t n{readTrimRegister()};`, cannot be used in any of those places. `constexpr` also states the requirement at the point of definition, so it is diagnosed there rather than at the first use that needed it.

(1 mark)

(c)

3 MARKS

What it guarantees. That no exception will propagate out of the function. (1 mark)

If one does anyway. The program calls `std::terminate()` immediately. There is no handler search and no guarantee of unwinding — the failure is turned into an abort at the point it occurred. (1 mark)

Two reasons to disable exceptions in an embedded build, any two of:

- Binary size: unwind tables and the exception-handling runtime cost flash the device may not have.
- Determinism: the cost of a throw depends on the depth of the stack to be unwound and is not statically bounded, which is unacceptable on a path with a deadline.
- A simpler runtime environment; the error-handling scheme is then return codes, status flags or error callbacks, which are visible in the signature.

(1 mark)

(d)

2 MARKS

The two are not in conflict because `noexcept` never meant “cannot throw”. It means “if an exception tries to leave here, terminate”.

Chapter 1’s rule is about a function whose failure the caller could reasonably handle: if a function might fail by throwing and that failure is recoverable, taking away the caller’s ability to catch it is wrong. Chapter 4 takes the opposite view of the same fact deliberately. `std::make_unique` can throw `std::bad_alloc`; on a device where the heap is exhausted, there is nothing useful to recover to. On such a target the surrounding code is typically compiled without exception support and has no handler anywhere. (1 mark)

The behavior selected is therefore: on allocation failure, `std::terminate()` at the point of allocation, rather than an exception propagating through code that has no handler for it. A predictable stop instead of undefined drift. (1 mark)

Question 2 Object lifetime, and a driver that nearly works

12 MARKS

(a)

4 MARKS

```
Init 9
Init 13
Init 7
Inner
Release 7
Release 13
Outer
Release 9
```

(2 marks; deduct 1 for one line out of place, 2 for a scrambled second half)

The two rules.

1. A destructor runs when the object goes out of scope, which for button and buzzer is the closing brace of the inner block, not the end of main. This is why Release 7 and Release 13 appear **before** Outer.
2. Objects in the same scope are destroyed in the **reverse** order of their construction. buzzer was built last, so it is released first.

(2 marks, 1 each)

Both rules together are the mechanism behind RAII, which is part (c).

(b)

6 MARKS

(1.5 marks per defect: 0.5 for naming it, 0.5 for the consequence, 0.5 for the corrected line.)

1. The constructor is not explicit. It takes a single argument, so it doubles as a conversion from `std::uint16_t` to `Timer`. Given `void arm(driver::Timer timer);`, the call `arm(500U)` compiles silently and builds a temporary timer out of a number. The compiler stops being able to tell a timer from an integer, which is precisely the mistake it exists to catch.

```
explicit Timer(const std::uint16_t timeout_ms) noexcept
```

2. The member initializer list is in the wrong order. Members are initialized in the order they are **declared**, never in the order written in the list. The declaration order is `myTimeout_ms`, `myCounter_ms`, `myRunning`; the list says `myCounter_ms`, `myTimeout_ms`, `myRunning`. Nothing goes wrong at runtime here because no initializer reads another member, but the compiler warns (`-Wreorder`, which `-Wall` enables, so under the book's `-Werror` the snippet would not even build) and the list is now actively misleading: the day somebody writes `myCounter_ms{myTimeout_ms}` it will read a member that has not been initialized yet, and the code will look correct.

```
: myTimeout_ms{timeout_ms}
, myCounter_ms{0U}
, myRunning{false}
```

3. timeout_ms() is a query but is not const. It reads a member and changes nothing, but without the trailing const it cannot be called on a const Timer&. Any function that takes the timer read-only is then unable to ask it its own timeout, which forces callers to pass non-const references and gives away the protection that const was there to provide. It should carry [[nodiscard]] too, as elapsed() already does.

```
[[nodiscard]] std::uint16_t timeout_ms() const noexcept { return myTimeout_ms; }
```

4. Copy and move operations are not deleted. A Timer stands for one hardware timer. The compiler supplies a copy constructor, so driver::Timer backup{timer}; compiles and produces a second object counting independently against the same peripheral, with its own idea of when the timeout is due.

```
Timer(const Timer&)           = delete; // No copy constructor.
Timer(Timer&&)                = delete; // No move constructor.
Timer& operator=(const Timer&) = delete; // No copy assignment.
Timer& operator=(Timer&&)      = delete; // No move assignment.
```

A candidate who also notes that no Timer() = delete; is needed here — declaring any constructor already suppresses the implicit default one — has understood the rule rather than copied the pattern. Credit it, but not at the expense of one of the four above.

(c)

2 MARKS

RAII, Resource Acquisition Is Initialization: a resource is acquired when the object is created and released automatically when the object is destroyed. Because destruction is driven by scope and the compiler emits it on every exit path, the release cannot be forgotten. (1 mark)

An embedded example, any one of: disabling the interrupt the driver enabled; returning a pin to its reset direction and pull configuration; stopping a timer or PWM channel so it stops toggling a pin; releasing a DMA channel or a bus lock; powering down the peripheral clock. (1 mark)

Question 3 Classes, and what belongs to the type

12 MARKS

(a)

2 MARKS

The formal difference is the default access level. Members of a class are private unless stated otherwise; members of a struct are public. Everything else — constructors, destructors, methods, inheritance, templates, encapsulation — is available to both. (*A candidate who also names the default access of a base class, private for class and public for struct, is right and should be credited.*) (1 mark)

The book's convention. struct is for simple data-oriented types: plain data, or a holder for static constexpr constants. class is for anything with real encapsulation, inheritance, polymorphism, or custom copy and move semantics. (1 mark)

(b)

3 MARKS

What the compiler does. playAlarm wants a Buzzer and is handed a std::uint16_t. The single-argument constructor is a viable implicit conversion, so the compiler calls it to build a Buzzer{4000U} directly in the by-value parameter (in C++17 no copy is made) and runs the function. The program prints Alarm at 4000 Hz and looks entirely correct. (1 mark)

Why it is a hazard. The call site says “play the alarm on this buzzer” and the program does something else: it manufactures a buzzer out of a number, uses it once and throws it away. Whatever buzzer the caller meant was never involved; any hardware setup in the constructor, and any teardown in the destructor, ran for a throwaway object. Worse, the conversion accepts anything convertible to std::uint16_t — a pin number, a duration in milliseconds, a loop counter — so an entire class of type errors passes review and compiles clean. (1 mark)

The change: explicit.

```
explicit Buzzer(const std::uint16_t frequency_hz) noexcept
```

After that, playAlarm(4000U) is a compile error and playAlarm(Buzzer{4000U}) is the only way to say it, which at least says what it means. (1 mark)

(c)

4 MARKS

```
public:
    /** Maximum number of GPIO instances supported by this MCU. */
    static constexpr std::uint8_t MaxInstances{40U};

private:
    /** Number of currently active GPIO instances. */
    static std::uint8_t ourInstanceCount;
```

(2 marks: 1 for the pair, 1 for the naming)

The naming mark is for the leading capital on the `static constexpr` and the `our` prefix on the private static.

Where each is defined. `MaxInstances` needs no definition anywhere. Since C++17 a `static constexpr` data member is implicitly `inline`, so the line inside the class is both its declaration and its definition and the header may be included in any number of translation units.

`ourInstanceCount` is a plain static and must be defined exactly once, in a source file:

```
std::uint8_t Gpio::ourInstanceCount{0U};
```

Without it the linker reports an undefined reference; put it in the header instead and every translation unit that includes it defines the same symbol, which is a duplicate-definition error. The two differ because one is a compile-time constant that never needs storage and the other is a mutable object that needs exactly one. (1 mark)

What a static method cannot access. A static method is called on the class, not on an object, so it has no `this` pointer. It can therefore only touch other static members — never `myPin`, `myState`, or any non-static method. Asking it to would be asking “which object’s `myState`?”, a question with no answer. (1 mark)

(d)

3 MARKS

final on a class forbids deriving from it. Any class `X : public Gpio` is then a compile error. (1 mark)

Why a driver normally has it. A concrete driver is the end of the chain: nothing is meant to specialize an ATmega328P LED further. Sealing it says so, removes the question of whether its destructor needs to be virtual, and lets the compiler devirtualize calls when it knows the static type is the final one. (1 mark)

Why you might leave it off. So that a test build can derive from it and override behavior — §3.1 names exactly this case. You cannot inherit from a `final` class even for a fake, so sealing a class is a decision about who is allowed to test it. (1 mark)

Question 4 The class that owns something

13 MARKS

(a)

3 MARKS

```
Buffer(const Buffer& other) noexcept;           // Copy constructor.
Buffer(Buffer&& other) noexcept;               // Move constructor.
Buffer& operator=(const Buffer& other) noexcept; // Copy assignment operator.
Buffer& operator=(Buffer&& other) noexcept;    // Move assignment operator.
```

(3 marks, or 2 if the names are right but a signature is not: the `&&` on the move operations and the returned reference on the assignments are the parts worth checking.)

(b)

6 MARKS

The copy constructor is a shallow copy.

(3 marks)

It copies the pointer, not the bytes. After `util::Buffer b{a};`, `a.myData` and `b.myData` hold the same address. There is now one 64-byte block and two objects that each believe they own it. Writes through `b` are visible through `a`, which is already wrong. The fatal part is destruction: both destructors run `delete[]` on that address. The first frees the block; the second frees a block that is no longer allocated. That is undefined behavior, and in practice a heap corruption abort or a silent corruption of whatever the allocator has since handed out.

The fix is to allocate a new block and copy `mySize` bytes into it — or, for a class representing a hardware buffer, to = delete the copy constructor and stop the question from arising.

The move assignment operator leaks, then dangles.

(3 marks)

The operator has three defects, and `c = std::move(a);` shows the first two:

1. **It leaks.** `c` already owned a 32-byte block. `myData` is overwritten without `delete[]` first, so those 32 bytes are unreachable for the rest of the program's life.
2. **It leaves the source owning what it gave away.** `other.myData` is not cleared, so `a` is still a live object whose destructor will `delete[]` the 64-byte block that `c` now points at. Whichever is destroyed first leaves the other holding a dangling pointer, and the second destructor double-frees.
3. **It has no self-assignment guard.** `a = std::move(a)` is harmless only because the operator is already broken; once `delete[] myData` is added in front, self-assignment frees the buffer and then assigns the freed pointer to itself.

Taken together, after the four lines in the question, one 64-byte block is referenced by `a`, `b` and `c`, the 32-byte block is lost, and three destructors will attempt to free the same address. (No extra marks, but a candidate who states this has read the whole snippet rather than each operation separately.)

Also creditable, not required: the constructor is `noexcept` and calls `new`, which can throw `std::bad_alloc`, so an allocation failure terminates. That is a defensible choice on an embedded target — it is the Chapter 4 position from Question 1(d) — but it is a choice, and it should be a deliberate one.

(c)

2 MARKS

```
Buffer& operator=(Buffer&& other) noexcept
{
    if (this != &other)
    {
        delete[] myData;

        myData = other.myData;
        mySize = other.mySize;

        other.myData = nullptr;
```

```

        other.mySize = 0U;
    }
    return *this;
}

```

(1 mark for releasing the current buffer and guarding self-assignment, 1 mark for leaving `other` in a valid, destructible state.)

Nulling `other.myData` is what makes the moved-from object safe: `delete[] nullptr` is defined and does nothing, so `other`'s destructor is now harmless. "Valid but unspecified" is the contract a moved-from object owes — it must still be destructible.

(d)

2 MARKS

`std::move` moves nothing. It is a cast: it returns an rvalue reference to its argument, `static_cast<T&&>`, and does not touch the object. Its entire effect is on overload resolution — it makes the compiler prefer an overload taking `T&&`, which is how a move constructor or move assignment operator gets chosen over the copying one. All of the actual work happens inside that operation. (1 mark)

Two consequences worth stating: an object is unchanged by `std::move` itself and only changes if the selected operation changes it; and `std::move` on a `const` object yields a `const` rvalue, which binds to `const T&` and therefore silently **copies**. (1 mark)

Worth knowing, and a good place to say it: *Buffer* has a user-declared copy constructor, a user-declared move assignment operator and a user-declared destructor, so no move constructor is implicitly declared. `util::Buffer d{std::move(a)}`; therefore selects the copy constructor — the broken, shallow one — and the word `move` at the call site is doing nothing at all. For the same reason the copy assignment operator is implicitly deleted, so `b = a;` does not compile. This is the argument for declaring all five or deleting all four, never some of them.

Question 5 Interfaces, and the keyword that is missing

13 MARKS

(a)

4 MARKS

`~Interface()` is **not** `virtual`. Deleting an object of derived type through a pointer to a base class with a non-virtual destructor is **undefined behavior**. (2 marks)

What a typical implementation actually does is worth spelling out, because it is the reason nobody notices: the call is resolved statically from the pointer's type, so only `~Interface()` runs. `~Bme280()` never runs, `delete[] mySamples` never happens, and $16 \times 2 = 32$ bytes leak on every sensor destroyed. The program does not crash, does not warn and passes its tests; on a device that constructs and destroys sensors over months of uptime, it runs out of heap and dies somewhere unrelated. (2 marks)

Also creditable: `operator delete` is handed the wrong type and, in a sized-deallocation build, the wrong size. Under AddressSanitizer this is exactly what the program reports — `new-delete-type-mismatch` — and, if it is allowed to continue past that report (`ASAN_OPTIONS=halt_on_error=0`), 32 byte(s) leaked in 1 allocation(s), one `std::uint16_t[16]` per sensor.

A note for the marker: GCC and Clang do warn here, `-Wdelete-non-virtual-dtor`, which `-Wall` enables. Two things blunt it. The warning fires at the line with the `delete`, not at the interface where the mistake was made, so it lands in code the interface’s author may never compile; and it disappears entirely once the `delete` is inside a `std::unique_ptr`’s deleter in some other header. A candidate who says “the compiler would have told you” is half right and should be told which half.

(b)

3 MARKS

The fix:

```
virtual ~Interface() noexcept = default;
```

(1 mark)

The other line. The derived destructor should have been written:

```
~Bme280() noexcept override { delete[] mySamples; }
```

With `override` on it, the broken version does not compile at all: `override` asserts that this function overrides a virtual one, and if the base destructor is not virtual there is nothing to override, so the compiler rejects it and points at the exact line. That is why §3.2 marks the derived destructor `override` — it is not decoration, it is a tripwire that converts a silent runtime leak into a build failure.

(2 marks)

(c)

3 MARKS

The base declares `read() const`. The colleague’s `read()` is not `const`, so it is a **different** signature and does not override anything.

With `override`: a hard compile error at that line, along the lines of “method declared ‘override’ but does not override a base class method”. The mistake is reported where it was made.

(1.5 marks)

Without `override`: the class silently fails to implement the pure virtual `read() const`, so it is still abstract. The compiler says nothing at the declaration; the error arrives later and somewhere else, at the first attempt to create one — “invalid new-expression of abstract class type”, or “cannot declare variable to be of abstract type” — followed by a note listing `read() const` as the pure virtual still unimplemented. The non-`const` `read()` also **hides** the base name for lookup on the derived type, so even where it compiles it does not do what the reader expects.

(1.5 marks)

Also creditable: the same trap exists for `noexcept`. An `override` may not have a looser exception specification than the function it overrides, so dropping `noexcept` from an `override` of a `noexcept virtual` is an error too — which is exactly why §3.2 puts `noexcept` in the interface, to force it on every implementation.

(d)

3 MARKS

Two advantages, any two of:

(2 marks)

- **Testability.** The logic can be exercised against a stub with no hardware present. This is the advantage that pays for the other ones, and the reason the factory in Chapter 4 exists.
- **Replaceability.** A second chip means a new class and no change to any existing code that uses the interface.
- **Reusability.** Code written against the abstraction is not tied to one MCU and can move between projects.

The runtime cost. Each object carries a hidden pointer to its class's vtable — one pointer of storage per object — and a call through the interface is an indirect call through that table rather than a direct one, so it normally cannot be inlined. That is the whole cost, and it is the same mechanism the C interface example that Chapter 3 points to builds by hand out of a struct of function pointers.

(1 mark)

Question 6 The factory, and who owns what

13 MARKS

(a)

4 MARKS

Purpose. A factory centralizes the creation of hardware-specific objects, so the decision about *which concrete driver type* exists in exactly one place instead of being scattered through the code that uses them. It plays the role a HAL or BSP plays in a C project. (1.5 marks)

What “depends only on interfaces” means. `app::logic::Logic` names `driver::gpio::Interface` and `driver::factory::Interface` and never names `Esp32s3` or `Stub`. It does not include their headers, does not know how a pin is configured on any particular chip, and does not change or need recompiling when a platform is added. It knows it will be given something that can read, write and toggle; it does not know what. (1.5 marks)

What has to change to move to stubs. One line in main — `driver::factory::Esp32s3 factory{};` becomes `driver::factory::Stub factory{};` — and the corresponding `#include`. Nothing under `app::logic` changes at all. (1 mark)

(b)

5 MARKS

Defect 1: the discarded owning pointer.

(2.5 marks)

```
factory.gpio(4U);
```

`gpio()` allocates with `new` and hands ownership of the result to the caller. Here the returned pointer is thrown away, so nothing in the program holds the object's address and nothing can ever delete it: a leak, and for a real driver, a pin that has been configured with no way to release it. This is exactly the case the factory interface marks `gpio()` `[[nodiscard]]` for.

A note for the marker, not required of the candidate: do not treat the attribute as a guarantee that the build would have caught this. Even where a diagnostic is produced it is a warning and not an error, and whether one appears at all depends on the compiler and on where the call is resolved. The attribute is not inherited by an override — which is why the book repeats it on every overriding method — and GCC 13 diagnoses only a call it can resolve statically. This very line is flagged, because `Esp32s3` is `final` and its override repeats the attribute (and under the book's `-Werror` it stops the build), but the same call made through a `driver::factory::Interface&` produces nothing at all. `[[nodiscard]]` documents an intention; holding the pointer is what enforces it.

Defect 2: Logic is copyable and owns raw pointers.

(2.5 marks)

```
app::logic::Logic backup{logic};
```

Logic declares a destructor that deletes both drivers, but declares neither a copy constructor nor a deleted one, so the compiler generates one that copies the two pointer **values**. `backup` and `logic` now hold the same two driver objects — both drive the same physical pins — and when the two are destroyed, at the end of `main` if `run()` ever returns, both destructors run and each deletes both objects. Two deletes per object: undefined behavior, and the failure lands during shutdown, which is the worst possible place to find it. Under AddressSanitizer, with a `run()` that returns, the program reports a heap-use-after-free inside `Logic::~Logic()`.

Nothing warns about this by default. The implicit copy constructor of a class with a user-declared destructor is *deprecated*, but the diagnostic for it is `-Wdeprecated-copy-dtor`, which neither `-Wall` nor `-Wextra` turns on.

The rule broken is the rule of three: a class that needs a destructor needs its copy operations written or deleted. For a class holding a unique hardware resource, deleted — which is what §4.1 does, and why the paper is careful to say that *this* version declares nothing but the constructor, the destructor and `run()`. A candidate who answers “it does not compile, because `Logic` deletes its copy constructor” has read the chapter rather than the question; award the marks if they then state what the deletion is there to prevent, since that is the same knowledge:

```
Logic(const Logic&)           = delete;
Logic(Logic&&)                = delete;
Logic& operator=(const Logic&) = delete;
Logic& operator=(Logic&&)      = delete;
```

(c)

4 MARKS

Logic's destructor disappears entirely. With

```
std::unique_ptr<driver::gpio::Interface> myLed;
std::unique_ptr<driver::gpio::Interface> myButton;
```

each member destroys the object it owns when `Logic` is destroyed, so there is no `delete` left to write and therefore no destructor left to write. The code that could be forgotten no longer exists. (2 marks)

Defect 2 becomes a compile error. `std::unique_ptr` represents exclusive ownership and its copy constructor is deleted. A class whose member cannot be copied cannot have an implicit copy constructor, so `Logic`'s is defined as deleted, and `app::logic::Logic backup{logic};` no longer compiles. The double delete has been moved from run time to build time, and nobody had to remember to write the four `= delete` lines: the member's own semantics propagated to the class that holds it. (2 marks)

Note for completeness: defect 1 does not become an error. Discarding the returned `std::unique_ptr` still compiles. What changes is the consequence — instead of leaking, the temporary is destroyed at the end of the full expression, so the object is created and immediately destroyed. No leak, but the driver you asked for does not exist.

Question 7 Templates

12 MARKS

(a)

4 MARKS

Why. A template is not code; it is a pattern from which code is generated. The compiler produces an actual function only when it sees a use with concrete template arguments, and it can only do that where the **definition** is visible. `bit_util.cpp` is compiled on its own, contains no use of the template, and therefore generates nothing at all. The translation unit that calls `clear<std::uint8_t>` sees only the declaration, assumes the function exists somewhere and emits a call to a symbol nobody ever defined. The failure is at link time, not compile time, which is why the error is `undefined reference` rather than anything that names templates. (2 marks)

The two arrangements:

(2 marks, 1 each)

1. Declaration and definition together in the header, e.g. all of `container/vector.hpp`.
2. Declaration in `container/vector.hpp`, definition in `container/impl/vector_impl.hpp`, with `#include "container/impl/vector_impl.hpp"` at the **bottom** of the first. The compiler sees the same thing; the reader gets a header that is readable.

Explicit instantiation in the .cpp for each type used is a third technically correct answer and should be credited, with the observation that it requires naming every type in advance and so gives up the genericity the template was for.

(b)

5 MARKS

The value:

```
reg = 0x00000000FFFFFFFFB
```

(2 marks)

How it got there. 1U is unsigned int, 32 bits wide. 1U << 2 is 4U, still unsigned int. ~4U is 0xFFFFFFFFB, still 32 bits. Only at the &= is that value converted to std::uint64_t, and because it is **unsigned** it is zero-extended: 0x00000000FFFFFFFFB. The mask therefore carries 32 leading zeros that were never asked for.

```
0xFFFFFFFFFFFFFFFF & 0x00000000FFFFFFFFB = 0x00000000FFFFFFFFB
```

Bit 2 is cleared as requested — and so is every bit from 32 to 63. The function did what it was told and half the register with it. It went unnoticed for a year because for T of 32 bits or fewer, the mask is at least as wide as the register and no bits are lost. (1 mark)

The second failure. For bit >= 32, 1U << bit shifts an unsigned int by at least its own width, which is **undefined behavior** — not merely a wrong answer. In practice many targets take the shift count modulo 32, so clear(reg, 32U) clears bit 0 of a 64-bit register, on top of the upper half it already loses. (1 mark)

The corrected body:

```
reg &= static_cast<T>(~(static_cast<T>(1U) << bit));
```

The cast forces the shift and the complement to happen in T's own width, which is what the original meant to say. (This is the same `static_cast<T>(1U)` that the `set()` function of §1.2.12 uses, and the reason it is there.) (1 mark)

Worth noting and creditable: for a narrow T such as std::uint8_t the shift still promotes to int, which is harmless here — ~(1 << 2) is -5, and converting that to std::uint8_t gives 0xFB — but a bit outside the register's own width silently clears nothing. Neither version defends against a caller asking for bit 9 of an 8-bit register.

(c)

3 MARKS

Two versions, clear<std::uint8_t> and clear<std::uint32_t>. reg1 and reg3 have the same type, so the third call reuses the first instantiation, and bit is an ordinary function parameter rather than a template parameter, so its differing values generate nothing extra. (1 mark)

The implication. Code size grows with the number of distinct **types** a template is used with, not with the number of call sites. Ten calls on std::uint8_t cost one function; one call each on five register widths costs five. On a resource-constrained target that is the number to watch. (1 mark)

Where a template is the wrong tool, either of:

(1 mark)

- When the implementation must be chosen at **run time** — a driver selected from a configuration byte or a detected board revision. A template resolves everything at compile time, so the choice has to exist in the binary as a virtual call behind an interface.
- When the body is large and the type list is long: N copies of a big function cost more flash than one indirect call ever costs cycles.

Question 8 Two threads and a shared struct

13 MARKS

(a)

6 MARKS

(2 marks per defect: 1 for identifying it, 1 for the consequence.)

1. TX never locks the mutex. `txThread` writes `shared.data` and `shared.newData` with no lock at all. A mutex excludes only the threads that lock it; RX locking on its own excludes nobody. Both fields are read by RX while TX writes them, which is a data race and therefore undefined behavior. The concrete failure is also easy to state: nothing orders the two writes, so RX can see `newData == true` while `data` still holds the previous value, and print a message that was never sent.

This is also the case where making each field `std::atomic` would **not** be a fix. The two fields have to change together, as one unit, and only a mutex can make a multi-variable update atomic.

2. stop is a plain bool shared between two threads. TX writes it, RX reads it, and nothing synchronizes the two: a data race, undefined behavior. Beyond the standard's verdict, the practical failure is specific. Nothing inside the RX loop can change `stop`, so the compiler is entitled to load it once and hoist the test out of the loop entirely, and RX then spins forever after TX has finished. The fix is `std::atomic<bool> stop{false};` with `stop.store(true)` and `stop.load()`. This is the case an atomic is exactly right for: one independent flag.

3. RX busy-waits. The loop locks and unlocks the mutex as fast as the CPU allows, with no sleep and no wait, to discover that nothing has changed. Between messages that is 10 ms of a core spent doing nothing — on a battery-powered target, the difference between idle current and full current, and on a single-core MCU it also delays every other thread. The structure that fixes it is a `std::condition_variable`, which is part (c).

A candidate who instead offers “a detached thread may still be touching shared, mutex and stop while those namespace-scope objects are being destroyed after main returns” has named a real fourth defect and should be given the marks for it.

(b)

3 MARKS

At the closing brace of `main`, `t2` is destroyed while still **joinable** — it was never joined and never detached. The destructor of a joinable `std::thread` calls `std::terminate()`. The program does not return 0; it aborts. (1.5 marks)

The two legal ways to end a `std::thread`'s lifetime are `join()`, which blocks the caller until the thread finishes, and `detach()`, which severs the association and lets the thread run on its own. Exactly one of them must happen before the `std::thread` object is destroyed. (1 mark)

Which this program should use: **`join()`, on both threads.** `detach()` on `t1` is the second half of the same bug — `main` returns immediately afterwards, so a detached thread is left running against objects whose lifetime is ending. (0.5 marks)

(c)

4 MARKS

```
void rxThread() noexcept
{
    while (!stop.load())
    {
        std::unique_lock<std::mutex> lock{mutex};
        condition.wait(lock, [] { return shared.newData || stop.load(); });

        // Terminate if the wait was ended by the stop flag rather than by data.
        if (!shared.newData) { break; }

        std::printf("RX: %u\n", shared.data);
        shared.newData = false;
    }
}
```

(2 marks for the structure and the predicate)

The predicate is `shared.newData || stop.load()` — “there is data, or we are shutting down”. The second term is what makes the wait exitable; without it the thread has no reason to ever wake.

Why `std::unique_lock` and not `std::lock_guard`. `wait()` must release the mutex while the thread sleeps and re-acquire it before returning, otherwise no other thread could ever take the lock to produce the data being waited for. `std::lock_guard` locks in its constructor and unlocks in its destructor and offers nothing in between. `std::unique_lock` exposes the unlock and relock operations that `wait()` needs. (1 mark)

What the writer of the stop flag must do. Set the flag while holding the mutex, and then call `condition.notify_all()`:

```
{
    std::lock_guard<std::mutex> lock{mutex};
```

```

    stop.store(true);
}
condition.notify_all();

```

Once TX has finished, nothing else will ever notify; a receiver already asleep inside `wait()` would stay there forever, holding a predicate that would now return true if only something woke it up to evaluate it. `notify_all()` wakes every waiter so each can re-check and exit. The lock matters even though `stop` is atomic: without it the store and the notification can both land after the receiver has evaluated the predicate as false but before it has gone to sleep, the notification is lost, and the receiver sleeps forever anyway. (1 mark)

Two things worth saying to a candidate who got the rest right. The predicate overload of `wait()` is also what handles spurious wakeups: it re-checks the condition on every wake and goes back to sleep if it is false, which is why this is written with a predicate rather than a bare `wait(lock)`. And TX must now take the mutex around its two writes and call `notify_one()` after releasing it — the fix to (a)(1) and this part are the same change seen from the two ends.

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing semicolons, missing includes and misplaced braces cost nothing. The questions are in Appendix C.

Question 1 The vocabulary

12 MARKS

(a)

3 MARKS

To the compiler, none. `#include` behaves identically whatever the extension, and the extension of a *header* has no bearing on the language it is compiled as. What decides that is the compiler invocation — `gcc` against `g++` — or the extension of the **source** file that pulled the header in. (1 mark)

By convention: `.h` is shared ground. A `.h` file may be valid C, valid C++, or both, and the extension alone does not say which. `.hpp` states that the header uses C++-only constructs — classes, templates, namespaces, references — and therefore needs a C++ compiler. (1 mark)

The vendor's register header carries .h. It is shipped for C users and included from C++ applications through an `extern "C"` block, so it must compile as either. `.hpp` would claim it is C++-only, which is exactly the thing it must not be. (1 mark)

(b)

3 MARKS

An anonymous namespace restricts everything declared inside it — variables, functions, types and compile-time constants — to a single translation unit. It replaces C's `static` on file-scope functions and variables, and it is used in `.cpp` files to hide internal helpers. (1 mark)

The purpose of `driver::gpio` is to avoid symbol collisions and to mirror the structure of the project, so that `driver::gpio::init()` and `driver::timer::init()` can both exist and the reader can see at the call site which module a function belongs to. (1 mark)

```
// Pre-C++17.
namespace driver
{
    namespace gpio
    {
        void init() {}
    } // namespace gpio
} // namespace driver

// Compact form, C++17 and later.
namespace driver::gpio
{
    void init() {}
} // namespace driver::gpio
```

(1 mark)

(c)

3 MARKS

Three advantages, any three of: a reference cannot be null; the call site needs no &; reading it needs no *; the syntax at the point of use is that of an ordinary variable, so the code reads the same whether the argument is a reference or a value. (2 marks)

What a pointer can do that a reference cannot: be *reseated*. A reference is bound to one object when it is initialized and can never be made to refer to another. A pointer can be reassigned, and can be null to mean “nothing”.

Where it matters in the book: `container::Vector<T>` holds its storage as `T* myData`, because `resize()` and `pushBack()` call `std::realloc` and the block’s address changes — a reference could not follow it. The raw-pointer factory in Chapter 4 is the other case: it must hand back the address of a heap object, and only a pointer can carry that. (1 mark)

(d)

3 MARKS

```
((reg |= (static_cast<T>(1U) << bits)), ...);
```

(2 marks)

Award 1 mark for the loop form for `(const auto& bit : {bits...})`, which is correct but is not what the question asked for.

A note for the marker. The operand of a comma fold must be a cast-expression, so the assignment needs its own pair of parentheses; written as `(reg |= (static_cast<T>(1U) << bits), ...)` — one pair short, the form §1.2.12 warns against — GCC rejects it with “binary expression in operand of fold-expression”. Award full marks either way. The paper marks semantics, and the missing parentheses are exactly the class of slip the rubric says costs nothing.

What the compiler generates. The pack is expanded and the operation performed once per element, folded together with the comma operator:

```
reg |= (static_cast<T>(1U) << 1U);
reg |= (static_cast<T>(1U) << 2U);
reg |= (static_cast<T>(1U) << 3U);
```

This is one instantiation of set, not three — but the pack's length and element types are part of the template arguments, so a call with a *different number* of bits instantiates a different specialization. That is the code-size caveat for parameter packs. (1 mark)

Question 2 The same driver, twice

12 MARKS

(a)

6 MARKS

```
/**
 * @file PWM driver implementation.
 */
#pragma once

#include <cstdint>

namespace driver::pwm
{
/**
 * @brief PWM driver.
 */
class Pwm final
{
public:
    explicit Pwm(const std::uint8_t pin, const std::uint16_t period_ms = 20U)
        noexcept
        : myPin{pin}
        , myPeriod_ms{period_ms}
        , myEnabled{false}
    {}

    ~Pwm() noexcept = default;

    [[nodiscard]] std::uint8_t pin() const noexcept { return myPin; }
    [[nodiscard]] std::uint16_t period_ms() const noexcept { return myPeriod_ms; }
    [[nodiscard]] bool isEnabled() const noexcept { return myEnabled; }
```

```

void setEnabled(const bool enabled) noexcept { myEnabled = enabled; }

Pwm() = delete; // No default constructor.
Pwm(const Pwm&) = delete; // No copy constructor.
Pwm(Pwm&&) = delete; // No move constructor.
Pwm& operator=(const Pwm&) = delete; // No copy assignment.
Pwm& operator=(Pwm&&) = delete; // No move assignment.

private:
    /** Pin the PWM output is connected to. */
    const std::uint8_t myPin;

    /** PWM period in milliseconds. */
    const std::uint16_t myPeriod_ms;

    /** Indicate whether the PWM output is enabled. */
    bool myEnabled;
};
} // namespace driver::pwm

```

Marks, 1 each, capped at 6: namespace `driver::pwm` and class `Pwm` final; explicit constructor with the default argument = 200; const on `myPin` and `myPeriod_ms`, both private with the `my` prefix; `[[nodiscard]]` on the three queries; trailing `const` on the three queries; `noexcept` throughout; the deleted copy and move operations.

Do not deduct for `#pragma` once or the includes; do deduct if a default argument appears anywhere other than the declaration.

(b)

4 MARKS

(1 mark each, any four.)

- The null self.** Every C function has to test it, the test costs a branch on every call, and it has to be remembered in each new function somebody adds. A member function is called *through* an object; `backlight.setEnabled(true)` cannot be written without a `backlight`. The check is not optimized away — it ceases to have a case to check.
- A forgotten `pwm_init`.** `pwm_t backlight;` is a perfectly legal C object full of whatever was on the stack, and `pwm_set_enabled(&backlight, true)` will happily operate on it. A constructor cannot be skipped: the object does not exist until it has run.
- An error returned as data.** `pwm_is_enabled(NULL)` returns `false`, which is indistinguishable from a genuine “not enabled”. The caller cannot tell a failure from an answer. The C++ query has no failure mode to disguise.

4. **Direct writes to the members.** `backlight.pin = 12;` compiles in C, and from then on the driver silently drives the wrong pin. Making the members private puts them out of reach, and `const` on `myPin` means even the class cannot change it after construction.
5. **Accidental duplication.** `pwm_t copy = backlight;` gives two objects for one peripheral. The deleted copy operations turn that into a compile error.

(c)

2 MARKS

Your header is .hpp. It contains a class, `noexcept`, default arguments and deleted operations — none of which a C compiler can read — so the extension should say so. (1 mark)

The vendor's header stays .h, because it must remain compilable as C. Before your C++ driver can call into that library, its declarations need a guarded `extern "C"` block:

```
#ifdef __cplusplus
extern "C" {
#endif
/* declarations */
#ifdef __cplusplus
}
#endif
```

Without it the C++ compiler mangles the names and the linker cannot match them against the C library's unmangled symbols. (1 mark)

Question 3 Enumerations, files, and lifetime

12 MARKS

(a)

5 MARKS

```
namespace driver::gpio
{
/**
 * @brief GPIO direction configuration.
 */
enum class Direction : std::uint8_t
{
    Input,          ///< Input without pull-up.
    InputPullup,    ///< Input with pull-up enabled.
    Output,          ///< Output.
    Count,           ///< Number of supported directions.
};

[[nodiscard]] constexpr bool isDirectionValid(const Direction direction) noexcept
{
```

```

    return static_cast<std::uint8_t>(Direction::Count) > static_cast<std::uint8_t>
        >(direction);
}
} // namespace driver::gpio

```

(2 marks: 1 for the enumeration with a fixed underlying type, 1 for the check. *Count* is the “whatever is needed” the question asked for — the check has nothing to compare against without it, and it must be last.)

Three reasons for an enumeration class, any three of:

(2 marks)

- **The enumerators are scoped.** They are named `Direction::Input`, not `Input`, so they cannot collide with anything and need no `GPIO_DIRECTION_` prefix. The prefix convention that C forces on you is built into the language here.
- **No implicit conversion to an integer.** A plain `enum` decays to `int` and will silently take part in arithmetic, in comparisons against an unrelated enumeration, and in overload resolution. An `enum class` requires an explicit `static_cast`, which is why the validity check above has two of them, and why it is obvious when a conversion is happening.
- **The enumerators do not leak into the enclosing scope**, so the namespace stays clean.
- **The underlying type is always fixed** — `int` unless you name another — so the size is known and the enumeration can be forward-declared. A plain `enum` gets that only if you write the type out, which C++11 allows and almost nobody does.

What fixing the underlying type buys. The size is known, minimal and the same on every compiler — one byte instead of whatever `int` happens to be — which matters when the value is a member of a struct that goes into a frame or is stored in RAM in quantity. Choosing an *unsigned* type also means no negative value can exist, which reduces validation to the single upper-bound comparison above.

(1 mark)

(b)

4 MARKS

(0.5 marks for each of the seven, counting = `default` and = `delete` separately, plus 0.5 for a coherent statement of the principle.)

Construct	Where	Why
<code>explicit</code>	Header only	Belongs to the declaration; repeating it on the out-of-class definition is an error.
Default argument	Header only	May be specified once per scope; repeating it in the definition is an error.
<code>[[nodiscard]]</code>	Header only	It is the declaration callers see. Repeating it is legal but redundant; the book's rule is declaration only.
Trailing <code>const</code>	Both	It is part of the function's type. Omit it in the source file and you are defining a <i>different</i> function, which does not match anything declared.
<code>noexcept</code>	Both	Every declaration must carry the same exception specification; omit it in the source file and the definition is rejected for having a different one. Since C++17 it is also part of the function type.
= <code>default</code> , = <code>delete</code>	Header only	Written in the class, they are complete definitions with no body to place anywhere else; = <code>delete</code> must be on the first declaration.

The principle: anything that is part of the function's **type** must appear in both places, or the definition does not define the function that was declared. Anything that describes how the function may be *called* — `explicit`, default arguments, `[[nodiscard]]` — belongs to the declaration alone.

Also creditable, from the same section: `const` on a parameter passed **by value** is dropped from the header declaration and kept in the definition, because it constrains only the function body and is not part of the type.

(c)

3 MARKS

(0.5 marks each for the three pairs, 1.5 for the hazard.)

- 1. A local object.** Constructed where control reaches its declaration; destroyed at the closing brace of its scope, in reverse order of construction.
- 2. An object at namespace scope.** Constructed during program start-up, **before main is entered**; destroyed **after main returns**, in reverse order of construction.
- 3. An object created with new.** Constructed as part of the new expression; destroyed only when `delete` is called on the pointer. If `delete` never happens, the destructor never runs — that, and not merely the unreleased memory, is what a leak costs you.

The hazard with namespace-scope objects. Their constructors run before `main`, and therefore before any clock, power or peripheral initialization the program performs in `main`. A driver object at namespace scope may configure a peripheral that is not yet clocked. On top of that, the order of construction between objects in *different* translation units is unspecified, so one such object cannot safely depend on another. And because they are destroyed after

main returns, anything still running at that point — a detached thread, for example — is touching objects whose lifetime is ending.

Question 4 What the compiler writes for you

13 MARKS

(a)

4 MARKS

Why the copy constructor works. A const non-static data member can be **initialized**, once, in the member initializer list, at the moment the object comes into existence. A copy constructor is building a brand-new object, so `myPin{other.myPin}` is a legal initialization and nothing is being modified. (1.5 marks)

Why the copy assignment operator cannot be written. Assignment operates on an object that already exists and whose `myPin` was fixed when it was constructed. The operator would have to write `myPin = other.myPin;`, which is ill-formed — a const object is never assignable. There is no ordering of statements that gets round this: the member simply cannot be made to hold a different value for the rest of its life. The simplified operator in §2.2 compiles precisely because it copies only `myState`: the target keeps its own pin, so the result is not a copy. (1.5 marks)

What the compiler does. It still implicitly *declares* a copy assignment operator, but because the class has a const non-static data member, that operator is **defined as deleted**. `led2 = led1;` is therefore a compile error about a *deleted function* rather than an assignment to a const: GCC reports use of deleted function '`Gpio& Gpio::operator=(const Gpio&)`', and only its follow-up note names the const member, which is why the message can be puzzling if you have not met the rule. The same applies to the move assignment operator. The class in §2.2 deletes all four explicitly anyway, which states the intent and produces a clearer diagnostic. (1 mark)

(b)

4 MARKS

```
Buffer(Buffer&& other) noexcept
    : myData{other.myData}
    , mySize{other.mySize}
{
    other.myData = nullptr;
    other.mySize = 0U;
}
```

(2 marks: 1 for taking the members, 1 for clearing other.)

Clearing `other.myData` is not tidiness. `other` is still a live object and its destructor will run and `delete[]` whatever it holds; leaving the old address there means the block this object now owns is freed behind its back. `delete[] nullptr` is defined and does nothing, so

nulling the pointer makes the moved-from object harmless while keeping it destructible — which is the whole contract a moved-from object owes.

What move assignment must do that a move constructor need not: release the resource the target **already owns** before taking the new one, and guard against self-assignment.

(1 mark)

Why the constructor is exempt. It is building an object that did not exist a moment ago. It owns nothing yet, so there is nothing to free; and the object under construction cannot possibly be the same object as other, so this `!= &other` is not merely unnecessary, it can never be false.

(1 mark)

(c)

5 MARKS

= default asks the compiler for its own implementation of a special member function, stated explicitly. The reader can see that the function exists and was intended, and you can attach `noexcept`, `virtual` or an access level to it while you are there.

= delete removes the function entirely. Any attempt to use it is a compile error that names the deleted function, rather than a program that compiles and does something you did not want.

(2 marks)

```
Gpio()                = delete; // No default constructor.
Gpio(const Gpio&)     = delete; // No copy constructor.
Gpio(Gpio&&)          = delete; // No move constructor.
Gpio& operator=(const Gpio&) = delete; // No copy assignment.
Gpio& operator=(Gpio&&)    = delete; // No move assignment.
```

The design reason: a `Gpio` object stands for one physical pin. A copy would be a second object that believes it owns the same pin and acts on it independently; nothing in the hardware was duplicated, only the software's idea of it. Deleting the operations makes the mistake a build failure. The deleted default constructor is there for a related reason — a GPIO with no pin number is not a thing.

(1.5 marks)

What `~Gpio()` `noexcept = default`; nevertheless changes. Declaring a destructor at all — even defaulted, even in the class body — makes it a **user-declared** destructor, and a class with a user-declared destructor gets **no implicitly declared move constructor and no implicitly declared move assignment operator**. The class silently loses its move operations and falls back to copying wherever a move would have been used. The copy operations are still generated, so nothing fails to compile and nothing warns.

That is precisely why the book writes all of them out: once you touch one special member function, the compiler stops supplying the others on the terms you assumed, and the only readable answer is to state every one of them.

(1.5 marks)

Question 5 Designing against an abstraction

13 MARKS

(a)

5 MARKS

```

/**
 * @file Serial driver interface.
 */
#pragma once

#include <stdint>

namespace driver::serial
{
    /**
     * @brief Serial driver interface.
     */
    class Interface
    {
    public:
        virtual ~Interface() noexcept = default;

        virtual void init(std::uint32_t baudrate) noexcept = 0;
        virtual void write(const char* message) noexcept = 0;
        [[nodiscard]] virtual bool isInitialized() const noexcept = 0;
    };
} // namespace driver::serial

```

(2 marks for the interface; the trailing *const* on the query and *= 0* on all three are the parts worth checking.)

Four conventions, 0.75 marks each (§3.2 lists five; the fifth is below):

1. **The destructor is *virtual*.** Concrete drivers are deleted through an *Interface**. Without *virtual* that is undefined behavior, and in practice the derived destructor never runs, so whatever the driver acquired is never released.
2. **The destructor is *= default*.** The interface owns nothing and has nothing to clean up, so the compiler's implementation is exactly right. Writing it explicitly shows it was decided rather than forgotten — and it is the only way to state *virtual* on it.
3. **Every method is pure virtual (*virtual ... = 0*).** This is what makes the class abstract, so nobody can create an *Interface*, and what obliges every concrete driver to provide the method. The interface defines *what*, never *how*.
4. ***noexcept* on every method.** It is part of the function type, and an override may not weaken it, so declaring it here forces every implementation on every platform to be

noexcept. On an embedded target that is exactly the constraint you want to impose from the abstraction downwards.

*A fifth, worth crediting in place of any of the above: `[[nodiscard]]` on the query, so that a caller who asks whether the port is initialized and then ignores the answer is warned, where the compiler supports it (§3.2 notes that GCC does not warn for a call made through the interface itself). Note that the attribute is **not** inherited — it belongs to the declaration it is written on — so every overriding implementation must repeat it, which is what the book does.*

(b)

4 MARKS

private against protected. Neither is accessible from outside the class. The difference is that a **derived** class can reach protected members and cannot reach private ones. That is the entire difference, and it is why a base class intended to be inherited from puts its shared state and helpers under protected. (1 mark)

The three kinds of inheritance:

(1.5 marks)

- **Public.** Public members of the base stay public in the derived class and protected stay protected. Used when the derived class *is a* specialization of the base. By far the most common.
- **Protected.** Public and protected members of the base become protected in the derived class. Like private inheritance, except that classes derived further down still get access.
- **Private.** Everything from the base becomes private in the derived class. Used when the base is an implementation detail of the derived class rather than something it presents to the world.

A concrete driver uses public inheritance, because the point of the exercise is that a `driver::serial::Esp32s3` *is a* `driver::serial::Interface` and can be handed to anything expecting one. With private or protected inheritance, the conversion from derived to base is not accessible to outside code, so passing the driver as an `Interface&` would not compile — the abstraction would exist and be unusable. (1.5 marks)

(c)

4 MARKS

It does not compile. Interface has pure virtual members and is therefore an **abstract class**. A parameter of abstract class type cannot be declared, because passing by value would require constructing one, and an abstract class cannot be instantiated. The compiler says so directly: “cannot declare parameter ‘serial’ to be of abstract type ‘driver::serial::Interface’”, followed by a note listing the pure virtual functions responsible. (1.5 marks)

Had the base not been abstract — every method with a default implementation — it would have compiled and **sliced**. Only the base subobject of the argument is copied into the parameter; the derived part is discarded and the parameter’s dynamic type is the base. `serial.write(...)` would then call the base version, not the driver’s override. Virtual dispatch works through references and pointers; a by-value copy has already thrown away

the thing that makes it work. The mechanism is **object slicing**, and it is the more dangerous of the two outcomes because it compiles. (1.5 marks)

The corrected signature:

```
void sendGreeting(driver::serial::Interface& serial) noexcept
```

Why not const. write() changes the state of the port and is therefore not a const method, so a const Interface& could not call it. A const reference to an interface restricts you to its query methods — here, isInitialized() alone. (1 mark)

Question 6 Factories

13 MARKS

(a)

3 MARKS

An arrow reads “depends on”, that is, includes the header of:

```
app::logic::Logic      --> driver::factory::Interface, driver::gpio::Interface
driver::factory::Interface --> driver::gpio::Interface (returned by gpio())
driver::factory::Esp32s3 --> driver::factory::Interface (implements)
                        --> driver::gpio::Esp32s3      (creates)
driver::factory::Stub   --> driver::factory::Interface (implements)
                        --> driver::gpio::Stub         (creates)
driver::gpio::Esp32s3    --> driver::gpio::Interface (implements)
driver::gpio::Stub       --> driver::gpio::Interface (implements)
```

(1.5 marks for the four layers and the direction of the arrows. The architecture overview in §4.1 draws the same four layers as a stack, top to bottom; a candidate who reproduces it has the layers but has drawn who sits above whom, not what depends on what.)

The dependency that makes it testable is the one from Logic to the **interfaces** rather than to any concrete type. Because Logic names only driver::factory::Interface and driver::gpio::Interface, the entire lower half of the graph can be replaced without Logic changing, or even recompiling for a different reason. Logic points only at abstractions, and nothing points from an abstraction down to an implementation. That is the property, and swapping in stubs is one consequence of it. (1.5 marks)

(b)

4 MARKS

```
#pragma once

#include <stdint>
#include <memory>
```

```

#include "driver/factory/interface.hpp"
#include "driver/gpio/stub.hpp"

namespace driver::factory
{
class Stub final : public Interface
{
public:
    Stub() noexcept = default;
    ~Stub() noexcept override = default;

    [[nodiscard]] std::unique_ptr<gpio::Interface> gpio(const std::uint8_t pin)
    {
        // Ignore the pin number since it is not used by the stub.
        (void)(pin);
        return std::make_unique<gpio::Stub>();
    }

    Stub(const Stub&) = delete;
    Stub(Stub&&) = delete;
    Stub& operator=(const Stub&) = delete;
    Stub& operator=(Stub&&) = delete;
};
} // namespace driver::factory

```

(1 mark each, capped at 4: `final` and `: public Interface`; `override` on both the method and the destructor; the return type matching the interface exactly and `std::make_unique`; `(void)(pin)`; `[[nodiscard]]` repeated on the override; the deleted copy and move operations.)

On `[[nodiscard]]`: the attribute belongs to the declaration it is written on and is **not** inherited by an override, so the book repeats it on every overriding method. A candidate who omits it has lost one of the six available marks, not four — but a candidate who omits it and explains that it is inherited from the interface has got the rule backwards and should be told so.

Worth stating: `std::make_unique<gpio::Stub>()` produces a `std::unique_ptr<gpio::Stub>`, which converts implicitly to `std::unique_ptr<gpio::Interface>` because `Stub` derives from `Interface`. That conversion is what makes the return statement work, and it is safe only because `gpio::Interface` has a virtual destructor.

(c)

4 MARKS

(0.75 marks each for the four, 1 mark for the `std::move` answer.)

- Ownership is stated in the type.** `std::unique_ptr` says “you own this, exclusively”. A raw pointer says nothing at all; the answer lives in a comment, or nowhere.

2. **Release is automatic.** The `unique_ptr`'s destructor deletes the object when it goes out of scope. The raw pointer needs a matching `delete` on every path out, and the class holding it needs a destructor written for the purpose.
3. **It cannot be copied.** `std::unique_ptr`'s copy constructor is deleted, so two owners of one object cannot be created by accident — the double delete that a copied raw pointer produces at run time becomes a compile error instead.
4. **Transfer is explicit.** Handing ownership on requires `std::move`, so the moment ownership changes hands is visible in the source. A raw pointer changes owner by plain assignment, and nothing marks it.

Where `std::move` becomes necessary. Not in the Chapter 4 constructor: `factory.gpio(pin)` returns a temporary, and initializing a member from a temporary already moves. It is needed as soon as the source is a **named** `unique_ptr` — a logic class taking one as a parameter and storing it, `myLed{std::move(led)}`, or reassigning `myLed = std::move(other)`.

Afterwards the source holds `nullptr`. It is still a valid object; it simply owns nothing. Dereferencing it is undefined behavior, which is why a moved-from pointer should not be used again.

(d)

2 MARKS

The cost: dynamic allocation. The factory creates its objects with `new` or `std::make_unique`, which means a heap on a target that may not want one: fragmentation over a long uptime, an allocation time that is not deterministic, and a failure mode that terminates the program (Paper A, Question 1(d)). It also adds a level of indirection and one more abstraction to read through. (1 mark)

The alternative shown in Chapter 4: construct the concrete drivers directly, as objects with automatic or static storage, and pass references into the logic class.

```
driver::gpio::Esp32s3 led{ledPin};
driver::gpio::Esp32s3 button{buttonPin};
app::logic::Logic logic{led, button};
```

with `Logic` holding `driver::gpio::Interface&` members. No heap is involved at all, everything is sized at compile time, and the logic is still written entirely against the interface. §4.1 says as much: this is “very efficient when dynamic memory allocation should be limited”. What it gives up is choosing the driver type at run time, and the single point of construction that a factory provides once there are many drivers to make. (1 mark)

Question 7 A container with its size in the type

12 MARKS

(a)

6 MARKS

```

/**
 * @file Fixed-capacity array implementation.
 */
#pragma once

#include <cstddef>

namespace container
{
    template<typename T, std::size_t Size>
    class Array final
    {
        static_assert(Size > 0U, "Array size must be greater than zero!");

    public:
        Array() noexcept
            : myData{}
            , mySize{}
        {}

        [[nodiscard]] bool push(const T& element) noexcept
        {
            // Reject the element if the array is already full.
            if (Size <= mySize) { return false; }
            myData[mySize++] = element;
            return true;
        }

        [[nodiscard]] std::size_t size() const noexcept { return mySize; }
        [[nodiscard]] constexpr std::size_t capacity() const noexcept { return Size; }

        T& operator[](const std::size_t index) noexcept { return myData[index]; }
        const T& operator[](const std::size_t index) const noexcept { return myData[↵
            index]; }

    private:
        /** Element storage. */
        T myData[Size];

        /** Number of elements currently held. */

```

```

    std::size_t mySize;
};
} // namespace container

```

Marks, 1 each: *template<typename T, std::size_t Size> with Size as a **non-type** parameter; the `static_assert` with a message; storage as a member array rather than a pointer, with no allocation anywhere; `push` returning `false` when full and marked `[[nodiscard]]`; both `operator[]` overloads, the second `const` and returning `const T&`; `size()` and `capacity()`, `const` and `[[nodiscard]]`.*

Creditable extras, not required: `constexpr` on `capacity()`, since the answer is a template argument and is known at compile time; the copy and move operations, which for a container are a design decision rather than an automatic deletion.

(b)

4 MARKS

Why the definitions cannot live in `array.cpp`. The compiler generates code for `Array<std::uint8_t, 8U>` only where it sees a use with those arguments **and** the definition of what to generate. `array.cpp` is compiled alone, contains no use, and generates nothing. The translation unit that does use it sees only a declaration and emits calls to functions nobody defined. The build therefore fails at **link** time with undefined reference, not at compile time — which is why the error mentions nothing about templates. (2 marks)

Three distinct types. A non-type template argument is part of the type exactly as a type argument is, so changing `8U` to `16U` produces a different type just as surely as changing `std::uint8_t` to `std::uint16_t` does. The three are unrelated: no conversion between them, no common base, and a function taking `Array<std::uint8_t, 8U>&` will not accept `b`. (1 mark)

For the binary: every member function that is actually used is generated once per instantiation, so `push`, once it is called on all three, exists three times over. Code size scales with the number of instantiations, and a non-type parameter makes those very easy to multiply — one set per buffer size anybody in the project happens to pick. (1 mark)

(c)

2 MARKS

The advantage: no heap. The storage is a member array whose size is known at compile time, so the object lives on the stack or in `.bss`. There is no allocation to fail, no fragmentation, no `std::realloc`, and no non-deterministic call on a path with a deadline. `push` cannot fail for want of memory — only for want of capacity, and it says so by returning `false`. (1 mark)

What you give up. The capacity is part of the type, so `Array<T, 8U>` and `Array<T, 16U>` are unrelated and no ordinary function can accept both — anything wanting to must become a template itself, which multiplies the instantiations again. And the full capacity is paid for in RAM whether it is used or not. (1 mark)

Question 8 Sharing state between threads

13 MARKS

(a)

3 MARKS

The three conditions, all of which must hold:

(1.5 marks)

1. Two or more threads access the same memory location.
2. At least one of those accesses is a **write**.
3. Nothing synchronizes them.

What the standard says: the behavior of the **whole program** is undefined. (0.5 marks)

Why that is stronger than “the value may be wrong.” Undefined behavior does not promise you a wrong value. The compiler is entitled to assume no race exists and to optimize on that basis: it may cache the variable in a register, hoist a load out of a loop, reorder the two writes, or read the variable twice and get different answers within one expression. The failure therefore need not look anything like a torn value, and the code that breaks may be nowhere near the racing access. It also means the program can pass every test at -O0 and fail in the field at -O2 — “it worked when we tested it” is not evidence of anything. (1 mark)

(b)

4 MARKS

Is the data race gone? Yes. Every shared access is now an atomic operation, so condition (3) of part (a) no longer holds and the program has no undefined behavior from this cause.

(1 mark)

Is the program correct? No. Atomicity per *variable* is not atomicity per *transaction*. The two fields have to change together, and nothing makes the pair of writes indivisible.

(1 mark)

An interleaving:

(1 mark)

TX writes data and then newData, exactly as it did under the mutex:

```
TX: data.store(1); newData.store(true)  <- message 1
RX: newData.load() == true              <- RX sees message 1
RX: data.load() == 1                   <- and consumes it
TX: data.store(2); newData.store(true)  <- message 2 is announced
RX: newData.store(false)                <- RX clears the flag: message 2 is lost
```

RX's test of newData and its clearing of it are two separate atomic operations, and TX can run between them. Let TX write data for message 2 between RX's two loads instead, and announce it after RX has cleared the flag, and RX prints message 2 twice and message 1 never. Each individual access is atomic; the check-then-act sequence is not.

Two things a mutex provides that a pair of atomics does not:

(1 mark)

1. **Mutual exclusion over a region**, not over a single access. Several statements run without another thread interleaving between them, which is exactly what a multi-field update needs.

2. **Memory synchronization between threads.** Everything written inside a critical section is guaranteed visible to the next thread that acquires the same mutex, and not only the fields you remembered to make atomic.

(c)

3 MARKS

std::launch::async — the task runs immediately, in a new thread. **std::launch::deferred** — no thread is created; the task runs synchronously on the **calling** thread, the first time `get()` or `wait()` is called on the future. (1 mark)

With no policy passed, the implementation is free to choose either, and nothing in the source says which you got. (0.5 marks)

The failure. Code written as “start the checksum, print the frame, then collect the result” silently becomes fully sequential when the implementation picks deferred: the frame is printed first, and only then does the checksum begin — on the same thread, inside `get()`. Nothing crashes and nothing warns; the concurrency you believed you had is simply absent, and the measurement that was supposed to show an improvement shows none. Worse, if `get()` is never called, a deferred task never runs at all. The fix is to always pass the policy explicitly. (1.5 marks)

Worth knowing, not required: the destructor of a future returned by `std::async` with the `async policy` blocks until the task completes, so discarding the future also turns the call back into a synchronous one.

(d)

3 MARKS

The sequence, with three threads:

(1.5 marks)

1. The **low**-priority thread locks a mutex and begins a long operation.
2. The **high**-priority thread wakes, tries to lock the same mutex, and blocks.
3. The **medium**-priority thread wakes and preempts the low-priority thread. It does not use the mutex; it is doing entirely unrelated work. The scheduler simply gives the CPU to the higher-priority runnable thread, which is a normal and correct scheduling decision.
4. The medium-priority thread runs on, so the low-priority thread cannot finish and cannot release the lock.
5. The high-priority thread stays blocked — not on its own work, and not on anything the medium thread holds, but because the medium thread is delaying the release.

The high-priority task ends up waiting on the medium-priority one. The intended scheduling order has been inverted, and the highest-priority thread — often the one with the deadline, resetting the watchdog — is the one that misses it.

Why `std::mutex` cannot prevent it. It has no **priority inheritance**: no mechanism to temporarily raise the lock holder’s priority to that of the highest-priority thread waiting on it, so that the holder can finish and release before anything of middling priority cuts in. The preemption in step 3 is a scheduler decision the mutex plays no part in. `std::thread` compounds this by exposing no notion of priority at all — setting one means going through

`native_handle()` and the platform API — so the standard library gives you nothing to work with here. (1 mark)

What does fix it: the platform's priority-inheritance mutex — an RTOS mutex configured for priority inheritance, or a POSIX mutex with the `PTHREAD_PRIO_INHERIT` protocol. If you are mixing thread priorities on an RTOS, that is what belongs in the code; `std::thread` itself is not the problem. (0.5 marks)